



คณะเทคโนโลยีสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

Chapter 05 Artificial Neural Networks (ANN) – Part 2

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรรยาโรจน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

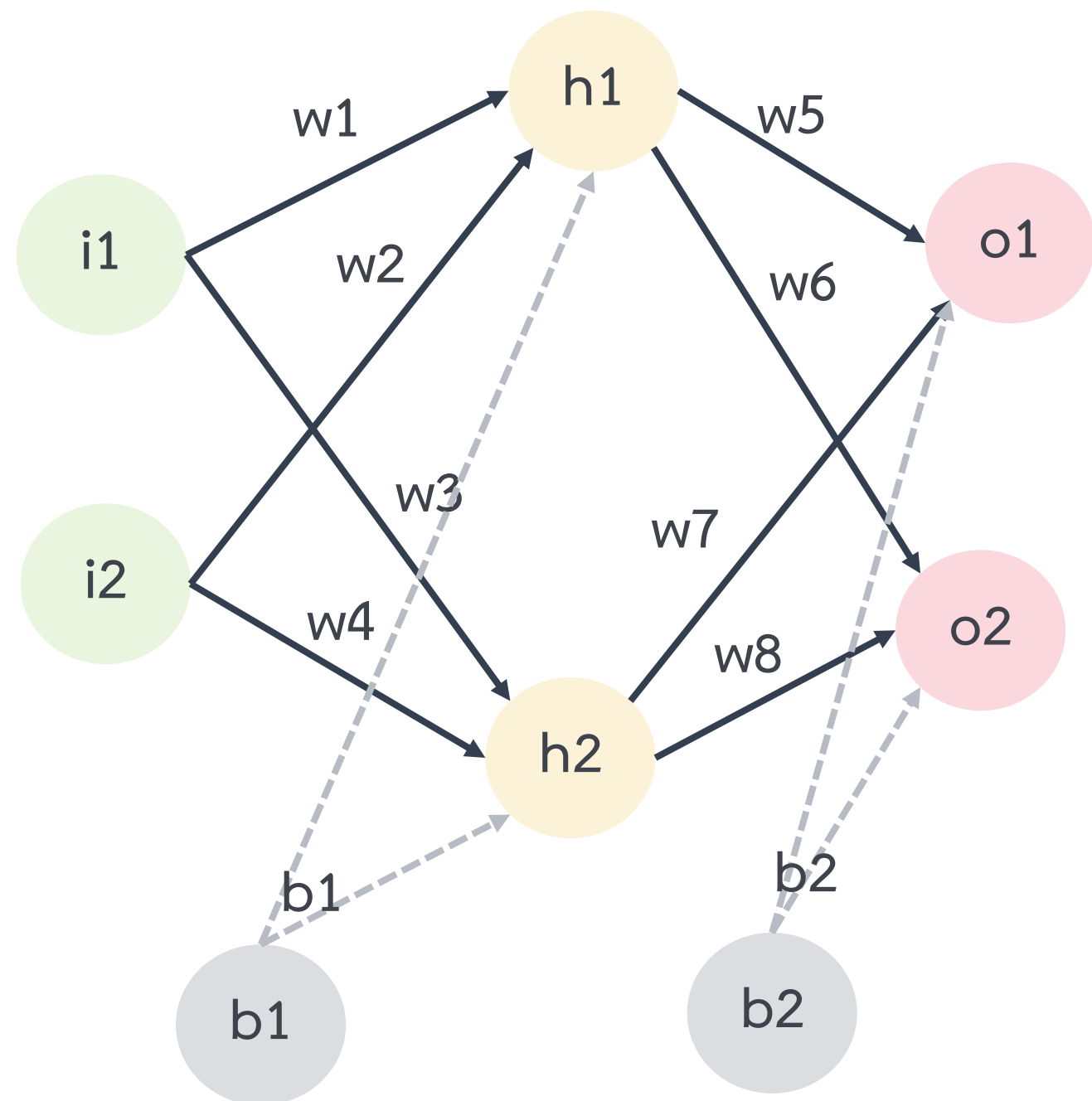
วิธีการแบ่งกลุ่มข้อมูลพื้นฐาน



- การวัดความคล้ายโดยอาศัยระยะทาง (Distance)
 - Manhattan distance (also called 1-norm)
 - Euclidean distance (also called 2-norm)
- การวัดความคล้ายโดยอาศัยองศา (Degree)
- โครงข่ายประสาทเทียม (Artificial neural networks: ANN)

โครงข่ายประสาทเทียม

Forward



Forward Propagation คือ กระบวนการที่ป้อน Input เข้าสู่แบบจำลองที่ฝึกสอนเรียบร้อยแล้ว และคืนค่าเป็นความน่าจะเป็น (Normalized Probabilities) ของแต่ละคลาส จากนั้นจะนำค่าดังกล่าวมาใช้ทำนายว่า Input ควรเป็นคลาสใด

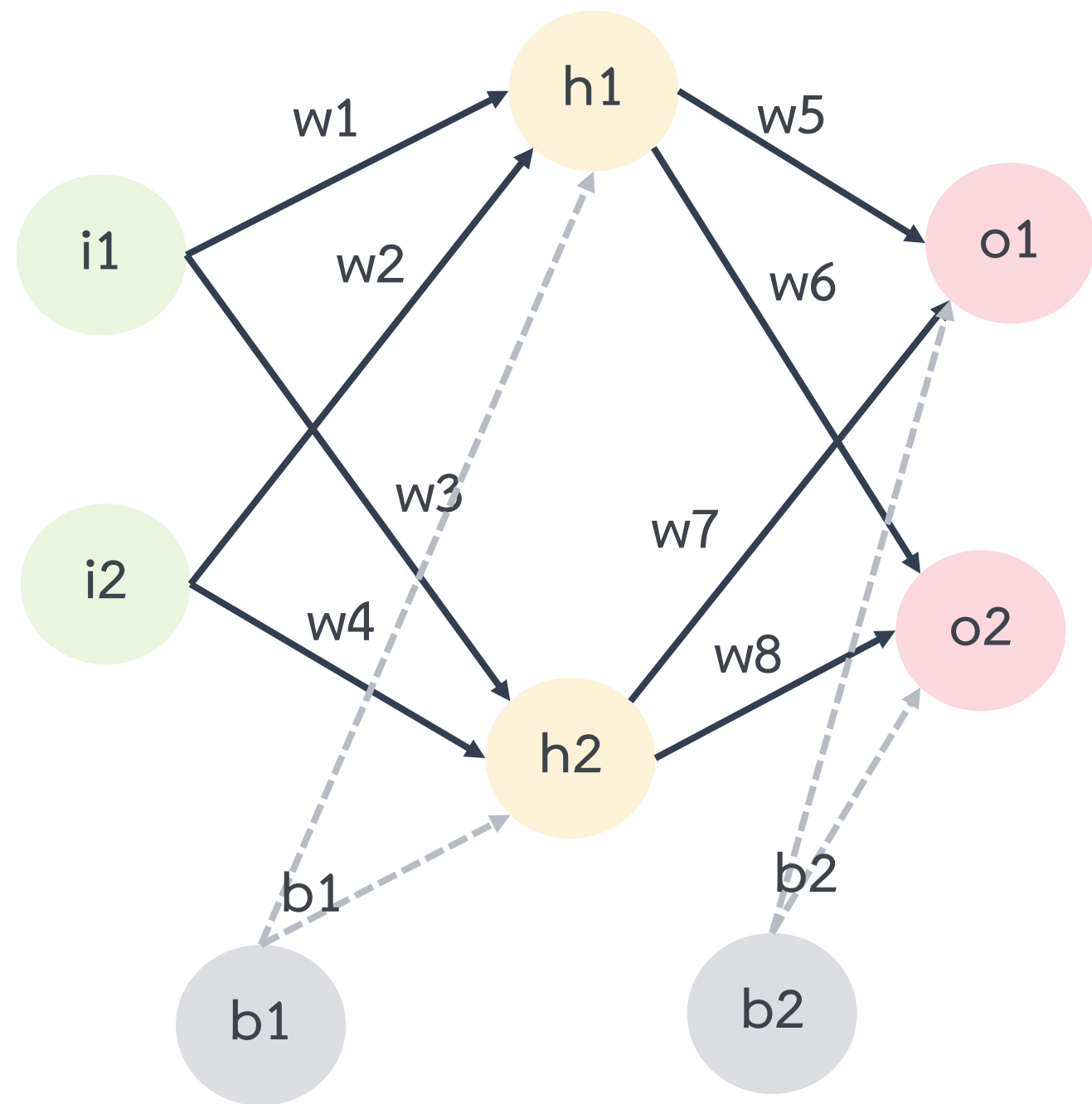
Backward Propagation คือ กระบวนการที่นำค่า Normalized Probabilities ไปใช้เพื่อปรับปรุง Weight ของแบบจำลองให้เกิดความแม่นยำมากยิ่งขึ้น

Backward

$$\begin{array}{ccccc}
 Y & = & W & \cdot & X \\
 \text{output} & & \text{weight} & & \text{input}
 \end{array}$$

โครงข่ายประสาทเทียม

Forward



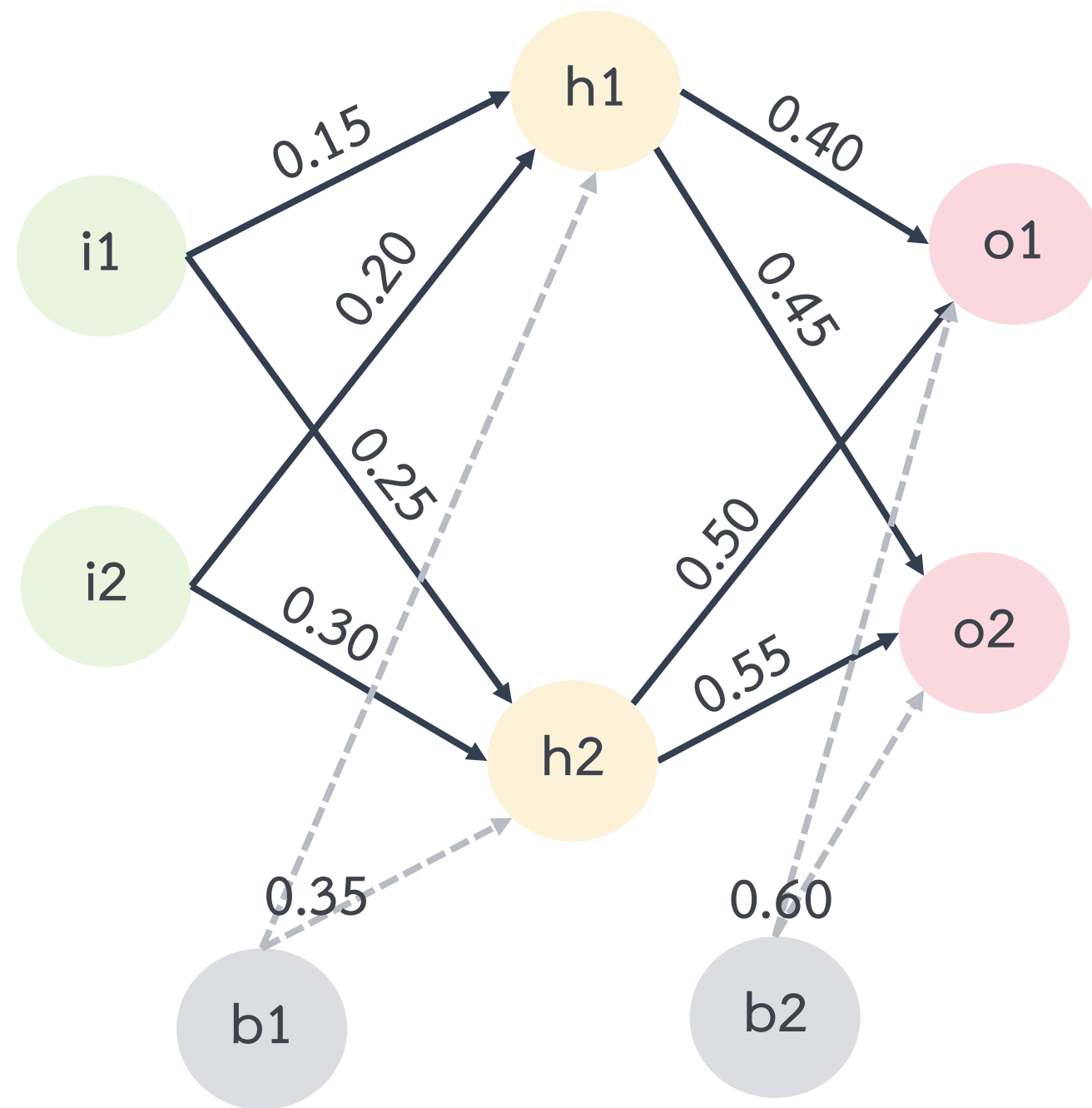
Forward Propagation คือ กระบวนการที่ป้อน Input เข้าสู่แบบจำลองที่ฝึกสอนเรียบร้อยแล้ว และคืนค่าเป็นความน่าจะเป็น (Normalized Probabilities) ของแต่ละคลาส จากนั้นจะนำค่าดังกล่าวมาใช้นำทายว่า Input ควรเป็นคลาสใด

Backward Propagation คือ กระบวนการที่นำค่า Normalized Probabilities ไปใช้เพื่อปรับปรุง Weight ของแบบจำลองให้เกิดความแม่นยำมากยิ่งขึ้น

Backward

$$\begin{array}{ccccc}
 ? & = & W & \cdot & X \\
 \text{output} & & \text{weight} & & \text{input}
 \end{array}$$

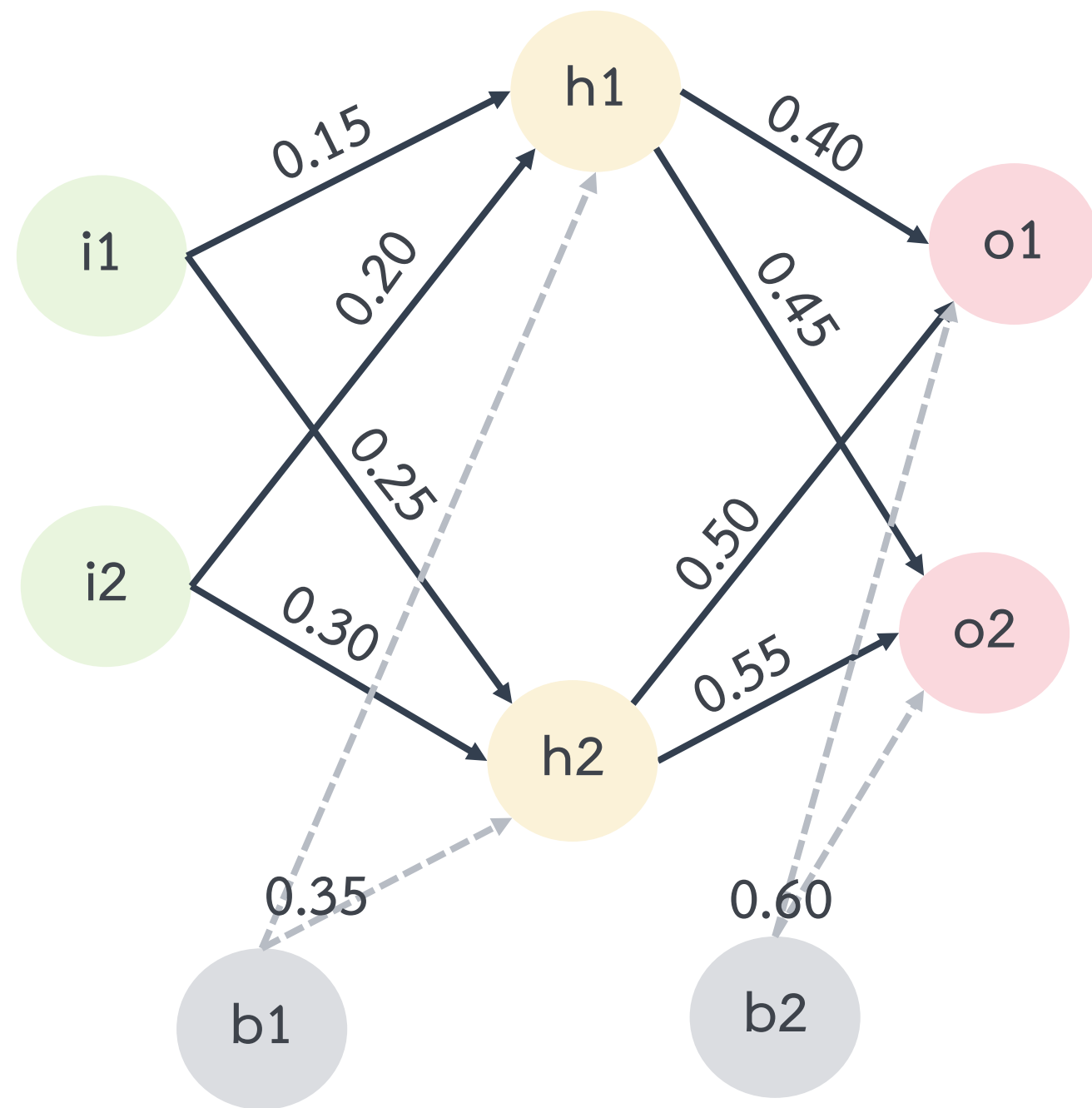
โครงข่ายประสาทเทียม



ตัวอย่างที่ 1 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น Linear และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

$input = \begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$ และ $expected\ output = \begin{bmatrix} 0.01 \\ 0.99 \end{bmatrix}$

โครงข่ายประสาทเทียม



ตัวอย่างที่ 1 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น Linear และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

วิธีทำ

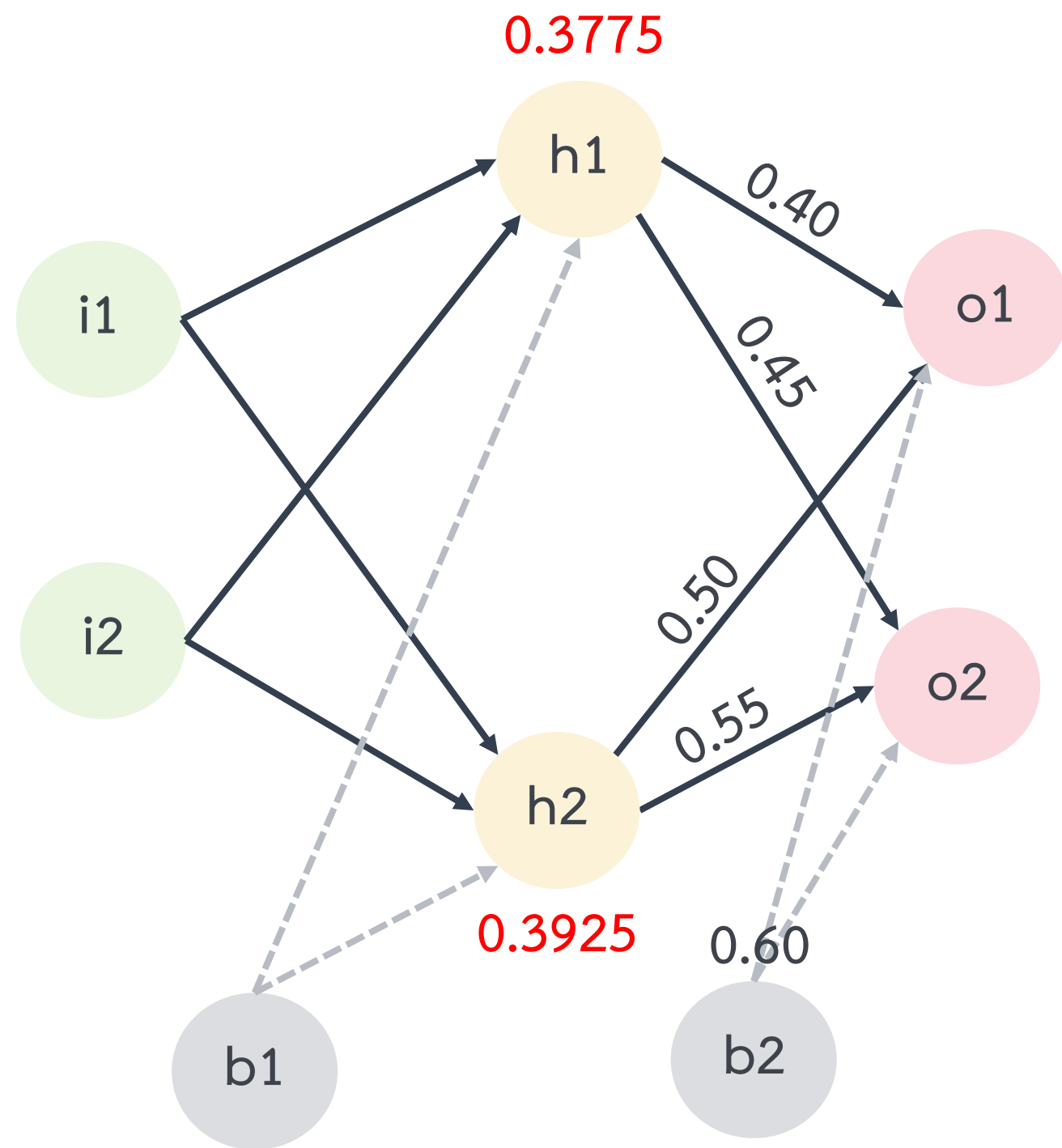
พิจารณาที่ (h1) จะพบว่าผลลัพธ์ที่ออกจาก (h1) จะเรียกว่า y1 สามารถคำนวณได้ดังนี้

$$\begin{aligned}
 y1 &= f((0.05)(0.15) + (0.10)(0.20) + 0.35) \quad \% x = f(x) \text{ เป็นรูปแบบ Linear (ไม่ทำ Activation)} \\
 &= 0.0075 + 0.02 + 0.35 = 0.3775
 \end{aligned}$$

พิจารณาที่ (h2) จะพบว่าผลลัพธ์ที่ออกจาก (h2) จะเรียกว่า y2 สามารถคำนวณได้ดังนี้

$$\begin{aligned}
 y2 &= f((0.05)(0.25) + (0.10)(0.30) + 0.35) \quad \% x = f(x) \text{ เป็นรูปแบบ Linear (ไม่ทำ Activation)} \\
 &= 0.0125 + 0.03 + 0.35 = 0.3925
 \end{aligned}$$

โครงข่ายประสาทเทียม



ตัวอย่างที่ 1 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น Linear และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

วิธีทำ

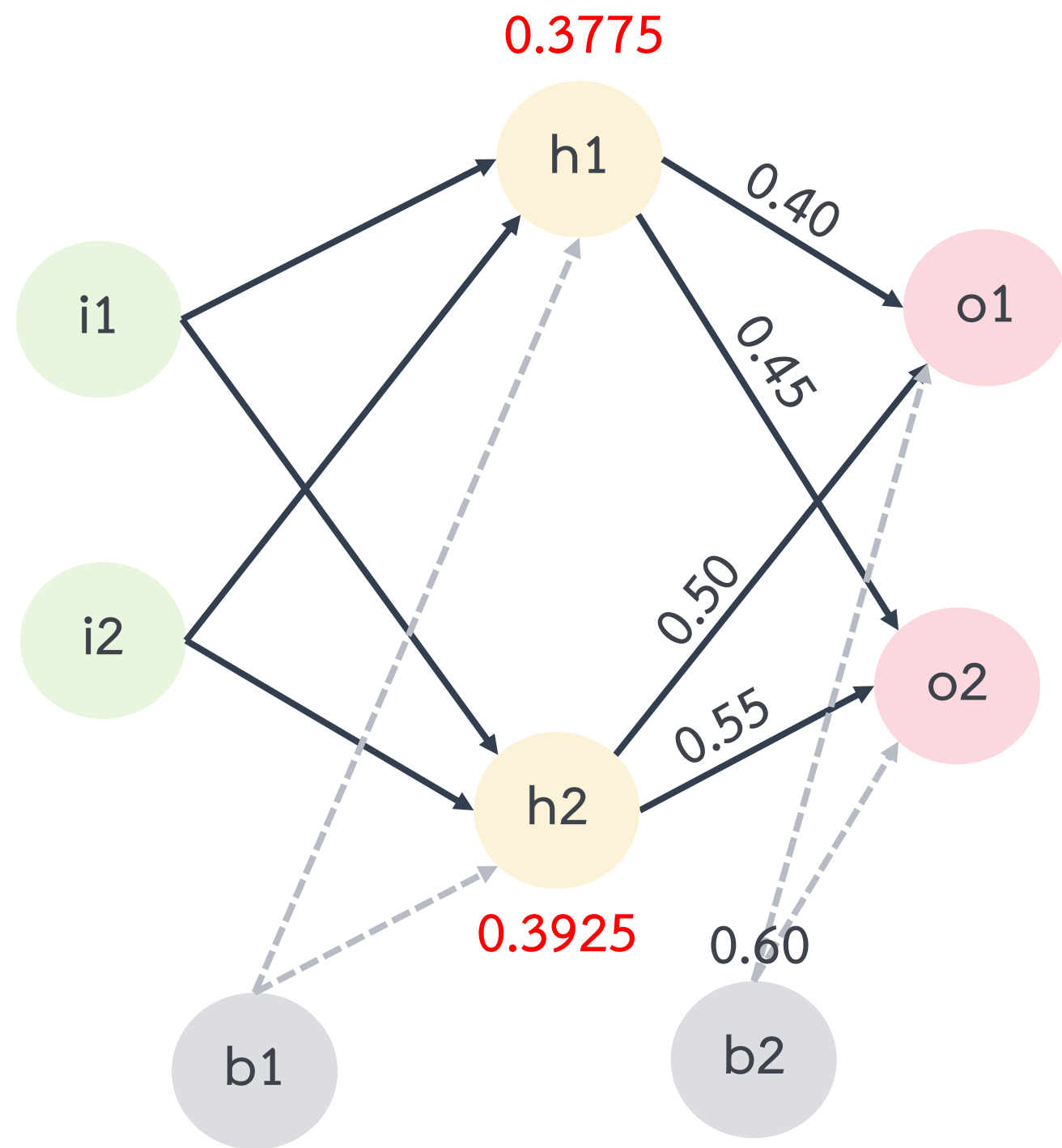
พิจารณาที่ (h1) จะพบว่าผลลัพธ์ที่ออกจาก (h1) จะเรียกว่า y1 สามารถคำนวณได้ดังนี้

$$\begin{aligned}
 y1 &= f((0.05)(0.15) + (0.10)(0.20) + 0.35) \quad \% x = f(x) \text{ เป็นรูปแบบ Linear (ไม่ทำ Activation)} \\
 &= 0.0075 + 0.02 + 0.35 = \mathbf{0.3775}
 \end{aligned}$$

พิจารณาที่ (h2) จะพบว่าผลลัพธ์ที่ออกจาก (h2) จะเรียกว่า y2 สามารถคำนวณได้ดังนี้

$$\begin{aligned}
 y2 &= f((0.05)(0.25) + (0.10)(0.30) + 0.35) \quad \% x = f(x) \text{ เป็นรูปแบบ Linear (ไม่ทำ Activation)} \\
 &= 0.0125 + 0.03 + 0.35 = \mathbf{0.3925}
 \end{aligned}$$

โครงข่ายประสาทเทียม



ตัวอย่างที่ 1 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น Linear และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

วิธีทำ

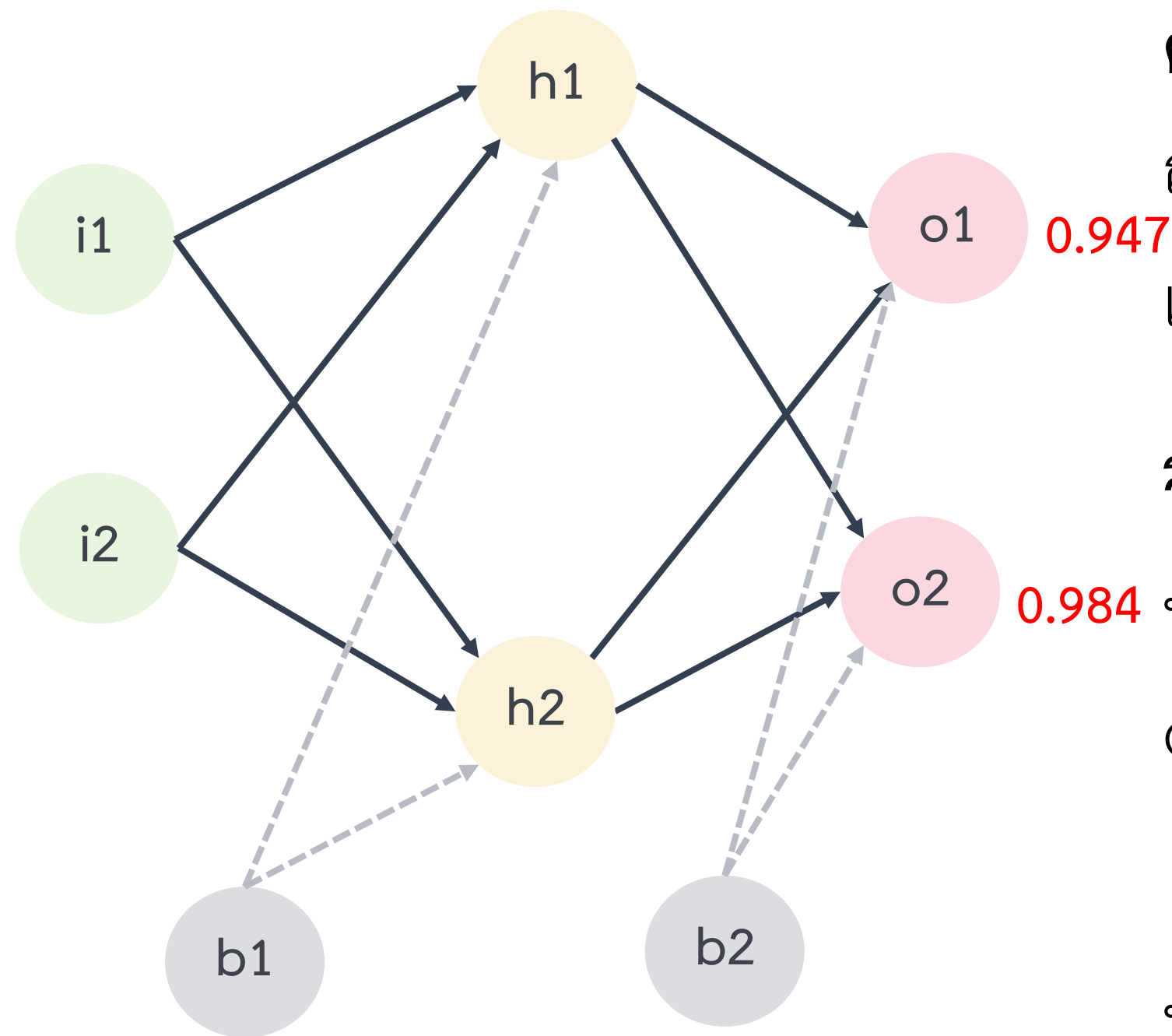
พิจารณาที่ (o1) จะพบว่าผลลัพธ์ที่ออกจาก (o1) สามารถคำนวณได้ดังนี้

$$\begin{aligned}
 o1 &= (0.3775)(0.40) + (0.3925)(0.50) + 0.60 \\
 &= 0.151 + 0.196 + 0.60 = 0.947
 \end{aligned}$$

พิจารณาที่ (o2) จะพบว่าผลลัพธ์ที่ออกจาก (o2) สามารถคำนวณได้ดังนี้

$$\begin{aligned}
 o2 &= (0.3775)(0.45) + (0.3925)(0.55) + 0.60 \\
 &= 0.169 + 0.215 + 0.60 = 0.984
 \end{aligned}$$

โครงข่ายประสาทเทียม



ตัวอย่างที่ 1 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น Linear และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

วิธีทำ

พิจารณาที่ (o1) จะพบว่าผลลัพธ์ที่ออกจาก (o1) สามารถคำนวณได้ดังนี้

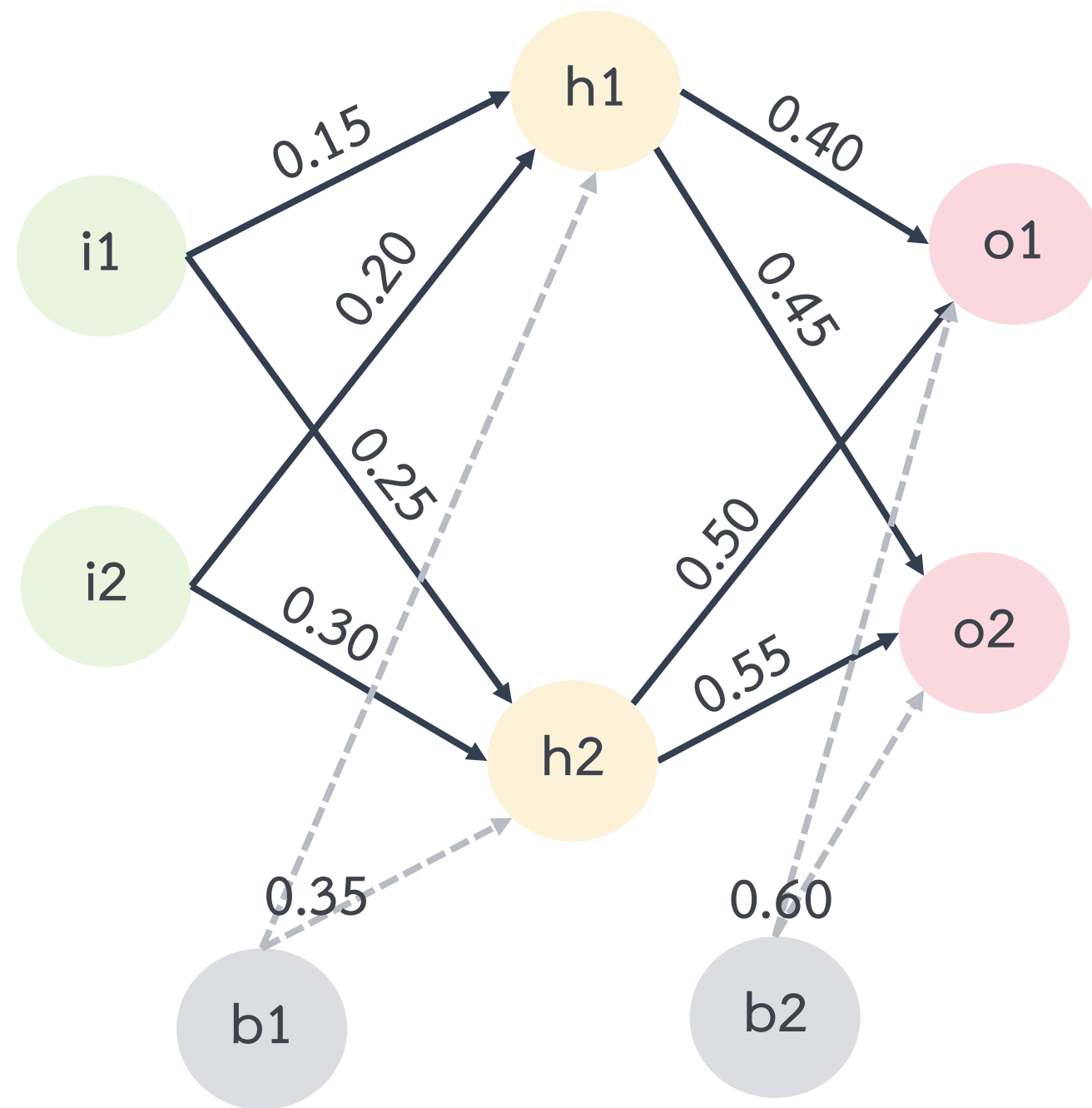
$$\begin{aligned}
 o1 &= (0.3775)(0.40) + (0.3925)(0.50) + 0.60 \\
 &= 0.151 + 0.196 + 0.60 = 0.947
 \end{aligned}$$

พิจารณาที่ (o2) จะพบว่าผลลัพธ์ที่ออกจาก (o2) สามารถคำนวณได้ดังนี้

$$\begin{aligned}
 o2 &= (0.3775)(0.45) + (0.3925)(0.55) + 0.60 \\
 &= 0.169 + 0.215 + 0.60 = 0.984
 \end{aligned}$$

$\therefore$ เมื่อนำ $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$ ไปทำนายจะได้ผลลัพธ์เป็นคลาสที่ 2 เนื่องจากค่าของ o2 มากกว่า o1

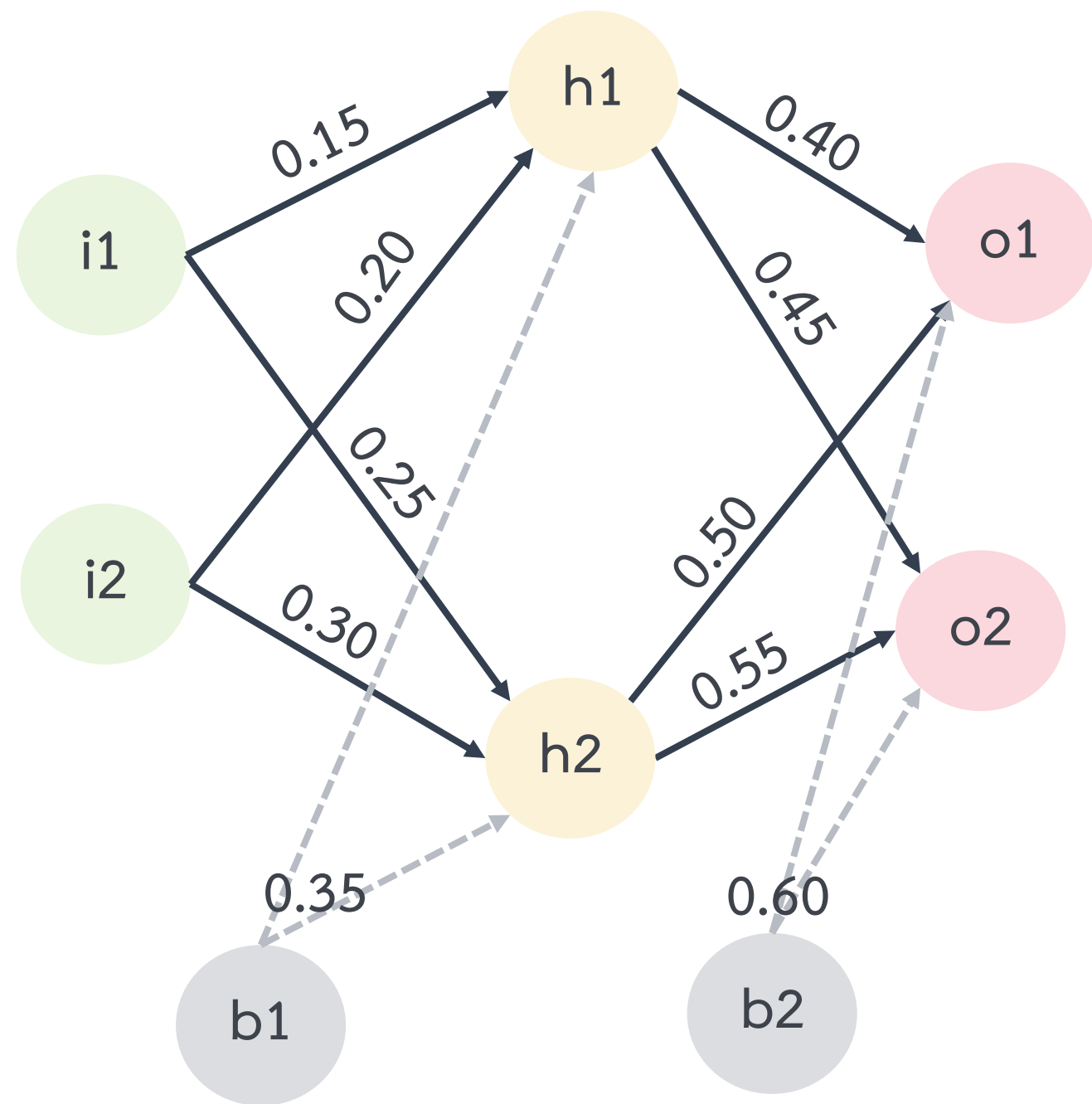
โครงข่ายประสาทเทียม



ตัวอย่างที่ 2 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น *Logistic function* และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

$input = \begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$ และ $expected\ output = \begin{bmatrix} 0.01 \\ 0.99 \end{bmatrix}$

โครงข่ายประสาทเทียม



ตัวอย่างที่ 2 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น **Logistic function** และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

วิธีทำ

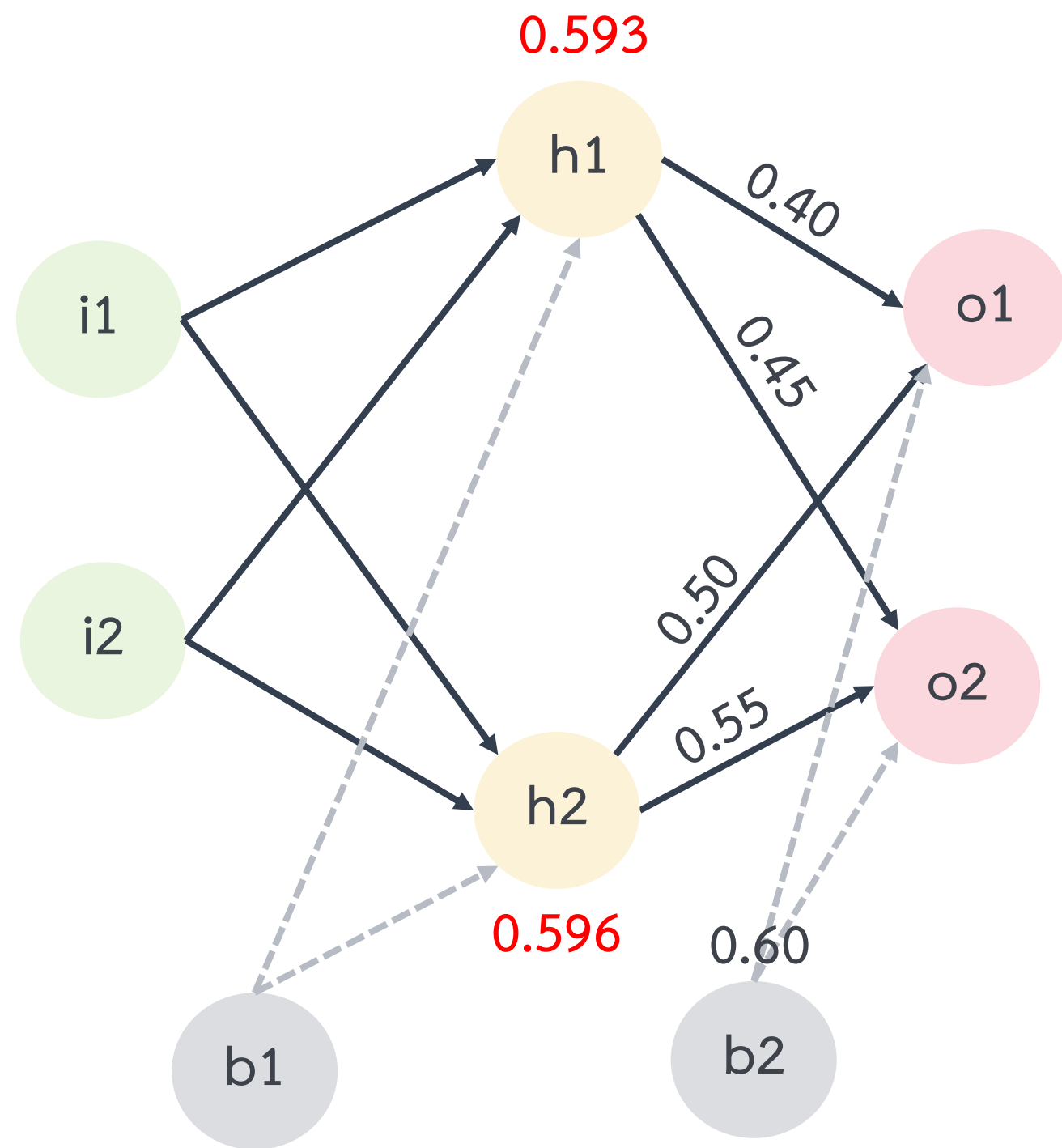
พิจารณาที่ (h1) จะพบว่าผลลัพธ์ที่ออกจาก (h1) จะเรียกว่า y1 สามารถคำนวณได้ดังนี้
 $y1 = f((0.05)(0.15) + (0.10)(0.20) + 0.35) = f(0.0075 + 0.02 + 0.35) = f(0.3775)$

$$\begin{aligned}
 \% \text{ out} = f(in) &= \frac{1}{1+e^{-in}} \text{ เป็นรูปแบบ Logistic function (ทำ Activation)} \\
 &= \frac{1}{1+e^{-(0.3775)}} = 0.593
 \end{aligned}$$

พิจารณาที่ (h2) จะพบว่าผลลัพธ์ที่ออกจาก (h2) จะเรียกว่า y2 สามารถคำนวณได้ดังนี้
 $y2 = f((0.05)(0.25) + (0.10)(0.30) + 0.35) = f(0.0125 + 0.03 + 0.35) = f(0.3925)$

$$= \frac{1}{1+e^{-(0.3925)}} = 0.596$$

โครงข่ายประสาทเทียม



ตัวอย่างที่ 2 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น **Logistic function** และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

วิธีทำ

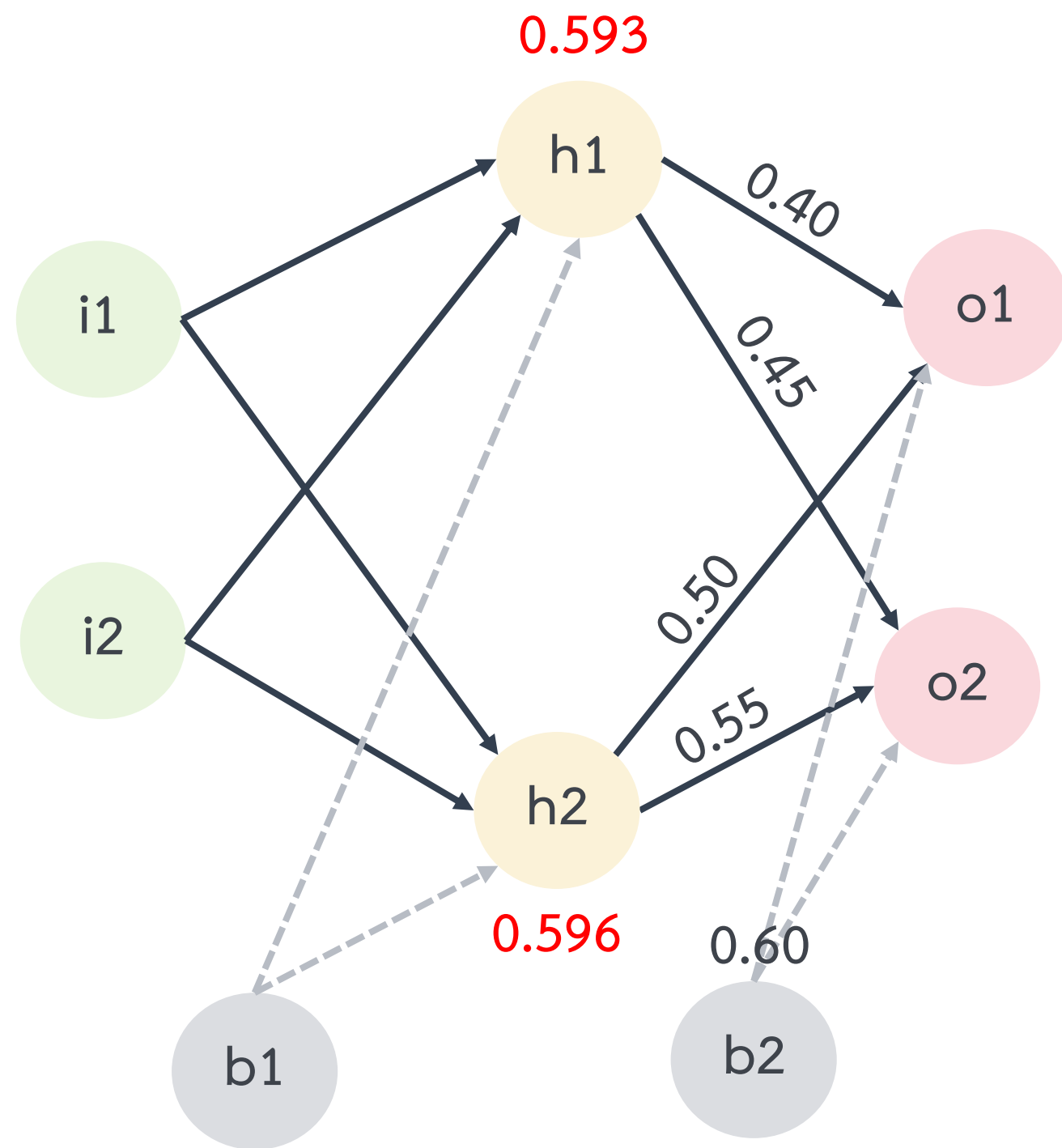
พิจารณาที่ (h1) จะพบว่าผลลัพธ์ที่ออกจาก (h1) จะเรียกว่า y1 สามารถคำนวณได้ดังนี้
 $y1 = f((0.05)(0.15) + (0.10)(0.20) + 0.35) = f(0.0075 + 0.02 + 0.35) = f(0.3775)$

$$\begin{aligned}
 \% \text{ out} = f(in) &= \frac{1}{1+e^{-in}} \text{ เป็นรูปแบบ Logistic function (ทำ Activation)} \\
 &= \frac{1}{1+e^{-(0.3775)}} = 0.593
 \end{aligned}$$

พิจารณาที่ (h2) จะพบว่าผลลัพธ์ที่ออกจาก (h2) จะเรียกว่า y2 สามารถคำนวณได้ดังนี้
 $y2 = f((0.05)(0.25) + (0.10)(0.30) + 0.35) = f(0.0125 + 0.03 + 0.35) = f(0.3925)$

$$= \frac{1}{1+e^{-(0.3925)}} = 0.596$$

โครงข่ายประสาทเทียม



ตัวอย่างที่ 2 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่าถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น *Logistic function* และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

วิธีทำ

พิจารณาที่ (o1) จะพบว่าผลลัพธ์ที่ออกจาก (o1) สามารถคำนวณได้ดังนี้

$$o1 = f((0.593)(0.40) + (0.596)(0.50) + 0.60) = f(1.13)$$

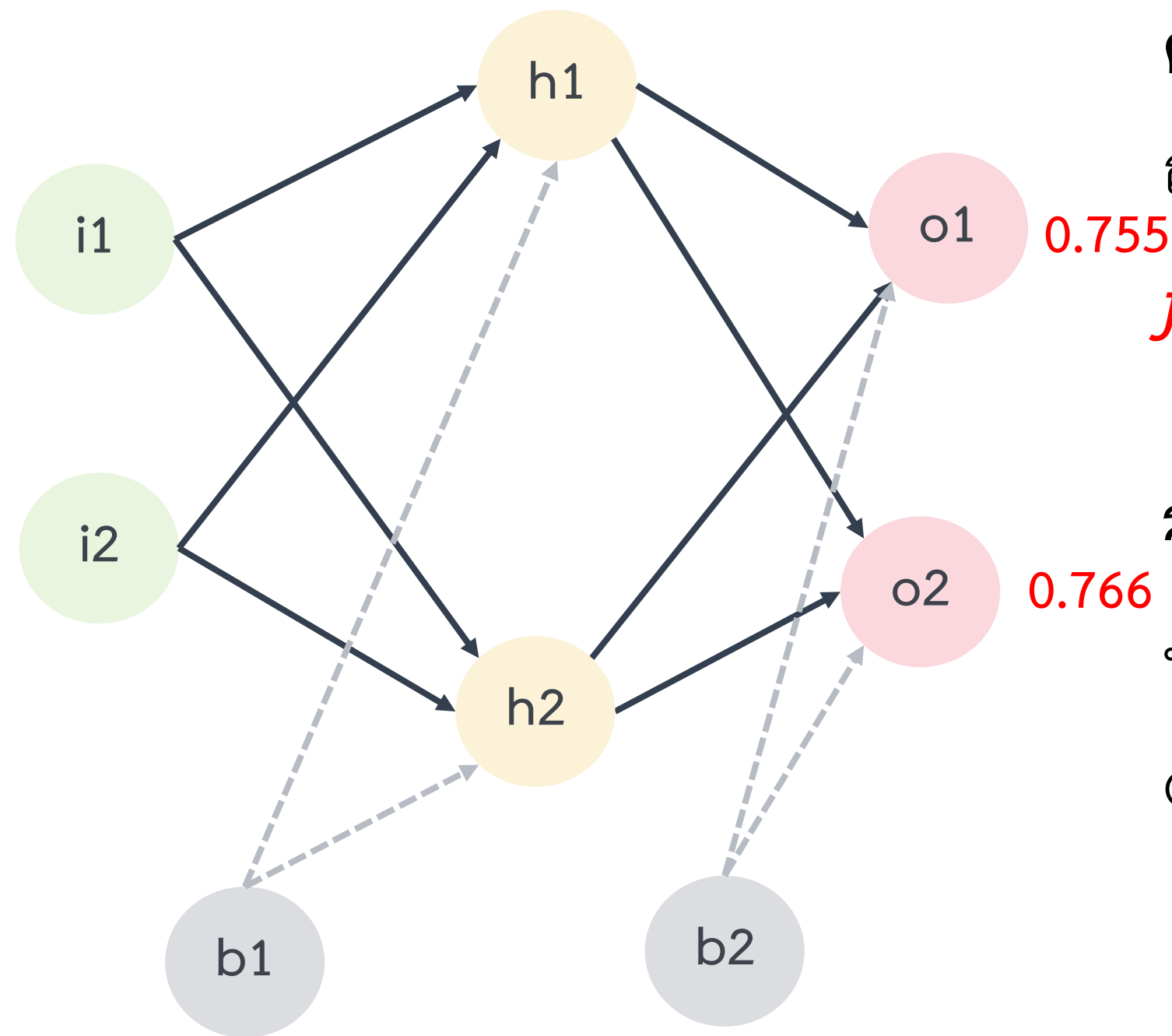
$$= \frac{1}{1+e^{-(1.13)}} = 0.755$$

พิจารณาที่ (o2) จะพบว่าผลลัพธ์ที่ออกจาก (o2) สามารถคำนวณได้ดังนี้

$$o2 = f((0.593)(0.45) + (0.596)(0.55) + 0.60) = f(1.19)$$

$$= \frac{1}{1+e^{-(1.19)}} = 0.766$$

โครงข่ายประสาทเทียม



ตัวอย่างที่ 2 การทำงานแบบ Forward ของโครงข่ายประสาทเทียม โดยกำหนดให้แบบจำลองมีค่า
 ถ่วงน้ำหนักและ bias ดังแสดงในรูป นอกจากนี้ กำหนดให้ Activation Function เป็น *Logistic*
function และ input vector = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$

วิธีทำ

พิจารณาที่ (o1) จะพบว่าผลลัพธ์ที่ออกจาก (o1) สามารถคำนวณได้ดังนี้

$$o1 = f((0.593)(0.40) + (0.596)(0.50) + 0.60) = f(1.13)$$

$$= \frac{1}{1+e^{-(1.13)}} = 0.755$$

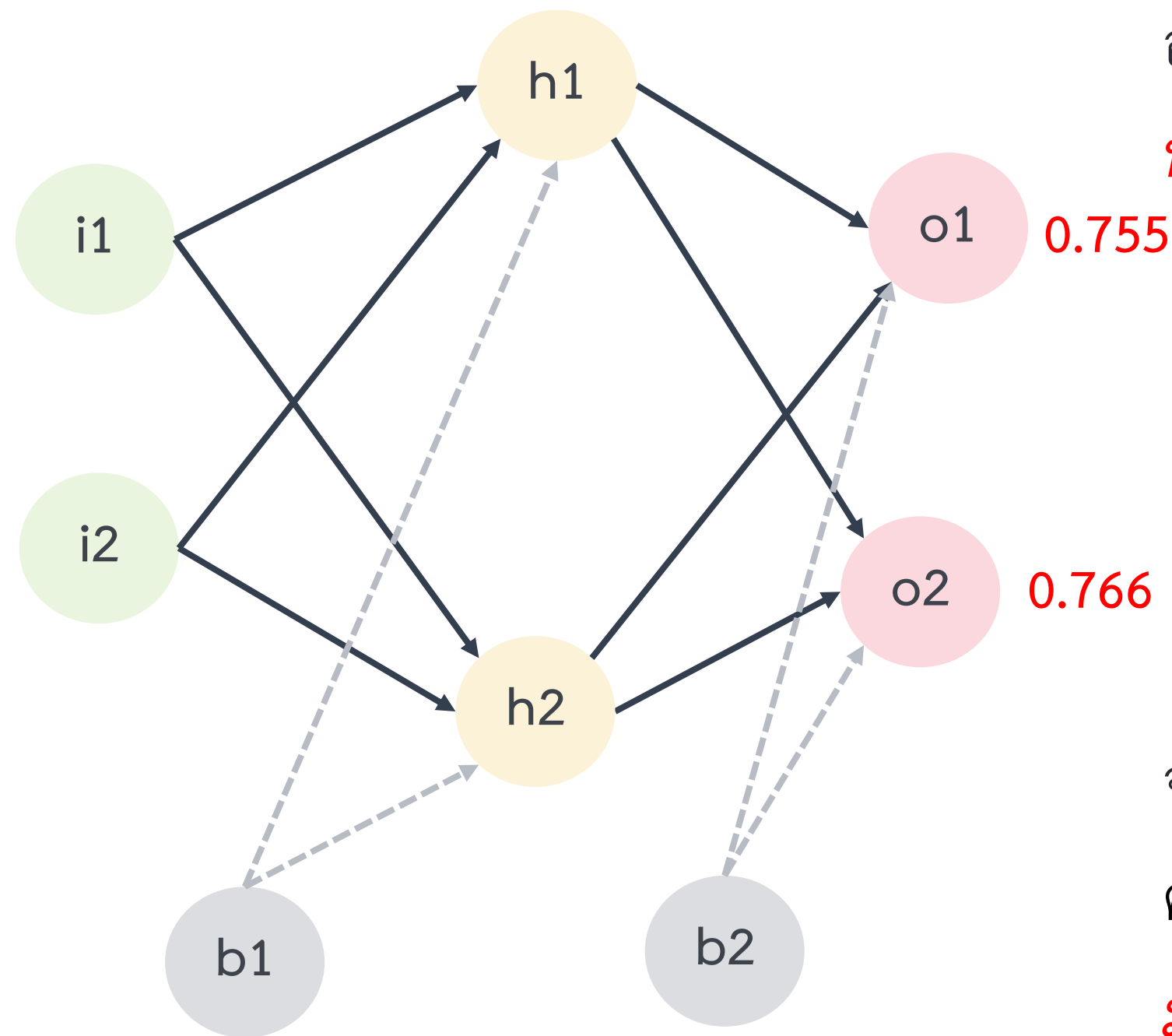
พิจารณาที่ (o2) จะพบว่าผลลัพธ์ที่ออกจาก (o2) สามารถคำนวณได้ดังนี้

$$o2 = f((0.593)(0.45) + (0.596)(0.55) + 0.60) = f(1.19)$$

$$= \frac{1}{1+e^{-(1.19)}} = 0.766$$

$\therefore$ เมื่อนำ $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$ ไปทำนายจะได้ผลลัพธ์เป็นคลาสที่ 2 เนื่องจากค่าของ o2 มากกว่า o1

โครงข่ายประสาทเทียม



อย่างไรก็ตาม การประเมินประสิทธิภาพของโครงข่ายประสาทเทียมจะพิจารณาจาก 2 ปัจจัย ได้แก่ (1) **ผลทำนาย** ถูกต้องหรือไม่ และ (2) **ค่าความมั่นใจ** ของแบบจำลองเป็นอย่างไร อาทิเช่น

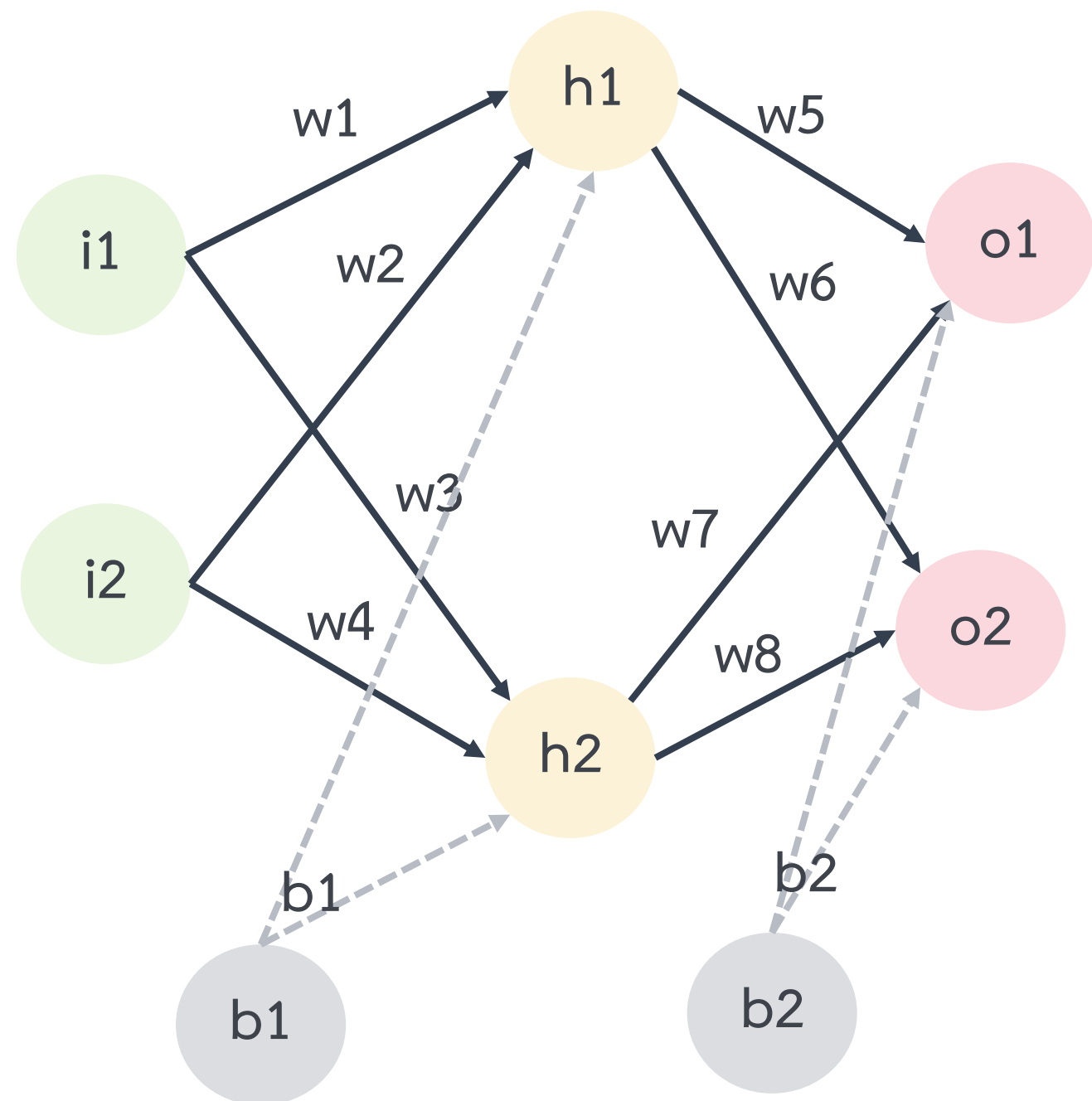
- กรณีที่ 1 : ทำนายถูก และมีความมั่นใจมาก
- กรณีที่ 2 : ทำนายถูก และมีความมั่นใจน้อย
- กรณีที่ 3 : ทำนายไม่ถูก และมีความมั่นใจมาก

จากตัวอย่างที่ 2 จะพบว่าเมื่อป้อน $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$ ไปทำนายจะได้ผลลัพธ์เป็น $\begin{bmatrix} 0.755 \\ 0.766 \end{bmatrix}$ ซึ่งสรุปได้ว่าผลลัพธ์เป็นคลาสที่ 2 เนื่องจากค่าของ o2 มากกว่า o1 **ถึงแม้ว่าผลการทำนายจะถูกต้อง แต่แบบจำลองไม่มีความมั่นใจ** เนื่องจากค่า o1 และ o2 ใกล้กันมากและยังห่างไกลจากผลลัพธ์ที่คาดหวัง $\begin{bmatrix} 0.01 \\ 0.99 \end{bmatrix}$

input = $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$ และ expected output = $\begin{bmatrix} 0.01 \\ 0.99 \end{bmatrix}$

โครงข่ายประสาทเทียม

Forward



Forward Propagation คือ กระบวนการที่ป้อน Input เข้าสู่แบบจำลองที่ฝึกสอนเรียบร้อยแล้ว และคืนค่าเป็นความน่าจะเป็น (Normalized Probabilities) ของแต่ละคลาส จากนั้นจะนำค่าดังกล่าวมาใช้นำทายว่า Input ควรเป็นคลาสใด

Backward Propagation คือ กระบวนการที่นำค่า Normalized Probabilities ไปใช้เพื่อปรับปรุง Weight ของแบบจำลองให้เกิดความแม่นยำมากยิ่งขึ้น

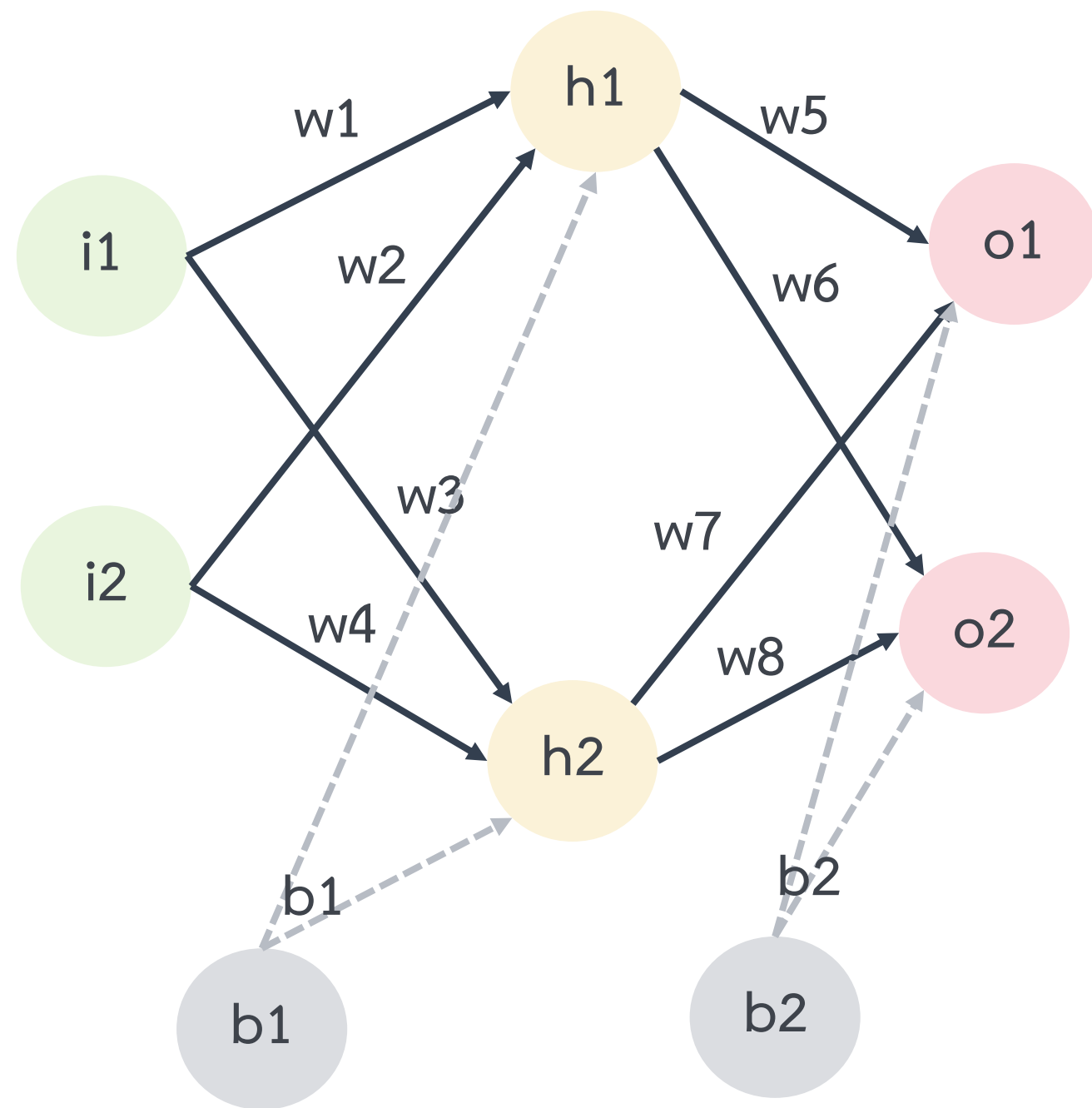
Backward

$$Y = W \cdot X$$

output weight input

โครงข่ายประสาทเทียม

Forward



Forward Propagation คือ กระบวนการที่ป้อน Input เข้าสู่แบบจำลองที่ฝึกสอนเรียบร้อยแล้ว และคืนค่าเป็นความน่าจะเป็น (Normalized Probabilities) ของแต่ละคลาส จากนั้นจะนำค่าดังกล่าวมาใช้นำทายว่า Input ควรเป็นคลาสใด

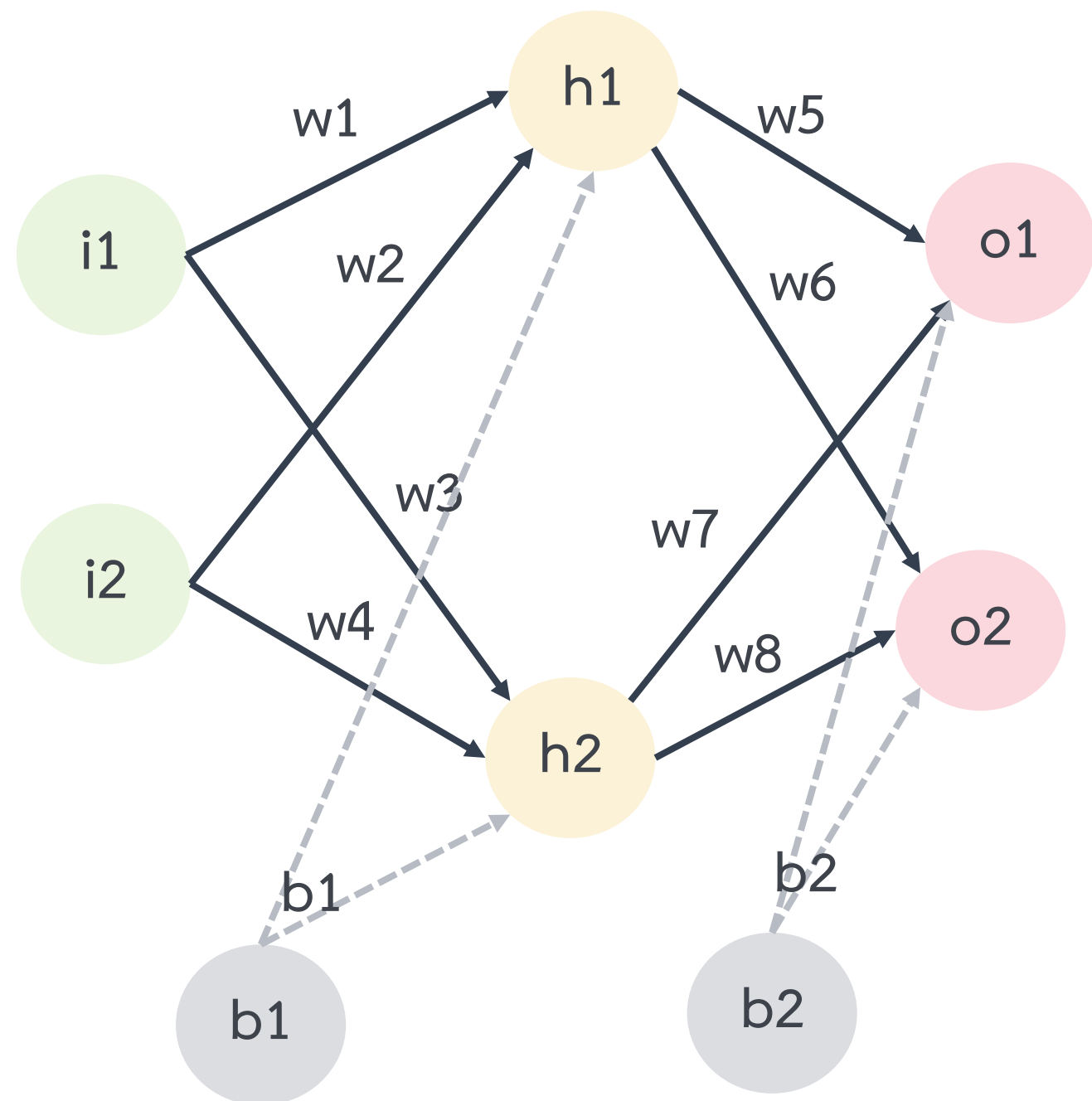
Backward Propagation คือ กระบวนการที่นำค่า Normalized Probabilities ไปใช้เพื่อปรับปรุง Weight ของแบบจำลองให้เกิดความแม่นยำมากยิ่งขึ้น

Backward

$$\begin{array}{ccccc}
 ? & = & W & \cdot & X \\
 \text{output} & & \text{weight} & & \text{input}
 \end{array}$$

โครงข่ายประสาทเทียม

Forward



Forward Propagation คือ กระบวนการที่ป้อน Input เข้าสู่แบบจำลองที่ฝึกสอนเรียบร้อยแล้ว และคืนค่าเป็นความน่าจะเป็น (Normalized Probabilities) ของแต่ละคลาส จากนั้นจะนำค่าดังกล่าวมาใช้ทำนายว่า Input ควรเป็นคลาสใด

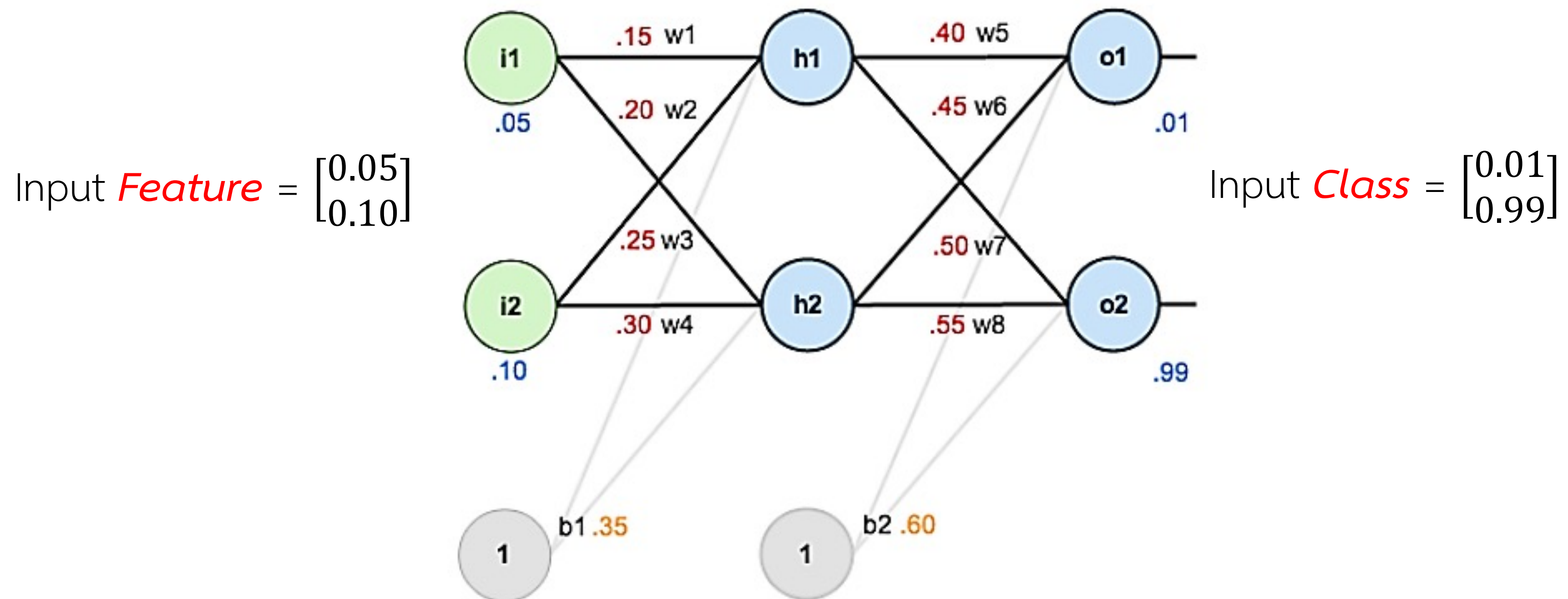
Backward Propagation คือ กระบวนการที่นำค่า Normalized Probabilities ไปใช้เพื่อปรับปรุง Weight ของแบบจำลองให้เกิดความแม่นยำมากยิ่งขึ้น

Backward

$$\begin{array}{ccccc}
 Y & = & ? & \cdot & X \\
 \text{output} & & \text{weight} & & \text{input}
 \end{array}$$

Back Propagation

Forward : ใช้อินพุตเพื่อทำนายคลาส



Backward : นำค่าความผิดพลาดไปใช้เพื่อปรับค่าถ่วงน้ำหนัก

องค์ประกอบที่ควรรู้สำหรับการฝึกสอนแบบจำลอง



- Loss Function (Objective or Cost Function)
- Optimization
- Learning Rate

องค์ประกอบที่ควรรู้สำหรับการฝึกสอนแบบจำลอง



- Loss Function (Objective or Cost Function)
- Optimization
- Learning Rate



Loss Function (Objective or Cost Function)

คือ สมการผลรวมของค่าความแตกต่าง (Difference) หรือค่าความผิดพลาด (Error) ระหว่างค่าที่ได้จากแบบจำลองกับค่าจากข้อมูลจริง ซึ่งนิยมนำมาใช้เพื่อวัดประสิทธิภาพของแบบจำลองว่ามีความเหมาะสมกับข้อมูลหรือไม่ โดยทั่วไปแล้วค่าของสมการยิ่งน้อยยิ่งดี

Loss Functions จะสะท้อนถึงประสิทธิภาพของแบบจำลอง หรือกล่าวได้ว่า “พารามิเตอร์แต่ละตัวของแบบจำลองมีความเหมาะสมกับข้อมูลหรือไม่” ซึ่ง Loss Functions จะรับค่าจาก output layer มาคำนวณเพื่อดูว่า (1) **ผลทำนาย** ถูกต้องหรือไม่ และ (2) **ค่าความมั่นใจ** ของแบบจำลองเป็นอย่างไร เช่น

- กรณีที่ 1 : ทำนายถูก และมีความมั่นใจมาก ซึ่งจะสามารถเรียกได้ว่า ดีและจะให้ค่า Loss ที่น้อย
- กรณีที่ 2 : ทำนายถูก และมีความมั่นใจน้อย ซึ่งจะสามารถเรียกได้ว่า ไม่ดีและจะให้ค่า Loss ที่มาก
- กรณีที่ 3 : ทำนายไม่ถูก และมีความมั่นใจมาก ซึ่งจะสามารถเรียกได้ว่า ไม่ดีและจะให้ค่า Loss ที่มาก

ในตัวอย่างในบทนี้อาศัย *Squared Error Function*

$$E_{total} = \frac{1}{2} (target - output)^2$$

จากตัวอย่างที่ 2 จะพบว่าเมื่อป้อน $\begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix}$ ไปทำนายจะได้ผลลัพธ์เป็น $\begin{bmatrix} 0.755 \\ 0.766 \end{bmatrix}$ ซึ่งสรุปได้ว่าผลลัพธ์เป็นคลาสที่ 2 เนื่องจากค่าของ o2 มากกว่า o1 **ถึงแม้ว่าผลการทำนายจะถูกต้อง แต่แบบจำลองไม่มีความมั่นใจ** เนื่องจากค่า o1 และ o2 ใกล้กันมากและยังห่างไกลจากผลลัพธ์ที่คาดหวัง $\begin{bmatrix} 0.01 \\ 0.99 \end{bmatrix}$

Loss Function (Objective or Cost Function)

Loss Functions คือ ฟังก์ชันสำหรับคำนวณหาความแตกต่างระหว่างผลลัพธ์ของโครงข่ายกับผลลัพธ์ที่คาดหวัง เพื่อใช้เป็นวิธีการประเมินประสิทธิภาพของโครงข่ายประสาทเทียม ซึ่งสามารถแบ่งได้เป็น 2 กลุ่ม (1) สำหรับงานทางด้าน Classification จะได้ผลลัพธ์เป็นค่าไม่ต่อเนื่อง (discrete values) อาทิเช่น 0,1,2... เป็นต้น และ (2) สำหรับงานทางด้าน regression จะได้ผลลัพธ์เป็นค่าแบบต่อเนื่อง (continuous values)

“A way to measure whether the algorithm is doing a good job — This is necessary to determine the distance between the algorithm’s current output and its expected output. The measurement is used as a feedback signal to adjust the way the algorithm works. This adjustment step is what we call learning.”

François Chollet, Deep learning with Python (2017), Manning, chapter 1 p.6



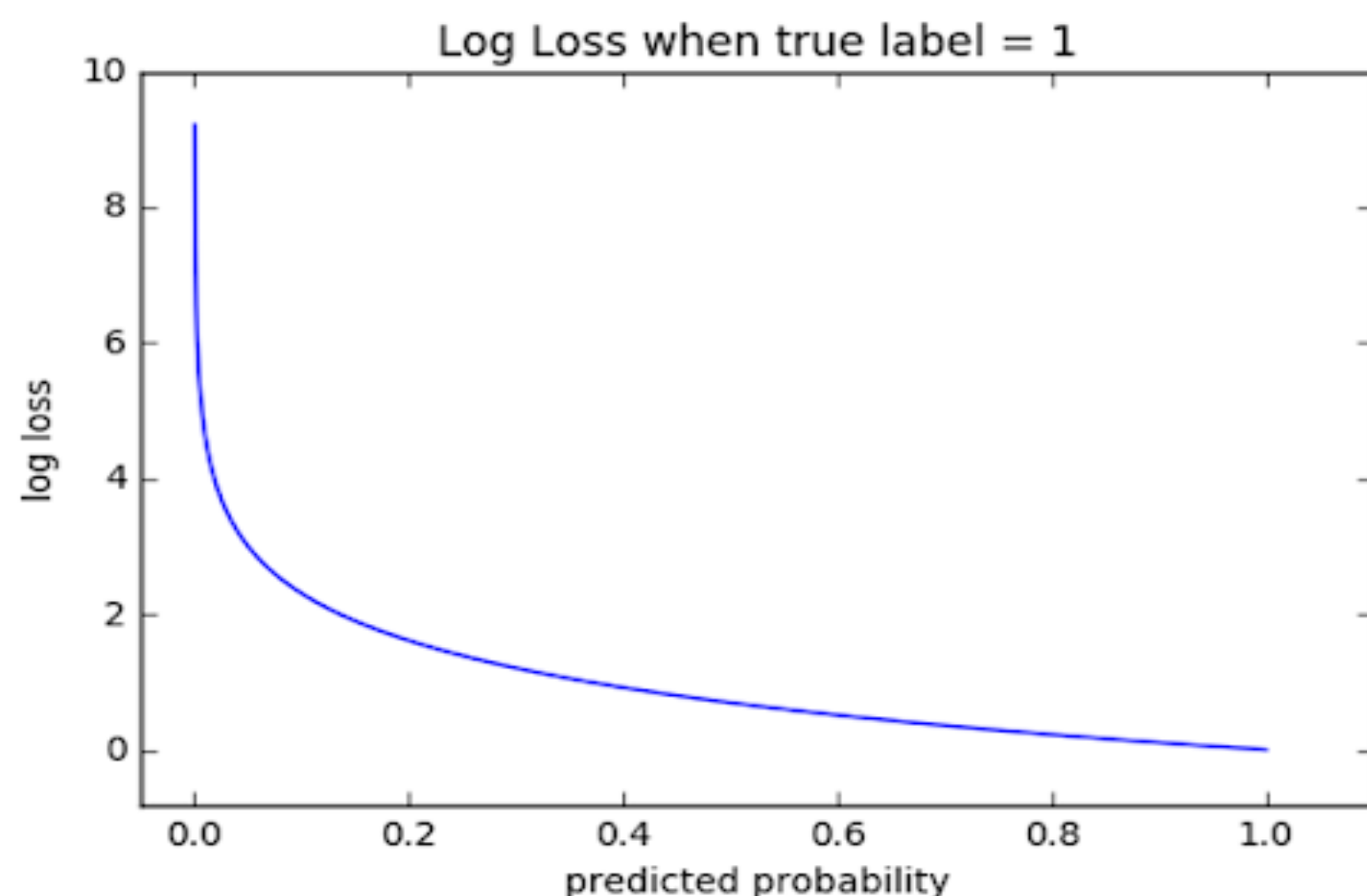
Loss Function (Objective or Cost Function)



- งานทางด้านการแบ่งแยกชนิดของข้อมูล (Classification)
 - Cross-entropy
 - Binary cross-entropy (Log-Loss) สำหรับ binary classification problem
 - Categorical cross-entropy สำหรับ binary และ multiclass problem โดยควรทำเฉลยแบบ one-hot encoding อาทิเช่น 3 คลาส $[1, 0, 0]$, $[0, 1, 0]$ และ $[1, 0, 0]$
 - Sparse cross-entropy สำหรับ binary และ multiclass problem โดยควรทำเฉลยด้วยเลขจำนวนเต็ม อาทิเช่น 3 คลาส 1, 2 และ
 - Hinge Loss
- งานทางด้านการทำนายค่าต่อเนื่อง หรือทดถอย (Regression)
 - Mean Absolute Error Loss (L1 regularization)
 - Mean Square Error Loss (L2 regularization)
 - Huber Loss (a combination of MAE and MSE)
 - Log-Cosh Loss

Loss Function (Objective or Cost Function)

Cross-entropy Loss



ช่วงของค่า	ความหมาย
เท่ากับ 0.00	Perfect probabilities
น้อยกว่า 0.02	Great probabilities
น้อยกว่า 0.05	In a good way
น้อยกว่า 0.20	Great
มากกว่า 0.30	Not great
เท่ากับ 1.00	Hell
มากกว่า 2.00	Something is not working

คือ Loss Function ที่นิยมนำมาใช้งานทางด้าน Classification โดยพัฒนาอยู่บนหลักการของ Log Loss ที่ใช้คำนวณความแตกต่างระหว่างสองความน่าจะเป็น ซึ่งค่าของ Cross-entropy จะลดลงเมื่อความน่าจะเป็นของค่าที่คาดการณ์ใกล้เคียงกับค่าของผลเฉลย นอกจากนี้ ยังนิยมนำมาใช้งานใน Machine Learning ค่อนข้างมาก เนื่องจากทำให้ได้รับแบบจำลองที่ better generalization และการฝึกสอนค่อนข้างรวดเร็ว โดยที่ ค่าของ Cross-entropy Loss คือ ค่าความน่าจะเป็นที่อยู่ในช่วงระหว่าง 0 ถึง 1 ซึ่งสามารถคำนวณได้จากสมการดังต่อไปนี้

สำหรับข้อมูลจำนวน 2 กลุ่ม

$$L = -\frac{1}{m} \sum_{i=1}^m (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

สำหรับข้อมูลจำนวนหลายกลุ่ม

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$



Loss Functions (Cost Function) for Classification Problem

ความเกี่ยวข้องกันระหว่าง Information, Entropy และ Cross-Entropy

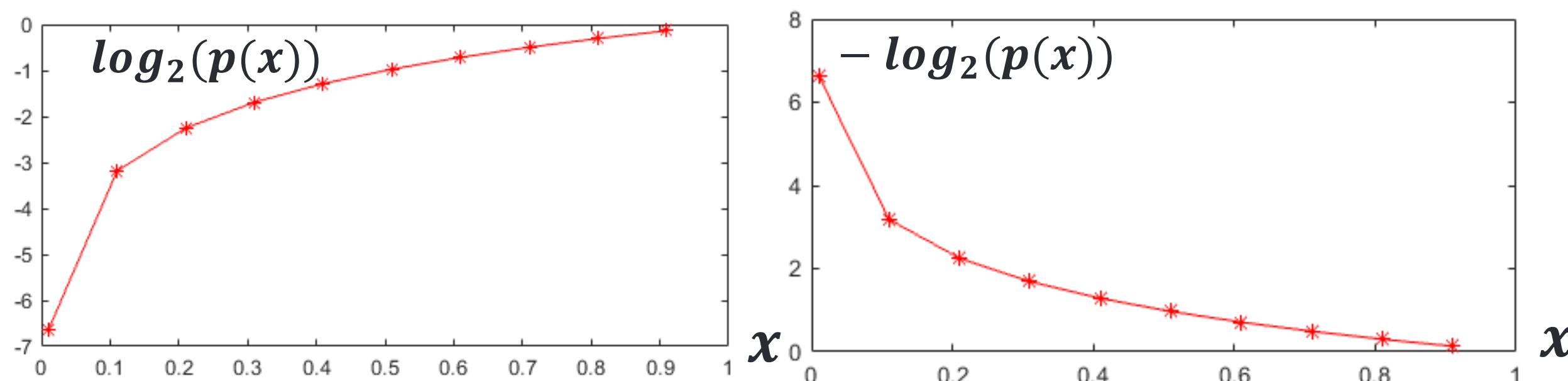
ในปี ค.ศ. 1948 Claude Shannon ได้ศึกษาเกี่ยวกับการหาปริมาณ Information การบีบอัดข้อมูล และขีดจำกัดของการประมวลผลสัญญาณ เขาพบว่า

“เหตุการณ์ที่มีโอกาสเกิดขึ้นต่ำ (Low Probability) จะมีความน่าสนใจ หรือมี Information สูง ในขณะที่เหตุการณ์ที่มีโอกาสเกิดขึ้นสูง (High Probability) จะมี Information ต่ำ”

ซึ่งสามารถนำมาสร้างสมการได้โดยอาศัยหลักการ Logarithm ต่อไปนี้

ค่าความน่าจะเป็นของโอกาสในการเกิด (ค่าที่ได้จาก Softmax)

$$\text{Information}(x) = -\log_2(p(x))$$



อ้างอิง <https://pytorch.org/docs/stable/nn.html> และ <https://blog.pjjop.org/loss-functions-for-training-deep-learning-model-part2/>

ตัวอย่างที่ 1

Output ของ Softmax



	Normalized Probabilities
Dog	0.0006
Cat	0.0596
Airplane	0.9315

$$\text{Information}(\text{Dog}) = -\log_2(0.0006) = 10.70$$

$$\text{Information}(\text{Cat}) = -\log_2(0.0596) = 4.0685$$

$$\text{Information}(\text{Airplane}) = -\log_2(0.9315) = 0.1024$$

จากวิธีการคำนวณข้างต้นจะพบว่าเราจะได้ค่า information มาทั้งหมด 3 ค่า



Loss Functions (Cost Function) for Classification Problem

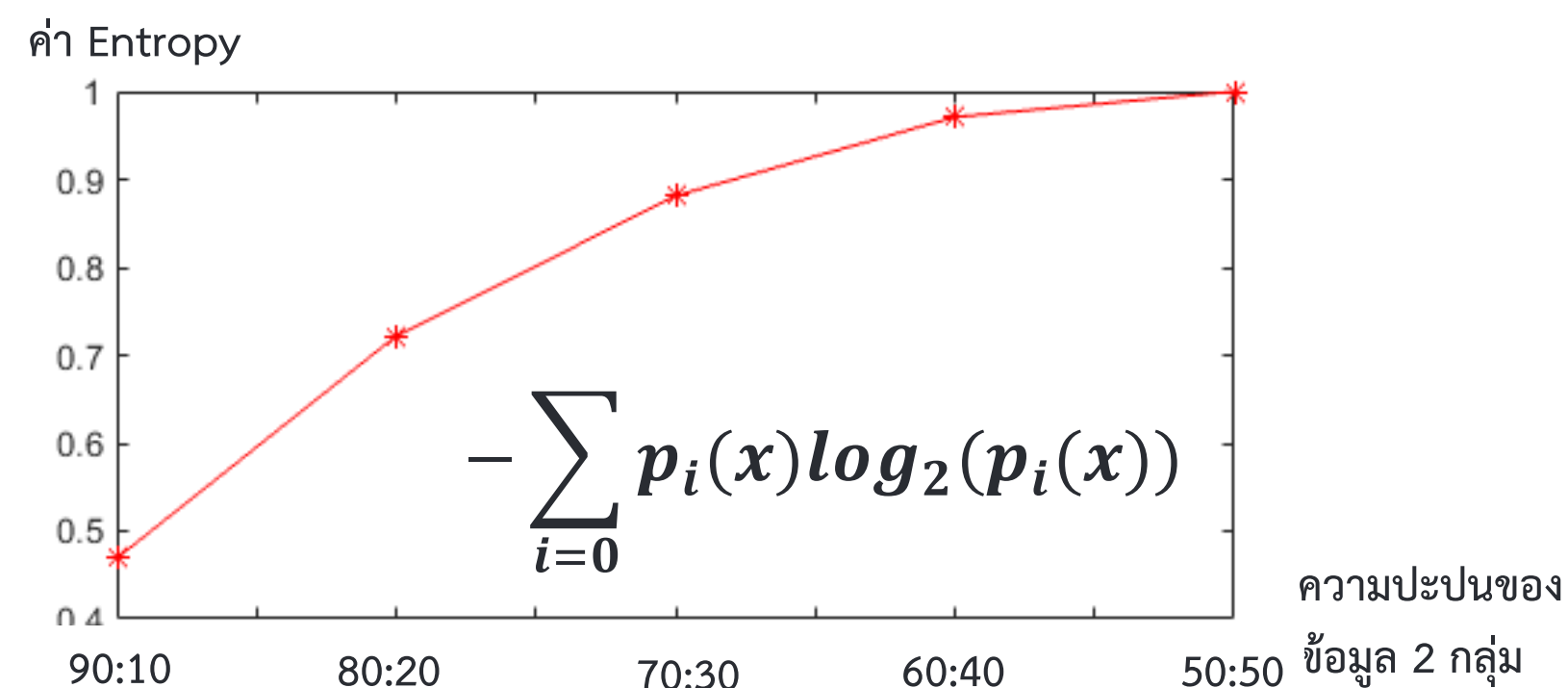
ความเกี่ยวข้องกันระหว่าง Information, Entropy และ Cross-Entropy

Entropy

คือ ค่าที่ใช้บ่งบอกความบริสุทธิ์ (Purity) หรือความปะปนกันของข้อมูล นอกจากนี้ ในงานทางด้านการเรียนรู้ของเครื่องจักรยังสามารถบ่งบอกถึงความน่าสนใจของข้อมูลได้ โดยที่ ค่า Entropy ยิ่งน้อยยิ่งดี

ค่าความน่าจะเป็นของโอกาสในการเกิด (ค่าที่ได้จาก Softmax)

$$Entropy(x) = - \sum_{i=0} p_i(x) \log_2(p_i(x))$$



ตัวอย่างที่ 1



Output ของ Softmax

	Normalized Probabilities
Dog	0.0006
Cat	0.0596
Airplane	0.9315

Entropy = -{

$$(0.0006 \times \log_2(0.0006)) + \\ (0.0596 \times \log_2(0.0596)) + \\ (0.9315 \times \log_2(0.9315))$$

}

$$= -(-0.3442)$$

$$= 0.3442$$

จากวิธีการคำนวณข้างต้นจะพบว่าค่า Entropy ของผลการรู้จำอยู่ที่ 0.3442 และได้รับมาเพียงค่าเดียว นั้นหมายความว่า 1 ภาพ ทำนาย ต่อ 1 ค่า Entropy



Loss Functions (Cost Function) for Classification Problem

ความเกี่ยวข้องกันระหว่าง Information, Entropy และ Cross-Entropy

Cross-Entropy Loss (หรือ Logistic Loss หรือ Multinomial Logistic Loss)

คือ Loss Function ซึ่งอยู่บนพื้นฐานของความน่าจะเป็นของ Shannon ที่นิยมนำมาใช้ในงาน multi-class classification

ค่าจาก Softmax ของ Actual

ค่าจาก Softmax ของ Predict

$$CrossEntropy(x) = - \sum_{i=0} q_i(x) \log_2(p_i(x))$$

ซึ่งสามารถลดรูปได้ดังสมการต่อไปนี้

$$CrossEntropy(x) = -q_{correct}(x) \log_2(p_{correct}(x))$$

โดยที่ค่า Cross-Entropy ยิ่งน้อยยิ่งดี ซึ่งในงานทางด้านการเรียนรู้ของเครื่องจักรหมายถึงแบบจำลองทำนายถูกและมีความมั่นใจมาก

ตัวอย่างที่ 1

Output ของ Softmax



	Normalized Probabilities
Dog	0.0006
Cat	0.0596
Airplane	0.9315

$$\begin{aligned} CrossEntropy = -\{ & (0 \times \log_2(0.0006)) + // \text{ Dog} \\ & (0 \times \log_2(0.0596)) + // \text{ Cat} \\ & (1 \times \log_2(0.9315)) // \text{ AirPlane} \\ & \} \end{aligned}$$

$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

(Actual)

$\begin{bmatrix} 0.0006 \\ 0.0596 \\ 0.9315 \end{bmatrix}$

(Predict)

$$= -(-0.1023) = 0.1023$$

จะพบว่าค่า Cross -Entropy ของผลการรู้จำอยู่ที่ 0.1023 และได้รับมาเพียงค่าเดียว นั่นหมายความว่า 1 ภาพทำนาย ต่อ 1 ค่า Cross-Entropy

Loss Function (Objective or Cost Function)

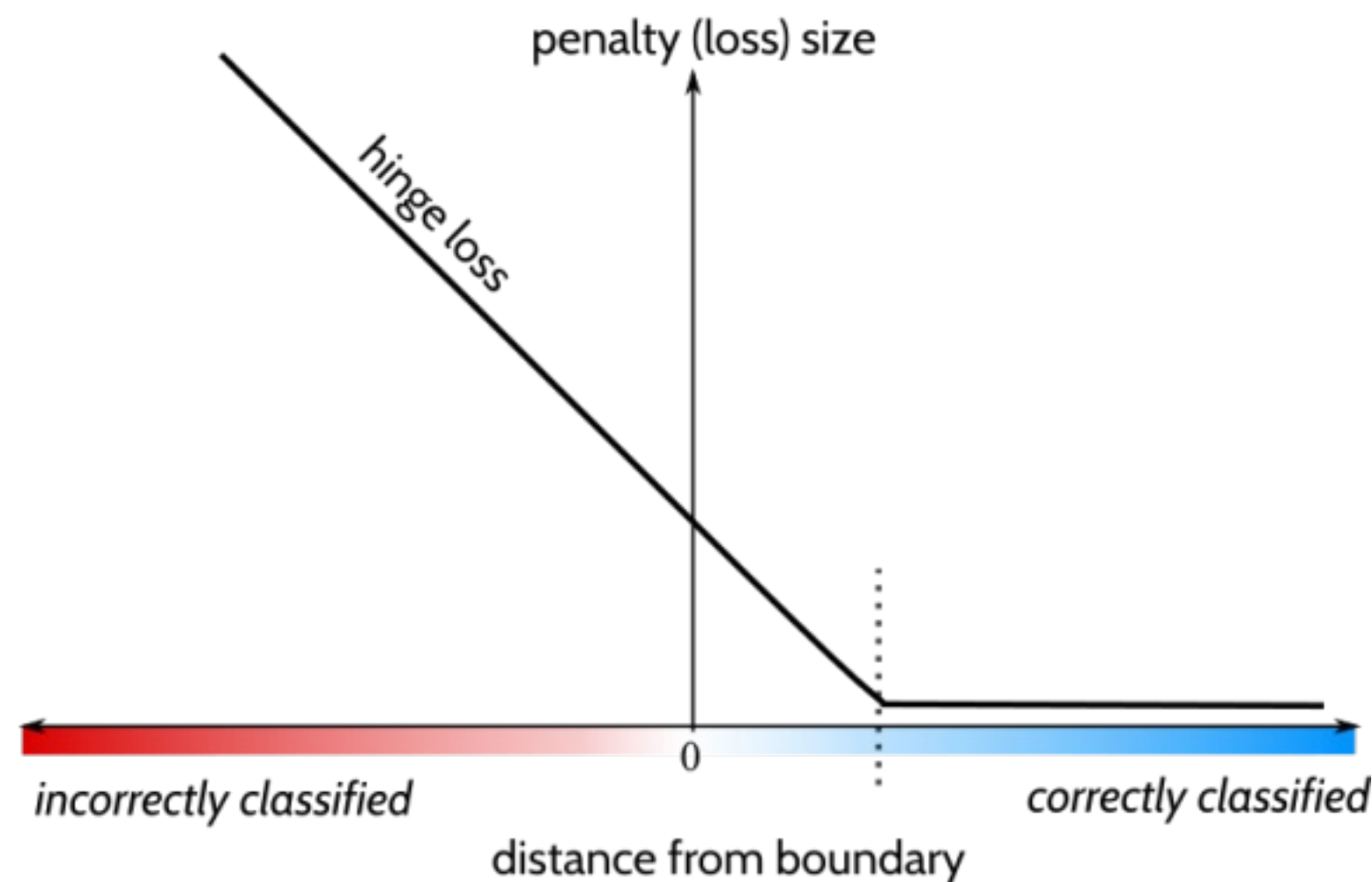
Weighted Cross-Entropy

$$\text{CrossEntropy}(\mathbf{x}) = - \sum_{i=0} w_i \overset{\text{ค่าจาก Softmax ของ Actual}}{\underset{\text{ค่าจาก Softmax ของ Predict}}{q_i(\mathbf{x})} \log_2(p_i(\mathbf{x}))}$$

คือ Loss Function ที่นิยมนำมาใช้งานทางด้าน Classification และ Segmentation ที่พัฒนาต่อยอดมาจาก Cross-Entropy โดยการเพิ่มพจน์ Weight ลงไปในพารามิเตอร์บางตัวของสมการทำให้สามารถหลีกเลี่ยงปัญหาที่เกิดจาก imbalanced datasets (จำนวนข้อมูลตัวอย่างในแต่ละกลุ่มมีจำนวนที่แตกต่างกันค่อนข้างมาก) ได้

Loss Function (Objective or Cost Function)

Hinge Loss



คือ Loss Function ที่นิยมนำมาใช้งานทางด้าน Classification และนิยมนำมาใช้งานกับ activation function แบบ Tanh ในเลเยอร์สุดท้ายของเครือข่าย เนื่องจาก Hinge Loss จะทำงานได้ดีกับค่าในช่วง -1 ถึง 1 แทนที่จะเป็น 0 ถึง 1 หลักการทำงานคล้ายกับ Cross-entropy Loss แต่จะลงโทษ (1) การทำนายผิดและ (2) การทายถูกที่ไม่มั่นใจ (ได้เพียงระดับหนึ่งเท่านั้น) ซึ่งสามารถคำนวณได้จากสมการดังต่อไปนี้

$$L = \max(0, 1 - \underset{(y_pred)}{y} * \underset{(y_true)}{f(x)})$$

จากสมการจะพบว่ามีการใช้เครื่องหมาย (+ / -) ของการจำแนกประเภทเพื่อนำมาคำนวณค่าของ Loss Function ซึ่งค่าของ Loss Function จะยิ่งมากขึ้นหากค่าของผลเฉลยมีเครื่องหมายตรงกันข้ามกับค่าที่ทำนายได้



Loss Functions (Cost Function) for Classification Problem

Hinge Loss

คือ Loss Functions ที่ออกแบบอยู่บนแนวคิดที่ว่า “maximum-margin classification และค่า (score) ของกลุ่มที่ถูกต้องจะต้องสูงกว่าค่าของผลรวมในกลุ่มที่ผิด”

$$\text{Loss} = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

ค่าความน่าจะเป็นจากกลุ่มถูก

ค่าความน่าจะเป็นจากกลุ่มผิด

โดยที่ s_j และ s_{y_i} คือ Normalized Probabilities ที่ได้รับจาก Softmax ของกลุ่มที่ถูกและกลุ่มที่ผิดตามลำดับ

ตัวอย่างที่ 1



Airplane vs Dog

Output ของ Softmax

	Normalized Probabilities
Dog	0.0006
Cat	0.0596
Airplane	0.9315

Airplane vs Cat

$$\begin{aligned} \text{Loss} &= \max(0, 0.0006 - 0.9315 + 1) + \max(0, 0.0596 - 0.9315 + 1) \\ &= \max(0, 0.0745) + \max(0, 0.1281) \\ &= 0.0745 + 0.1281 \\ &= 0.2026 \end{aligned}$$

จากวิธีการคำนวณข้างต้นจะได้รับค่า Loss เท่ากับ 0.2026 แสดงว่าทำนายได้ถูกต้องและมีค่าความมั่นใจมาก เนื่องจาก *ค่า Loss เข้าใกล้ 0*



Loss Functions (Cost Function) for Classification Problem

Hinge Loss

คือ Loss Functions ที่ออกแบบอยู่บนแนวคิดที่ว่า “maximum-margin classification และค่า (score) ของกลุ่มที่ถูกต้องจะต้องสูงกว่าค่าของผลรวมในกลุ่มที่ผิด”

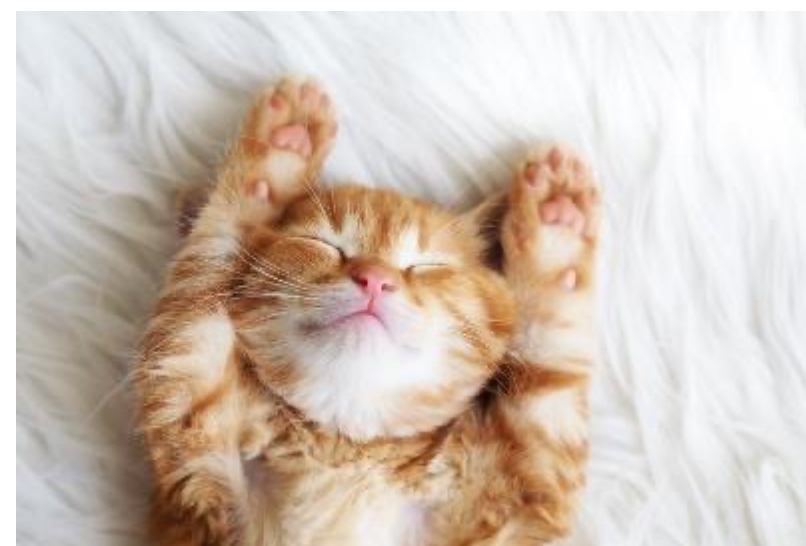
$$\text{Loss} = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

ค่าความน่าจะเป็นจากกลุ่มถูก

ค่าความน่าจะเป็นจากกลุ่มผิด

โดยที่ s_j และ s_{y_i} คือ Normalized Probabilities ที่ได้รับจาก Softmax ของกลุ่มที่ถูกและกลุ่มที่ผิดตามลำดับ

ตัวอย่างที่ 2



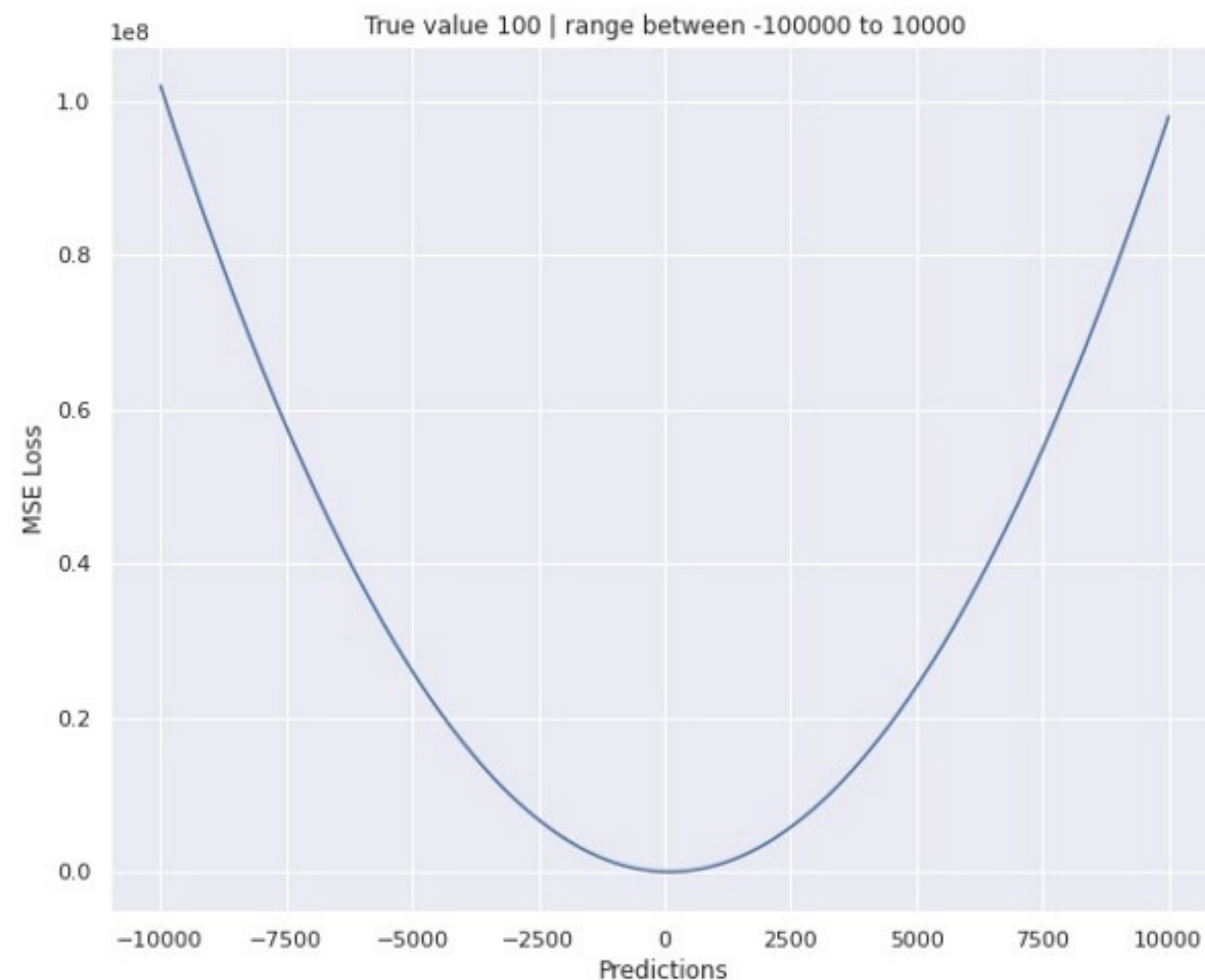
	Normalized Probabilities
Dog	0.691
Cat	0.245
Airplane	0.064

$$\begin{aligned}\text{Loss} &= \max(0, 0.691 - 0.245 + 1) + \max(0, 0.064 - 0.245 + 1) \\ &= \max(0, 1.446) + \max(0, 0.819) \\ &= 1.446 + 0.819 \\ &= 2.265\end{aligned}$$

จากวิธีการคำนวณข้างต้นจะได้รับค่า Loss เท่ากับ 2.265 แสดงว่าทำนายได้ไม่ถูกต้องและมีค่าความมั่นใจน้อย เนื่องจาก *ค่า Loss เข้ามากกว่า 0*

Loss Function (Objective or Cost Function)

Mean Square Error (MSE) or Squared Error Loss or Quadratic Loss (L2 (Ridge) regularization)

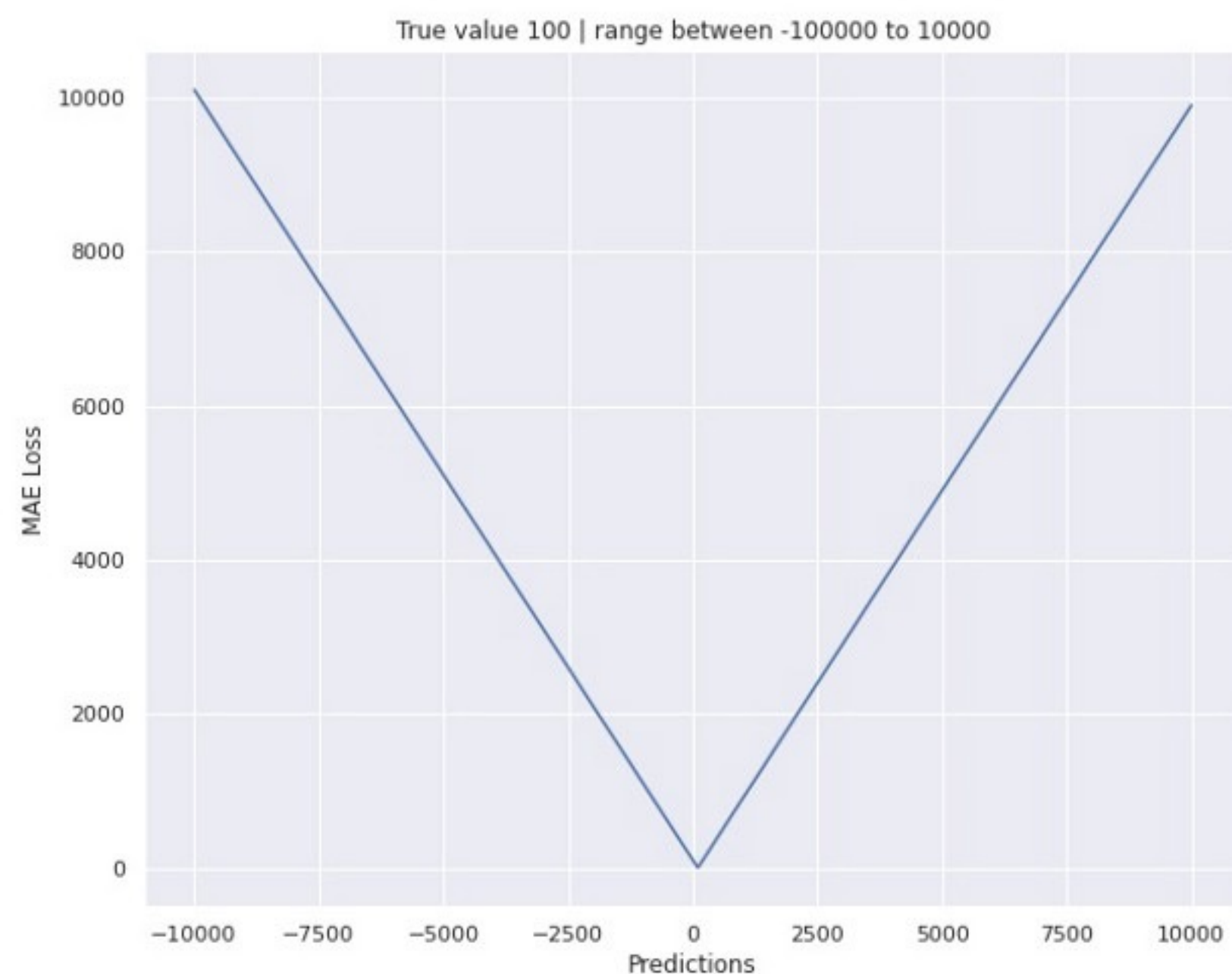


คือ Loss Function ที่นิยมนำมาใช้งานทางด้าน Regression ซึ่งจะพิจารณาจากค่าเฉลี่ยของค่าความแตกต่าง (หรือค่าความคลาดเคลื่อน) ระหว่างผลลัพธ์ของโครงข่ายประสาทเทียม (y_{pred}) กับผลลัพธ์ที่คาดหวัง (y_{true}) ในรูปแบบผลต่างยกกำลังสอง ซึ่ง MSE ไม่เหมาะสมกับข้อมูลที่มี Outlier ค่อนข้างมาก เพราะการคำนวณหาค่า MSE จะคิดจากผลต่างยกกำลังสองทำให้ข้อมูล Outlier ให้ค่าผลต่างค่อนข้างสูง ส่งผลให้การฝึกสอนแบบจำลองมีการคลาดเคลื่อนตามมาค่าที่ไม่ได้สะท้อนกลุ่มข้อมูลหลัก

$$Loss = \frac{1}{n} * \sum_{i=0}^n (Y_i - Y_{pred_i})^2$$

Loss Function (Objective or Cost Function)

Mean Absolute Error (MAE) Loss (L1 (Lasso) regularization)



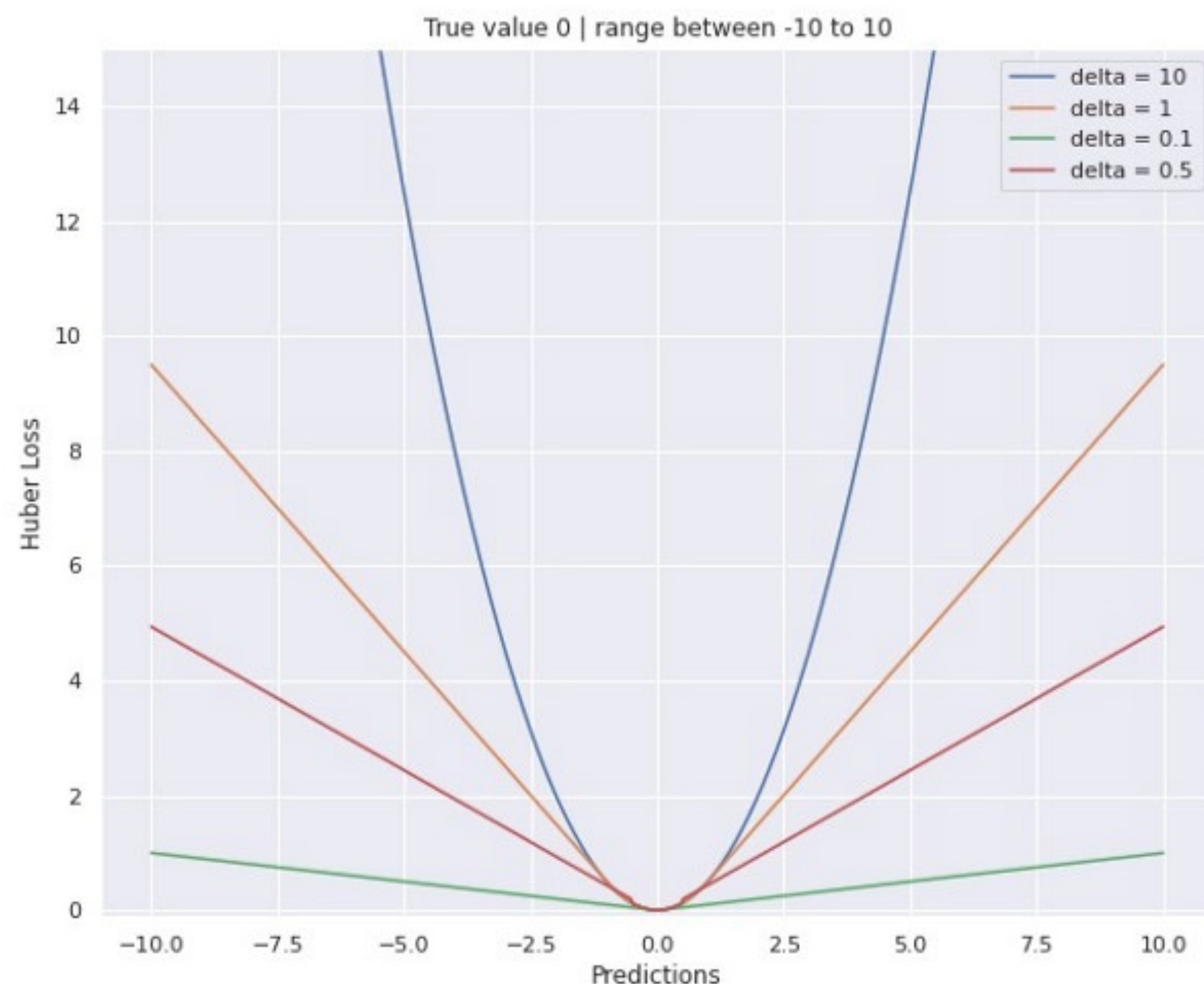
คือ Loss Function ที่นิยมนำมาใช้งานทางด้าน Regression ซึ่งจะพิจารณาจากค่าเฉลี่ยของค่าความแตกต่าง (หรือค่าความคลาดเคลื่อน) ระหว่างผลลัพธ์ของโครงข่ายประสาทเทียม (y_{pred}) กับผลลัพธ์ที่คาดหวัง (y_{true}) ในรูปแบบค่าสมบูรณ์ ซึ่ง MAE มีความทนทานกับข้อมูลที่พบ Outlier มากกว่า MSE เนื่องจาก MAE มีพฤติกรรมคล้ายกับ Median ของทางสถิติ แต่อย่างไรก็ตาม การนำค่า MAE ไปใช้เพื่อ Optimization แบบจำลองอาจจะใช้เวลาในการฝึกสอนนานเกินความจำเป็น เนื่องจากขยับค่าอยู่ในช่วง +1 ถึง -1

$$Loss = \frac{1}{n} \sum_{i=0}^n |y_i - y_{predict_i}|$$

Loss Function (Objective or Cost Function)



Huber Loss (Smooth Mean Absolute Error)

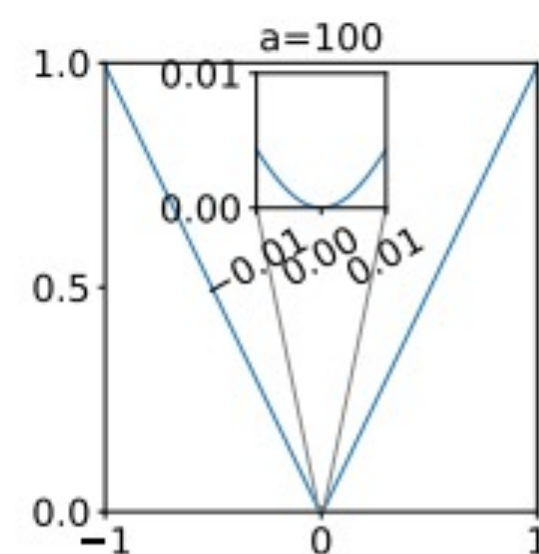
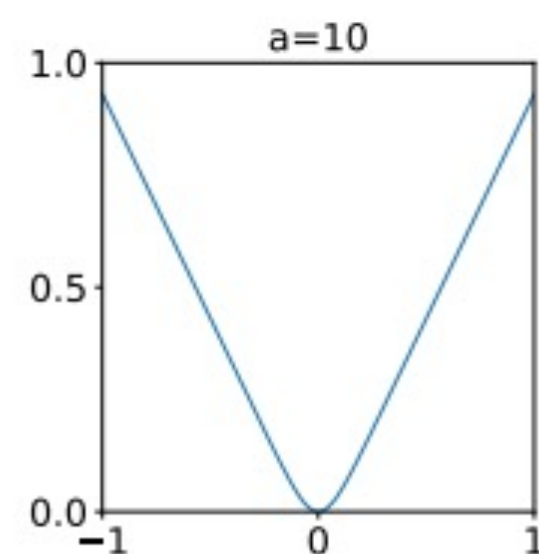
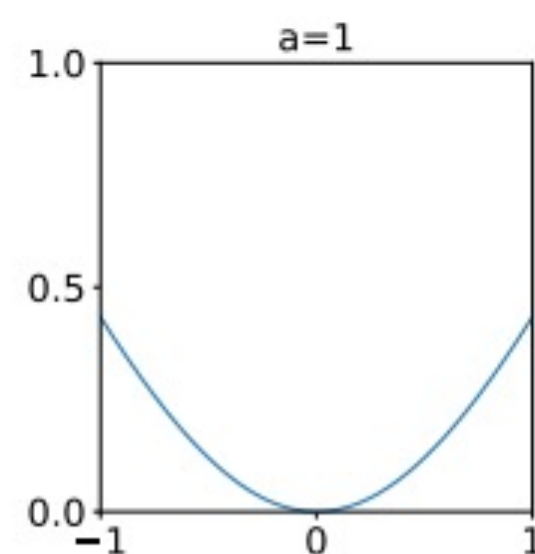
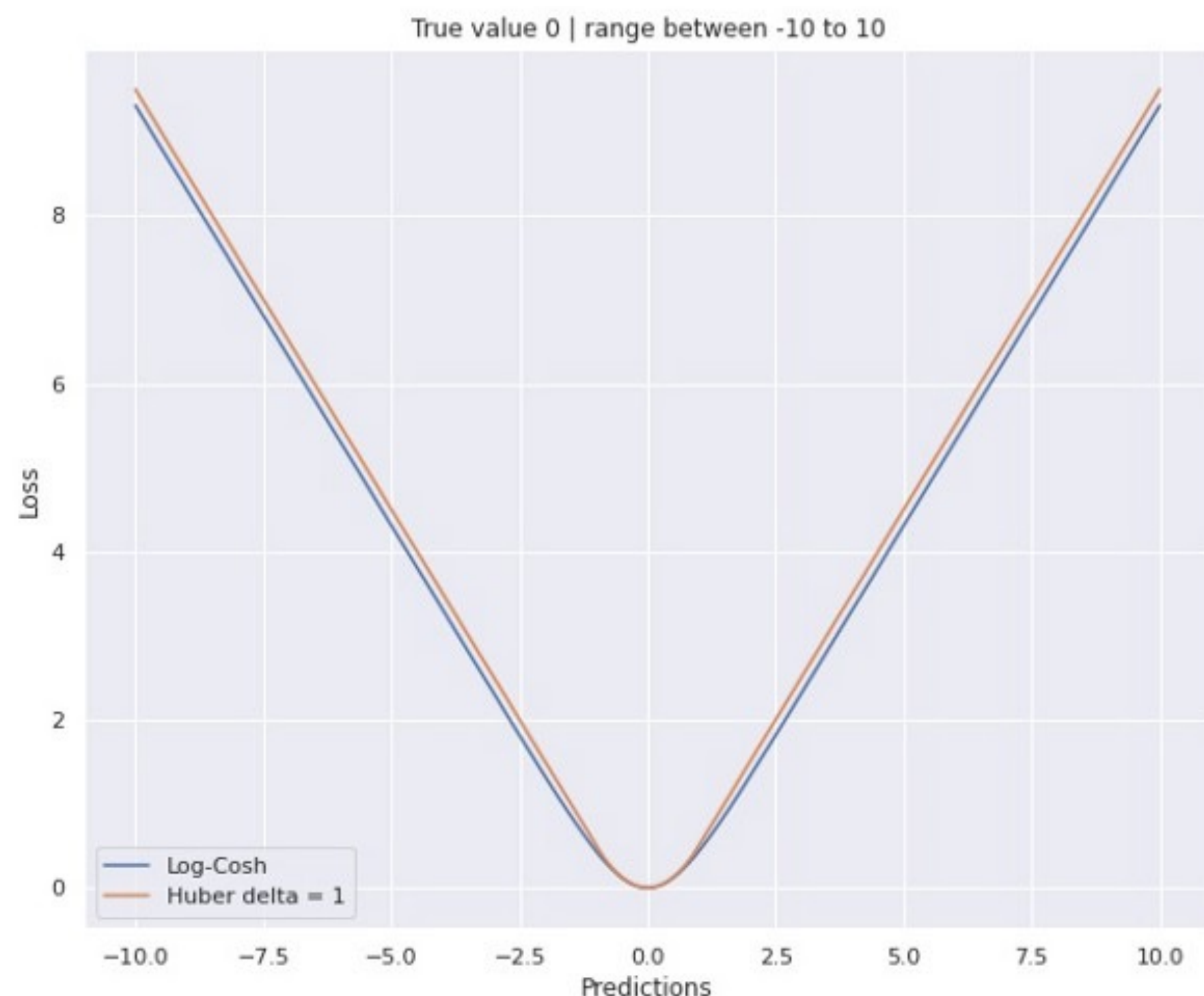


คือ Loss Function ที่ผสมผสานการทำงานระหว่าง MAE และ MSE เข้าด้วยกัน และนิยมนำมาใช้งานทางด้าน Regression โดยอาศัยหลักการ Log ของ hyperbolic cosine ซึ่งพัฒนามาจากแนวความคิดที่ว่า MSE เหมาะสมกับข้อผิดพลาดเล็กน้อย ขณะที่ MAE เหมาะสมกับข้อผิดพลาดขนาดใหญ่ โดยอาศัยค่า Delta เป็นตัวกำกับการตัดสินใจ นอกจากนี้ Huber Loss ยังคงเหมาะสมกับงานทางด้าน Regression แม้ว่า Huber Loss ได้รวมเอาข้อดีของ MSE และ MAE เข้าด้วยกัน แต่ค่า Delta จะกลายเป็นหนึ่งในไฮเปอร์พารามิเตอร์อีกอันที่ต้องฝึกสอนแบบจำลองเพื่อค้นหาค่าที่เหมาะสม

$$L_{\delta}(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta, \\ \delta |y - f(x)| - \frac{1}{2}\delta^2 & \text{otherwise.} \end{cases}$$

Loss Function (Objective or Cost Function)

Log-Cosh Loss



คือ Loss Function ที่คล้ายคลึงกับ MSE ที่มีความทนทานต่อข้อมูล Outlier และนิยมนำมาใช้งานทางด้าน Regression ที่พยายามจะนำข้อดีของ MSE Loss และ MAE มาผสมผสานเข้าด้วยกัน

$$f(t; a) = \frac{1}{a} \log(\cosh(at)) = \frac{1}{a} \log \frac{e^{at} + e^{-at}}{2} \rightarrow \begin{cases} |t| - \frac{1}{a} \log 2, & \text{when } |t| \rightarrow \infty \\ 0.5at^2, & \text{when } |t| \rightarrow 0 \end{cases}$$

โดยที่ $a \in \mathbb{R}^+$ เป็นพารามิเตอร์ที่ต้องฝึกสอน และลอการิทึมเป็นฐานธรรมชาติ ค่ามาก Log-Cosh Loss อยู่ในช่วง -1 ถึง 1 สำหรับ a ที่ต่างกัน ซึ่งถ้าพิจารณาจะพบว่า ค่าของ t คือผลต่างระหว่างผลลัพธ์ของโครงข่ายประสาทเทียม (y_{pred}) กับผลลัพธ์ที่คาดหวัง (y_{true}) ซึ่ง ถ้า $|t|$ มีค่ามาก Log-Cosh Loss จะมีพฤติกรรมคล้ายกับ MAE ขณะที่ ถ้า $|t|$ มีค่าน้อย Log-Cosh Loss จะมีพฤติกรรมคล้ายกับ MSE

นอกจากนี้ Log-Cosh Loss ยังใช้เวลาคำนวณน้อยกว่า Huber Loss แต่อย่างไรก็ตาม Log-Cosh Loss สามารถปรับเปลี่ยนลักษณะของกราฟได้น้อยกว่า Huber Loss

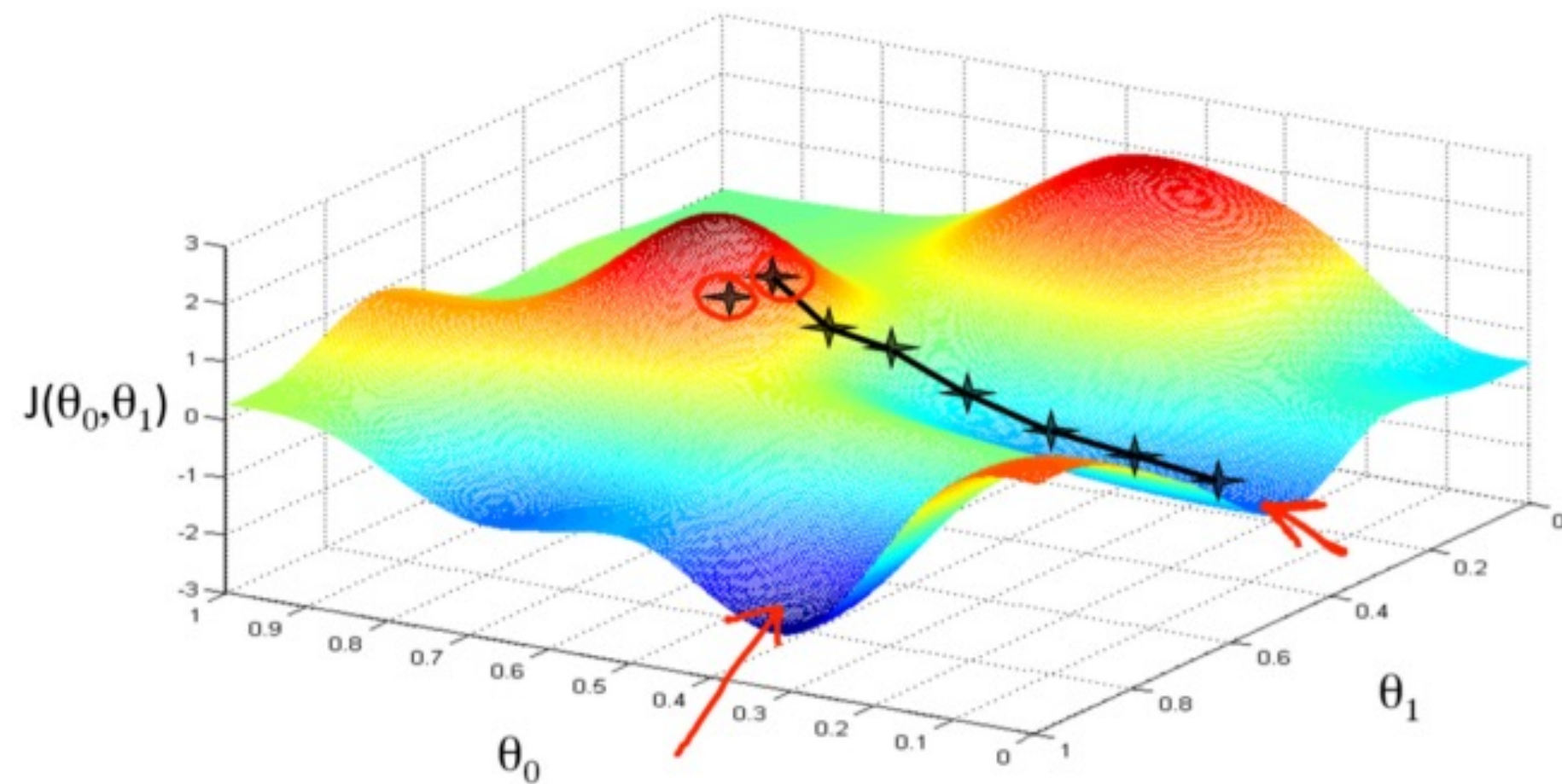
องค์ประกอบที่ควรรู้สำหรับการฝึกสอนแบบจำลอง



- Loss Function (Objective or Cost Function)
- Optimization
- Learning Rate



Optimization



$$\theta^{(\text{next step})} = \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$$

θ เริ่มต้นเป็นค่า random

learning rate จะก้าวใหญ่แค่ไหน

gradient ของ $\text{MSE}(\theta)$ w.r.t. θ เป็นตัวบอกทิศทางลงภูเขา

$$\nabla_{\theta} \text{MSE}(\theta) = \begin{bmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\theta) \end{bmatrix} = \frac{2}{m} \mathbf{X}^T \cdot (\mathbf{X} \cdot \theta - \mathbf{y})$$

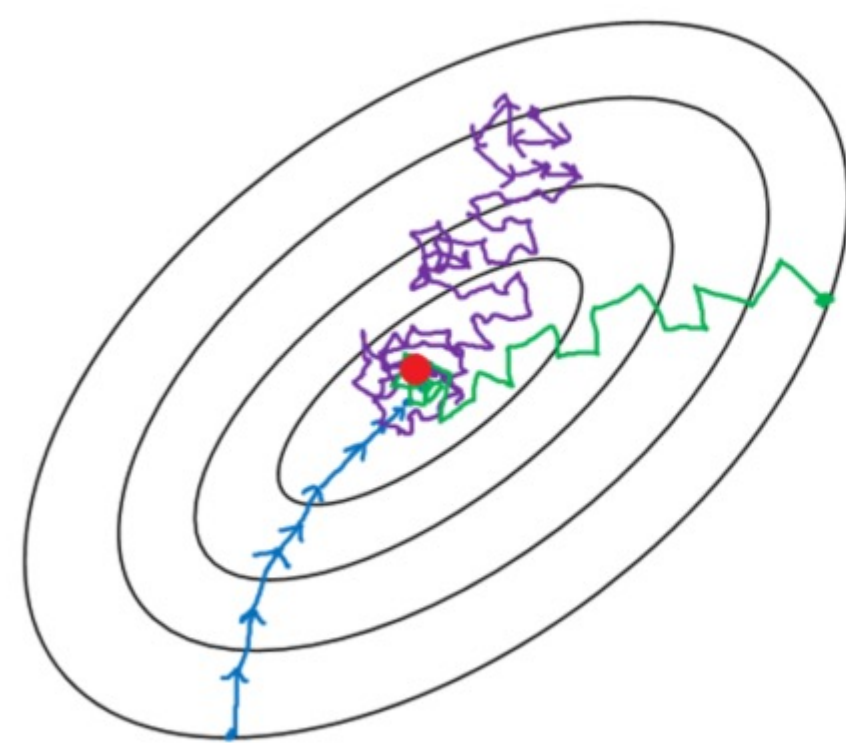
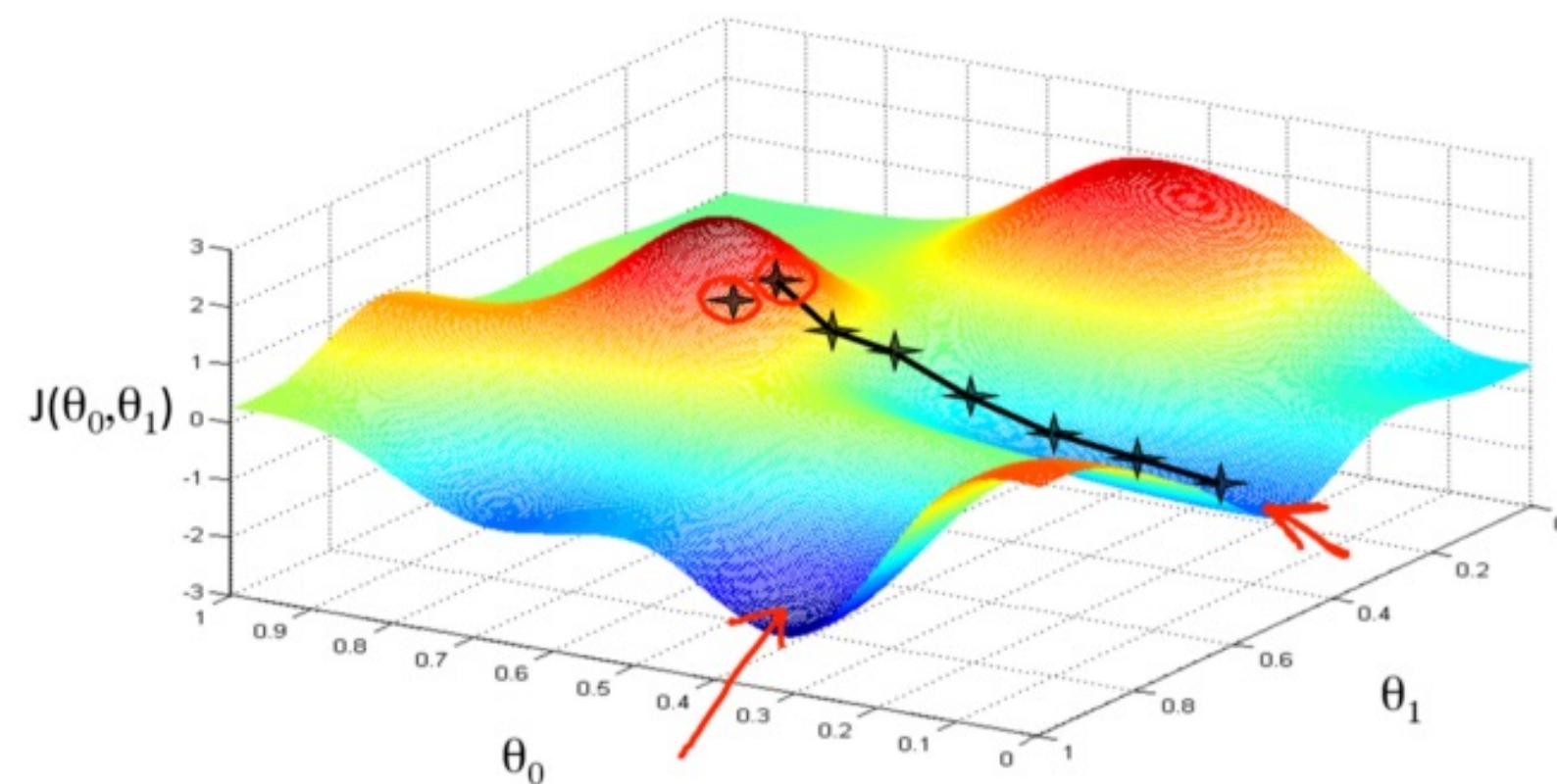
คือ อัลกอริทึมสำหรับปรับปรุงแบบจำลอง (Optimization Algorithm) ให้มีประสิทธิภาพมากยิ่งขึ้น โดยพิจารณาจากค่าความผิดพลาดที่เกิดขึ้น เพื่อไปคำนวณหาค่าพารามิเตอร์ต่าง ๆ ของแบบจำลองใหม่ นอกจากนี้ Optimizer ยังเก็บข้อมูลสถานะและค่าพารามิเตอร์ต่าง ๆ ของแบบจำลองไว้ และ Optimizer ต้องการกำหนดค่าพารามิเตอร์ต่าง ๆ ที่จำเป็น อาทิเช่น learning rate, optimizer-specific เป็นต้น

ในปัจจุบันนี้ มีวิธีการสร้าง Optimizer ค่อนข้างหลากหลาย แต่ตัวที่เป็นพื้นฐาน ได้แก่ Gradient Descent ร่วมกับ Loss Function โดยที่ Gradient Descent มีหน้าที่บ่งบอกทิศทาง ขณะที่ Loss Function มีหน้าที่บ่งบอกว่าถูกหรือผิด



Gradient Descent

คือ อัลกอริทึมปรับปรุงแบบจำลอง (Optimization Algorithm) แบบทำงานซ้ำไปเรื่อย ๆ จนกว่าจะพบ (1) ค่าต่ำสุดของฟังก์ชัน หรือ (2) จนกว่าจะเข้าใกล้ค่าต่ำสุดที่สามารถยอมรับได้



— Batch gradient descent
— Mini-batch gradient Descent
— Stochastic gradient descent

นอกจากนี้ Gradient Descent ยังเป็นอัลกอริทึมการเพิ่มประสิทธิภาพแบบอนุพันธ์ลำดับที่หนึ่งที่นิยมในการเรียนรู้ของเครื่องและการเรียนรู้เชิงลึก ซึ่งจะพิจารณาเฉพาะอนุพันธ์ลำดับแรกเมื่อทำการอัปเดตพารามิเตอร์ต่าง ๆ ในทิศทางตรงกันข้ามกับการไล่ระดับของฟังก์ชันจุดประสงค์ (Objective Function: $f(\cdot)$) ในการทำซ้ำแต่ละรอบ ซึ่งขนาดในการเปลี่ยนแปลงจะขึ้นอยู่กับอัตราการเรียนรู้ (Learning rate: α) โดยอาศัยสมการต่อไปนี้

$$\mathbf{x}'_i = \mathbf{x}_i - \alpha \nabla f(\mathbf{x}_i)$$

Diagram illustrating the update rule for Gradient Descent:

- $\mathbf{x}_i$: current position
- α : learning rate (step size)
- $\nabla f(\mathbf{x}_i)$: gradients of loss function
- $\mathbf{x}'_i$: next position
- opposite direction: indicated by the minus sign in the equation

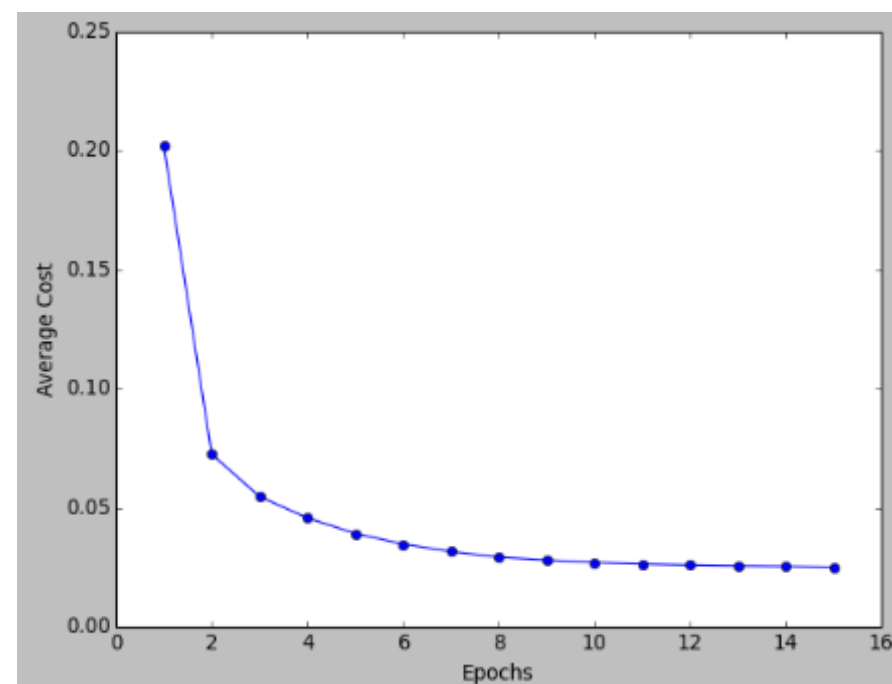
ซึ่ง Gradient Descent มีหลายประเภท อาทิเช่น (1) Batch Gradient Descent, (2) Mini-batch Gradient Descent, และ (3) Stochastic Gradient Descent.



Gradient Descent

Gradient Descent มีหลายประเภท อาทิเช่น (1) Batch, (2) Mini-batch และ (3) Stochastic Gradient Descent

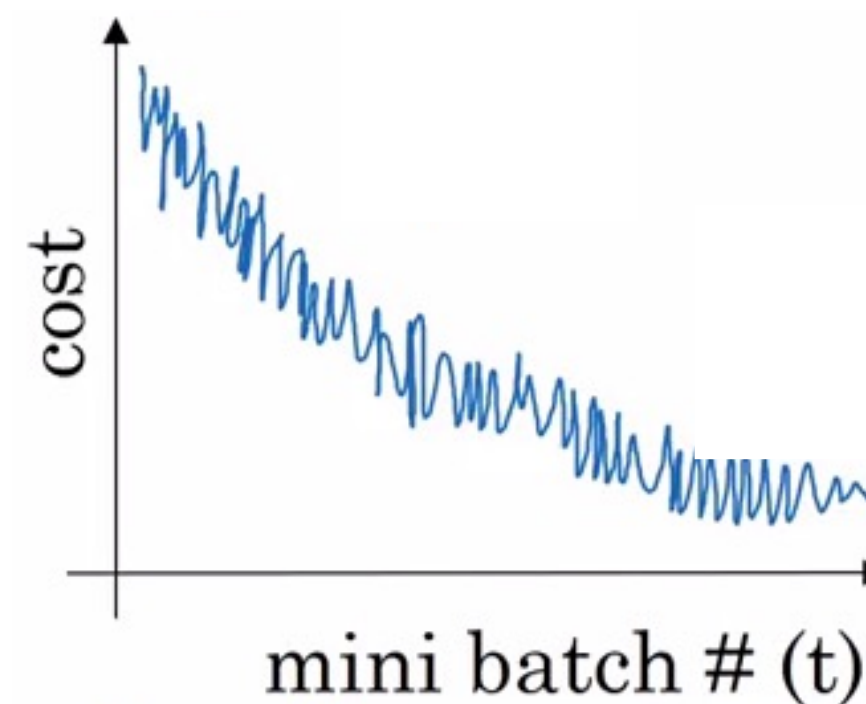
Batch Gradient Descent



ภาพรวมของกราฟค่า loss function จะค่อย ๆ ลดลงในทุก ๆ epoch

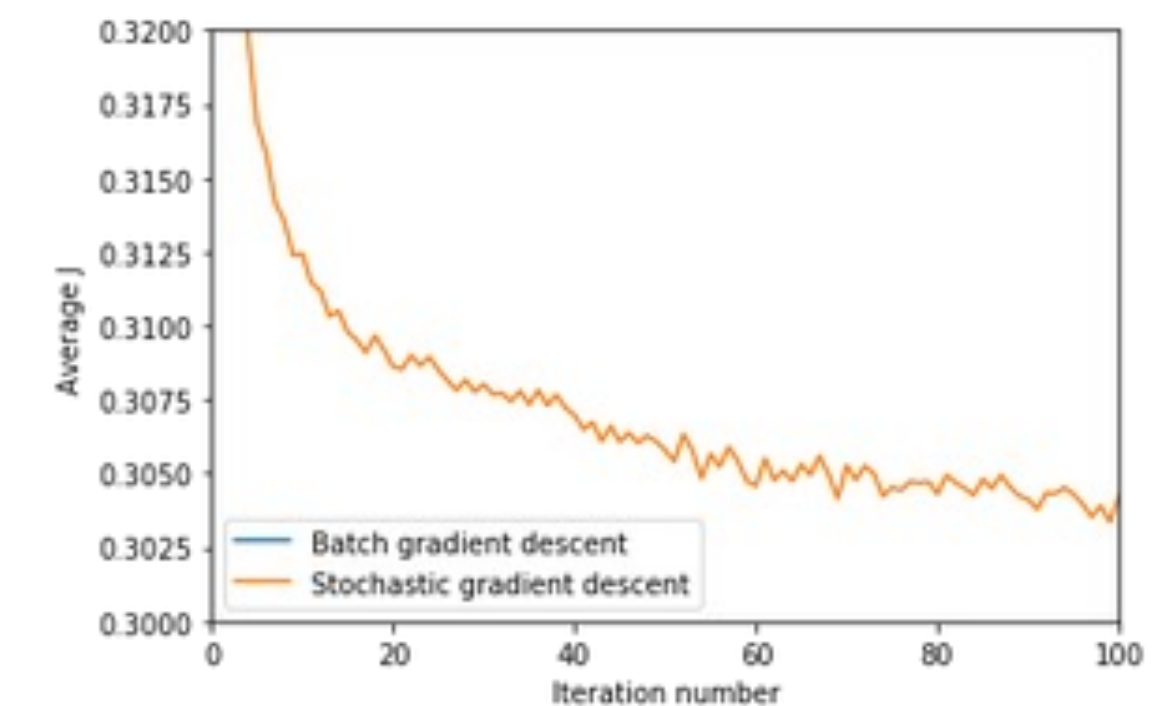
อัปเดตค่าพารามิเตอร์ของแบบจำลองเพียงครั้งเดียวต่อการประมวลผลข้อมูลทั้งหมดจากชุดฝึกสอน ซึ่งเป็นการปรับปรุงแบบจำลองตามค่าเฉลี่ยของ Loss Function

Mini-batch Gradient Descent



อัปเดตค่าพารามิเตอร์ของแบบจำลองเพียงครั้งเดียวต่อการประมวลผลข้อมูล N ภาพจากชุดฝึกสอน ซึ่งเป็นการปรับปรุงแบบจำลองตามค่าเฉลี่ยของ Loss Function จากข้อมูล N ภาพ

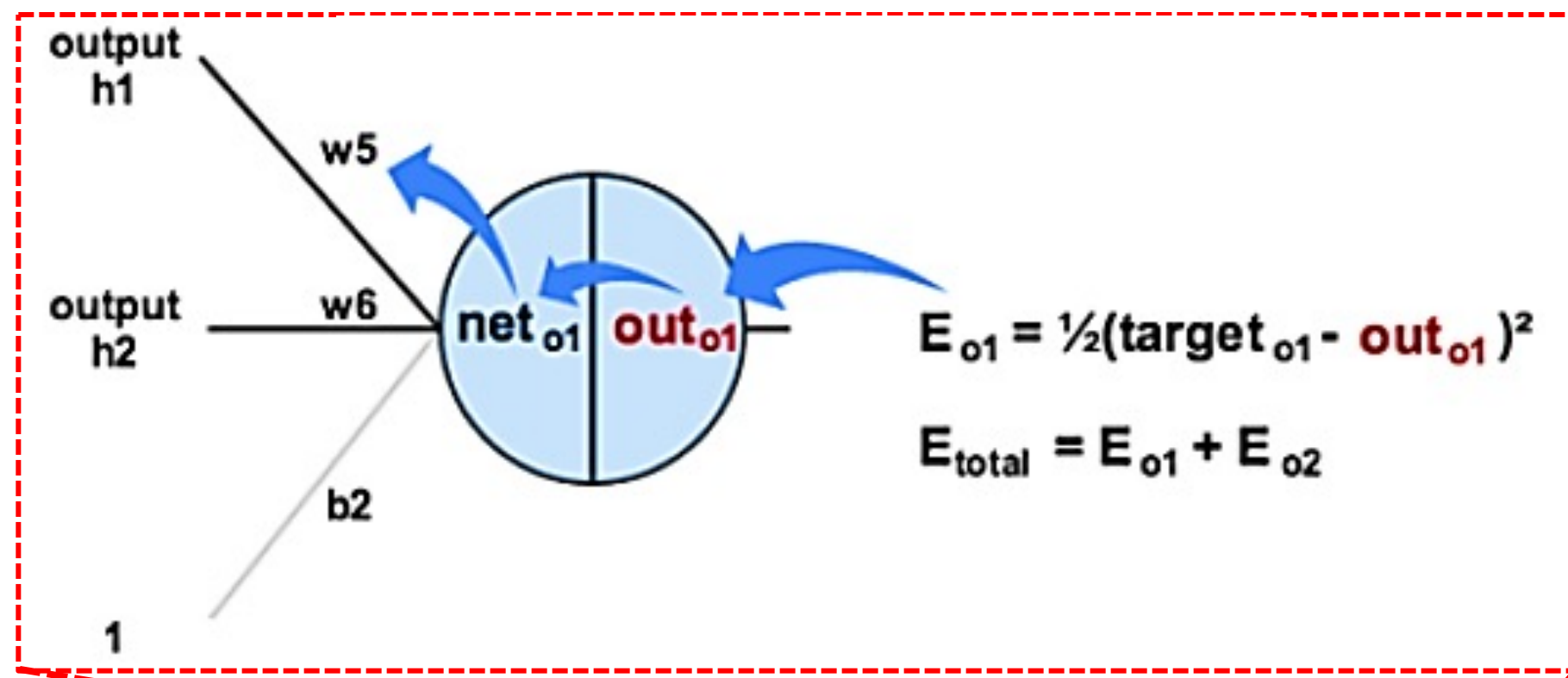
Stochastic Gradient Descent



แต่แต่ละครั้งไม่ได้การันตีว่าค่า Loss Function จะลดลง ซึ่งอาจจะเพิ่มขึ้นก็ได้

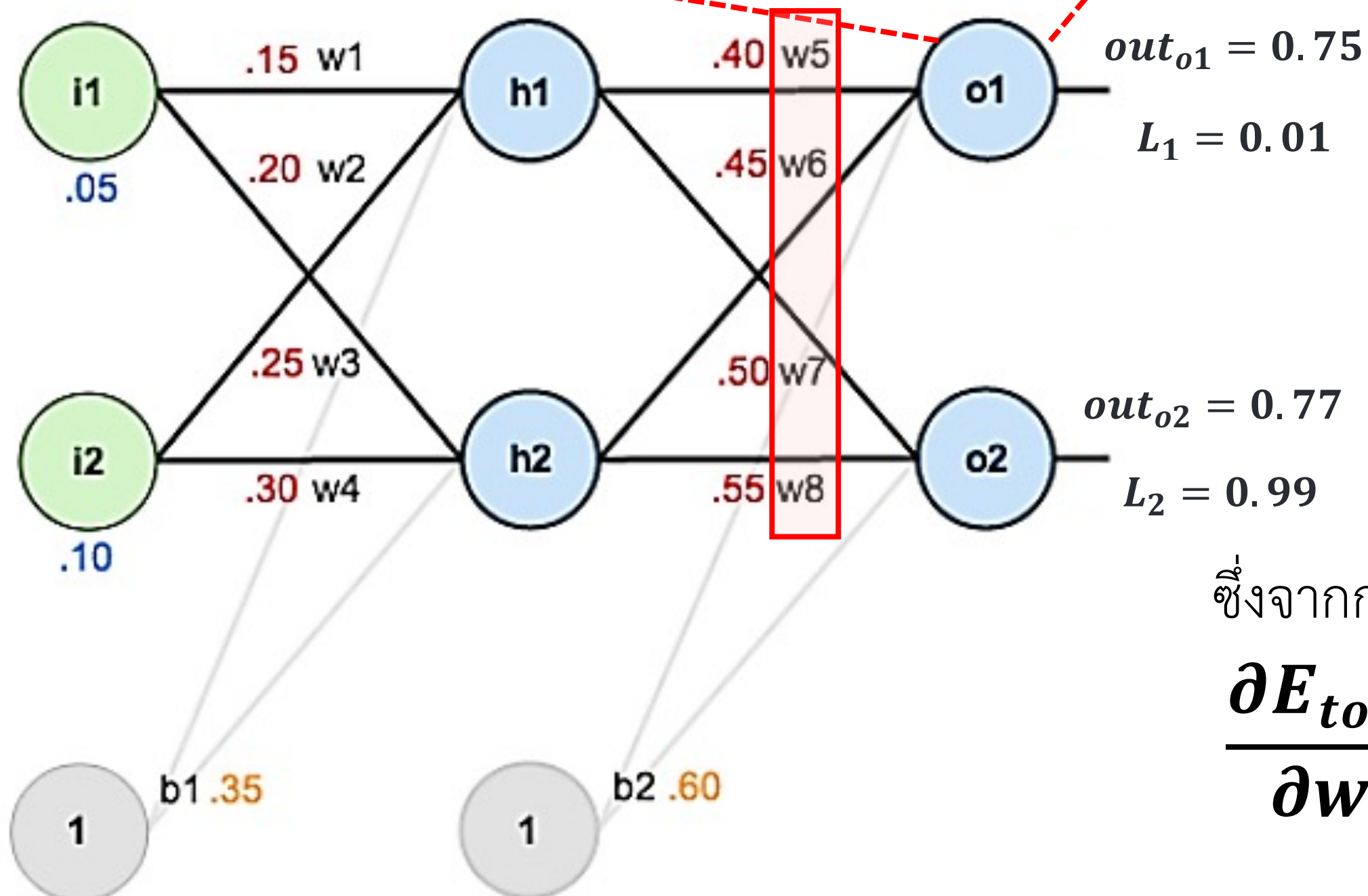
อัปเดตค่าพารามิเตอร์ของแบบจำลองเพียงหนึ่งครั้งต่อการประมวลผลข้อมูล 1 ภาพจากชุดฝึกสอน ซึ่งเป็นการปรับปรุงแบบจำลองตามค่า Loss Function ของภาพนั้น ๆ และมีความเหมาะสมกับฐานข้อมูลขนาดใหญ่ เพราะจะทำให้ค่า Loss Function ลดลงเร็วกว่า

Back Propagation : Output Layer



จุดประสงค์ คือ “เราจะใช้ค่าความผิดพลาดทั้งหมดเพื่อใช้ในการปรับค่าถ่วงน้ำหนัก ณ ตำแหน่ง w_5 ”

$\frac{\partial E_{total}}{\partial w_5}$ คือ การเปลี่ยนแปลงของค่าถ่วงน้ำหนักตัวที่ 5 (Weight) ที่ส่งผลต่อค่าความผิดพลาด (Error) จากรูปนักศึกษาคงพบว่าการที่จะใช้ค่าความผิดพลาดเพื่อปรับปรุงค่าถ่วงน้ำหนักนั้นไม่สามารถคำนวณได้โดยตรง ซึ่งจะพบว่า



ค่าความผิดพลาด (E_{total})



ผลลัพธ์ของเครือข่าย (out_{o1})



ผลลัพธ์ของเลเยอร์ก่อนหน้า ($net_{o1} = h1 \cdot w_5$)

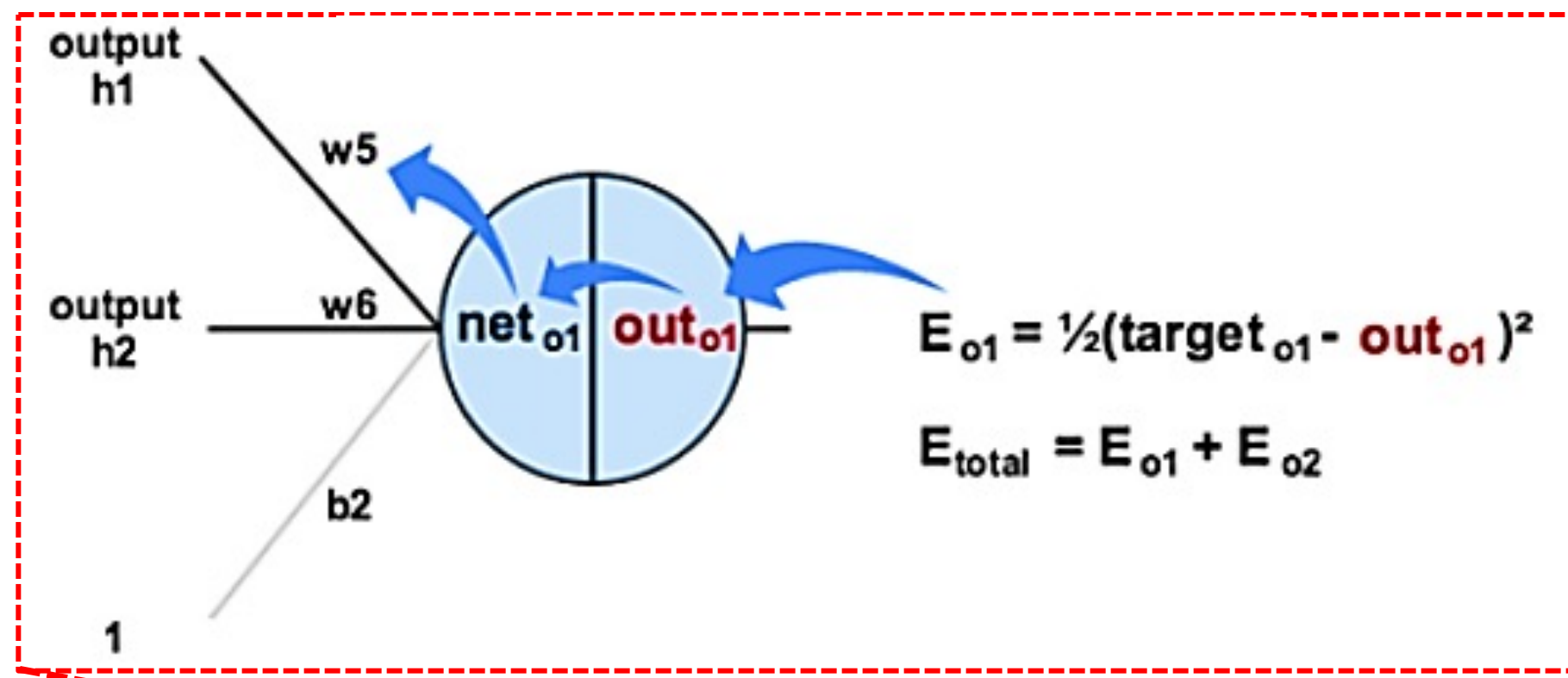


ค่าถ่วงน้ำหนัก (w_5)

ซึ่งจากกฎของ Chain rule จะสามารถเขียนเป็นสมการคณิตศาสตร์ได้ดังนี้

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5} = \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

Back Propagation : Output Layer



ซึ่งจากกฎของ Chain rule จะสามารถเขียนเป็นสมการคณิตศาสตร์ได้ดังนี้

$$\frac{\partial E_{\text{total}}}{\partial w_5} = \frac{\partial E_{o1}}{\partial \text{out}_{o1}} * \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} * \frac{\partial \text{net}_{o1}}{\partial w_5}$$

$$\frac{\partial}{\partial \text{out}_{o1}} \left(\frac{1}{2} (L_1 - \text{out}_{o1})^2 \right) = -(L_1 - \text{out}_{o1})$$

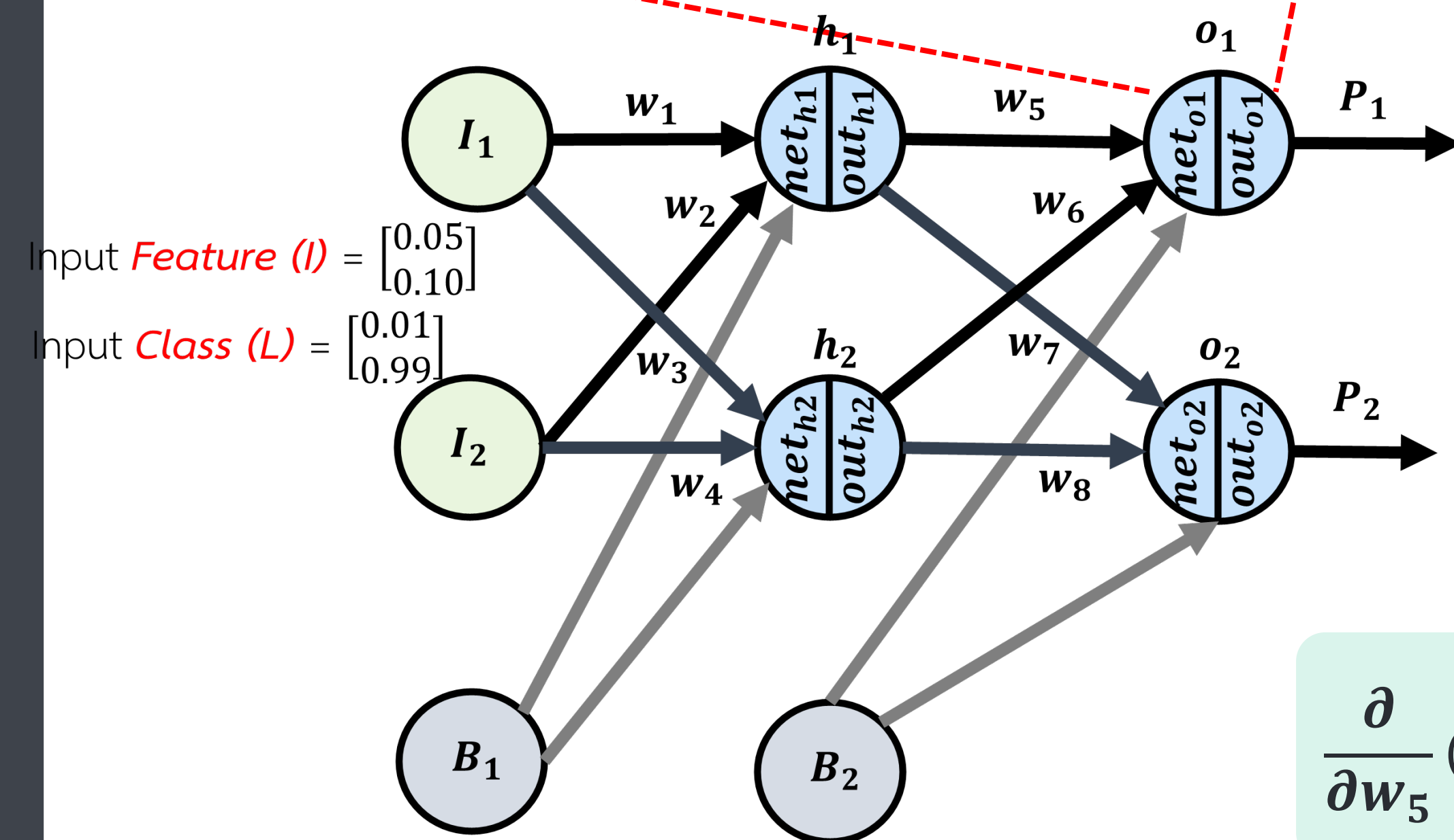
Diff ตัว Error

$$\frac{\partial}{\partial \text{net}_{o1}} \left(\frac{1}{1 + e^{-\text{net}_{o1}}} \right) = \text{out}_{o1} (1 - \text{out}_{o1})$$

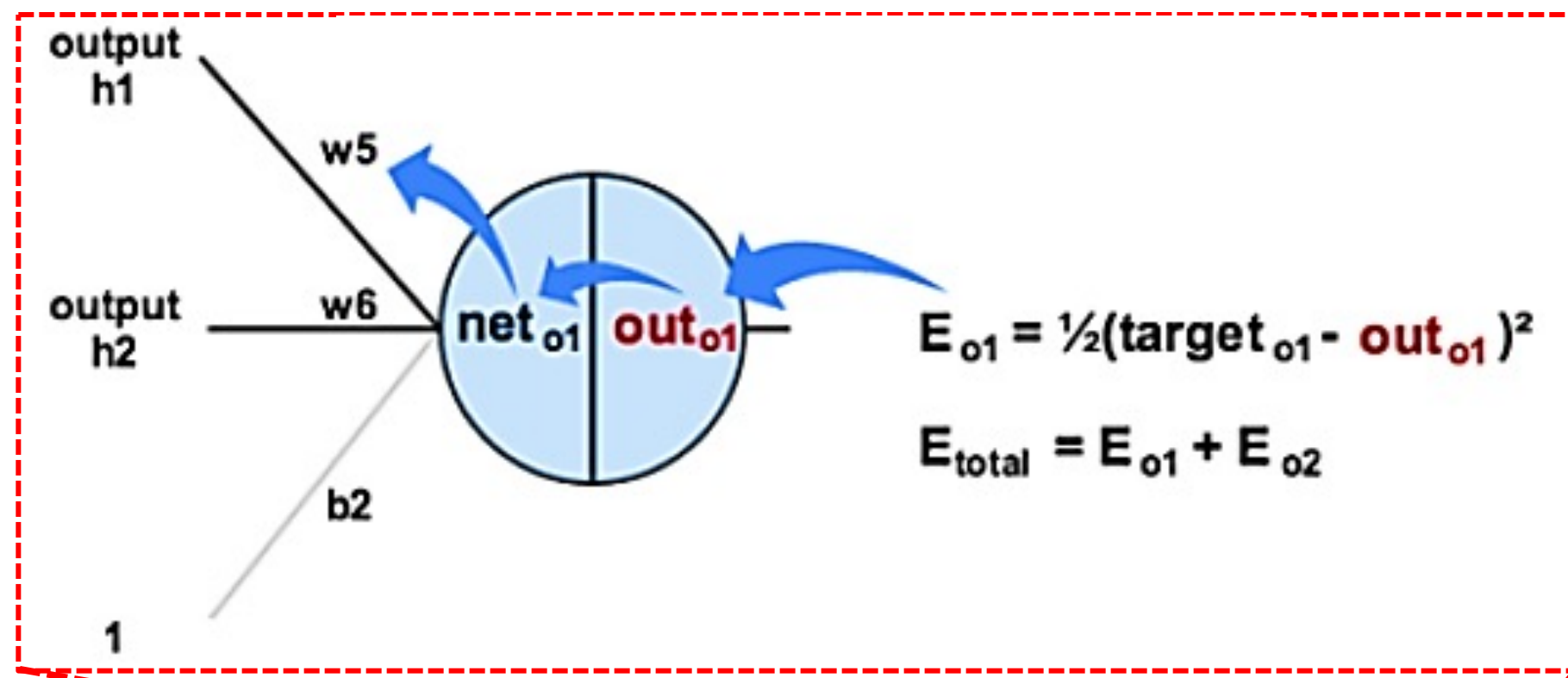
Diff ตัว Activation Function

*สมมติใช้สมการ Logistic Function: $\text{out}_{o1} = f_L(\text{net}_{o1}) = \frac{1}{1 + e^{-\text{net}_{o1}}}$

$$\frac{\partial}{\partial w_5} (w_5 \cdot \text{out}_{h1} + w_6 \cdot \text{out}_{h2} + b_2) = 1 \cdot w_5^{1-1} \cdot \text{out}_{h1} + 0 + 0 = \text{out}_{h1}$$



Back Propagation : Output Layer



ซึ่งจากกฎของ Chain rule จะสามารถเขียนเป็นสมการคณิตศาสตร์ได้ดังนี้

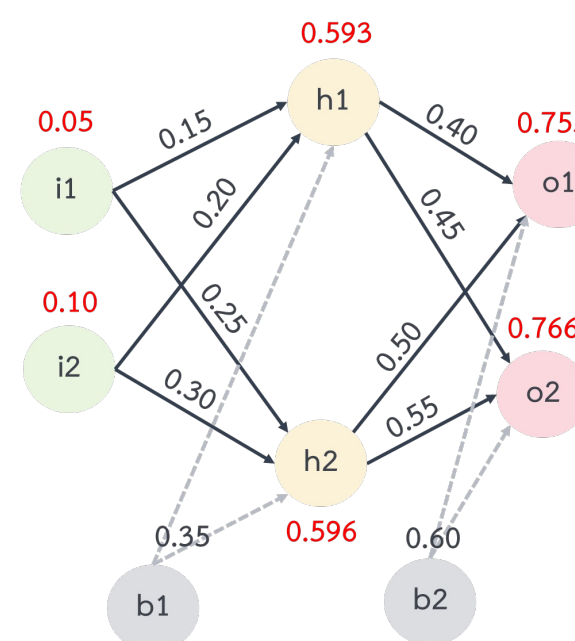
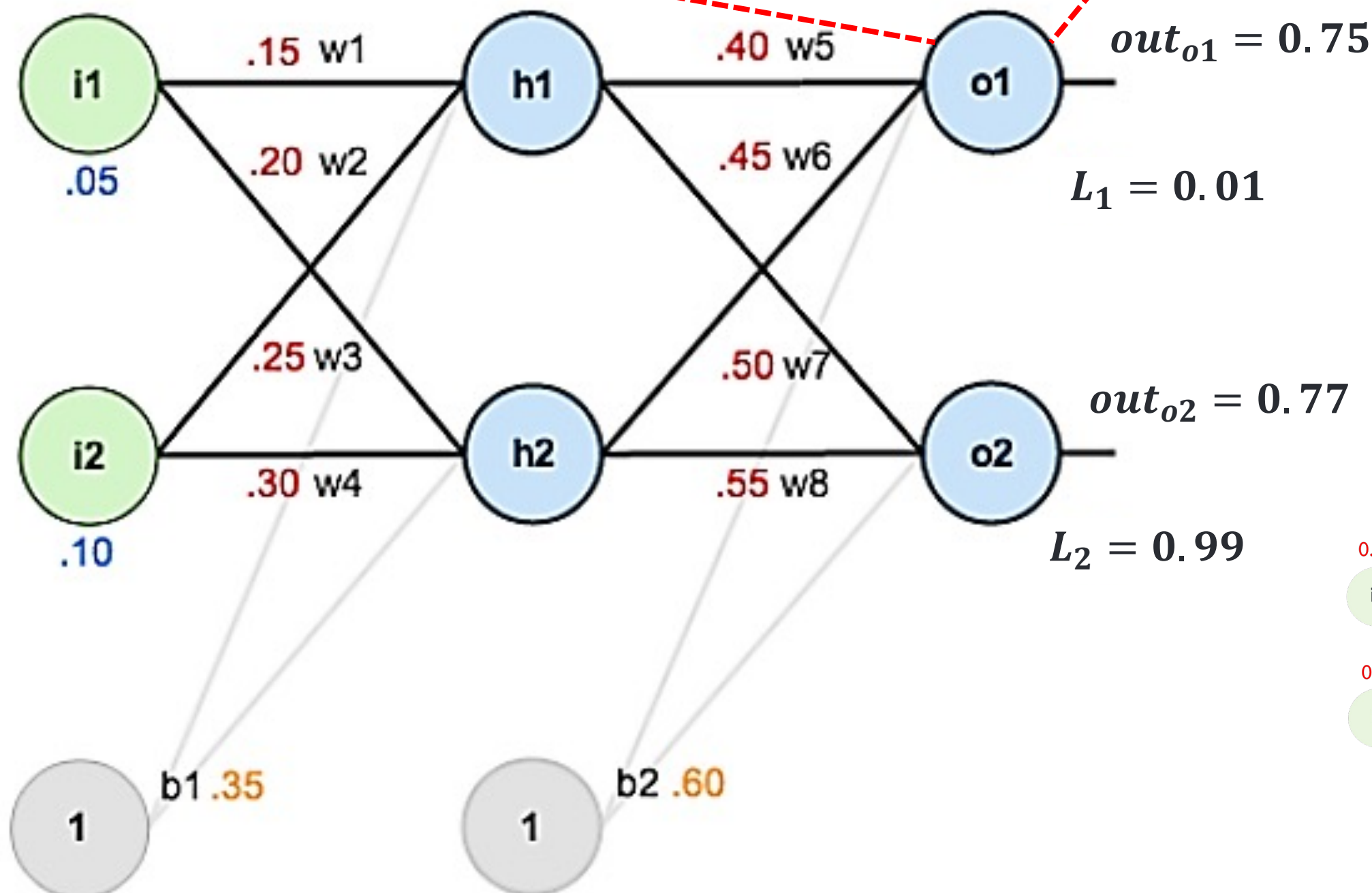
$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

$$\frac{\partial E_{total}}{\partial w_5} = [-(L_1 - out_{o1})] * [out_{o1}(1 - out_{o1})] * [out_{h1} \cdot f_L(i_1 \cdot w_1 + i_2 \cdot w_2 + b_1)]$$

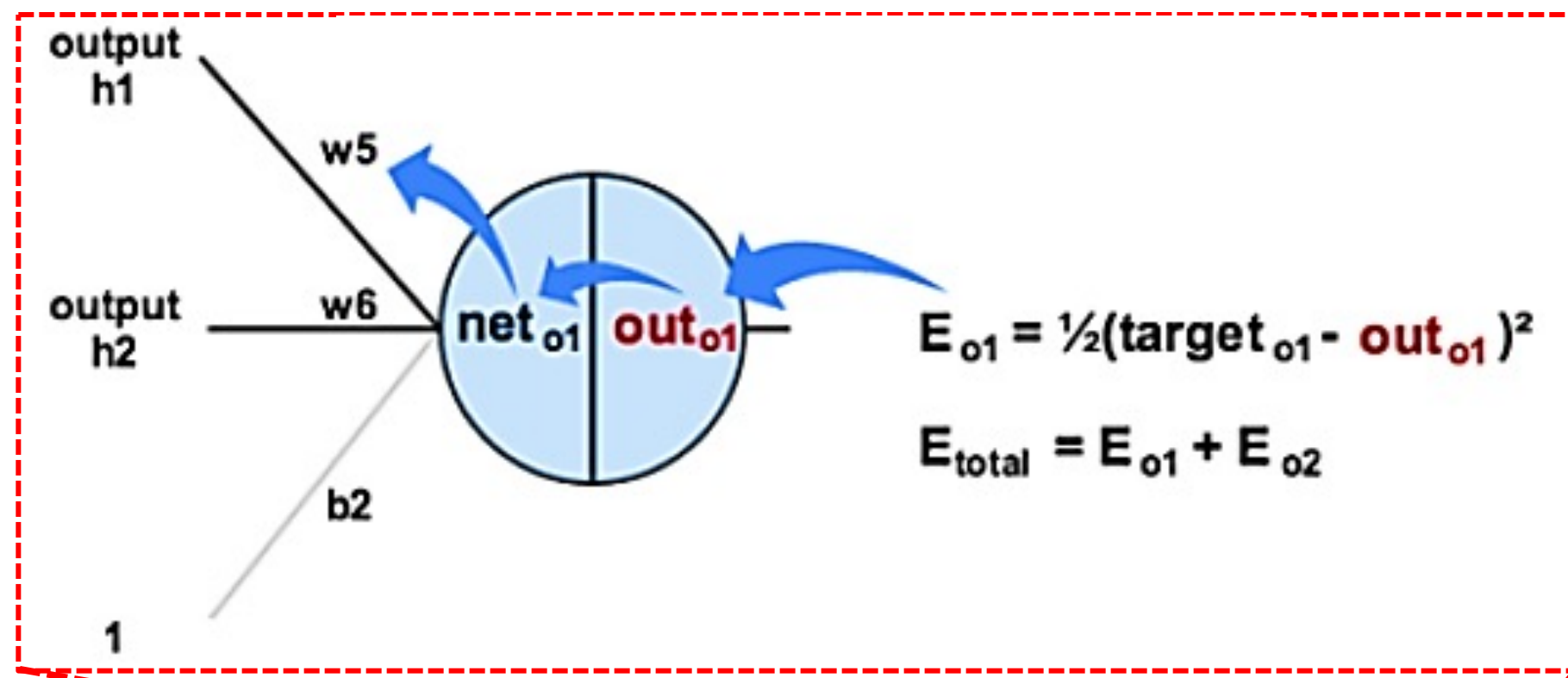
$$\frac{\partial E_{total}}{\partial w_5} = [-(0.01 - 0.75)] * [0.75(1 - 0.75)] * [0.59]$$

$$\frac{\partial E_{total}}{\partial w_5} = [0.74] * [0.19] * [0.59]$$

$$\frac{\partial E_{total}}{\partial w_5} = 0.082954$$



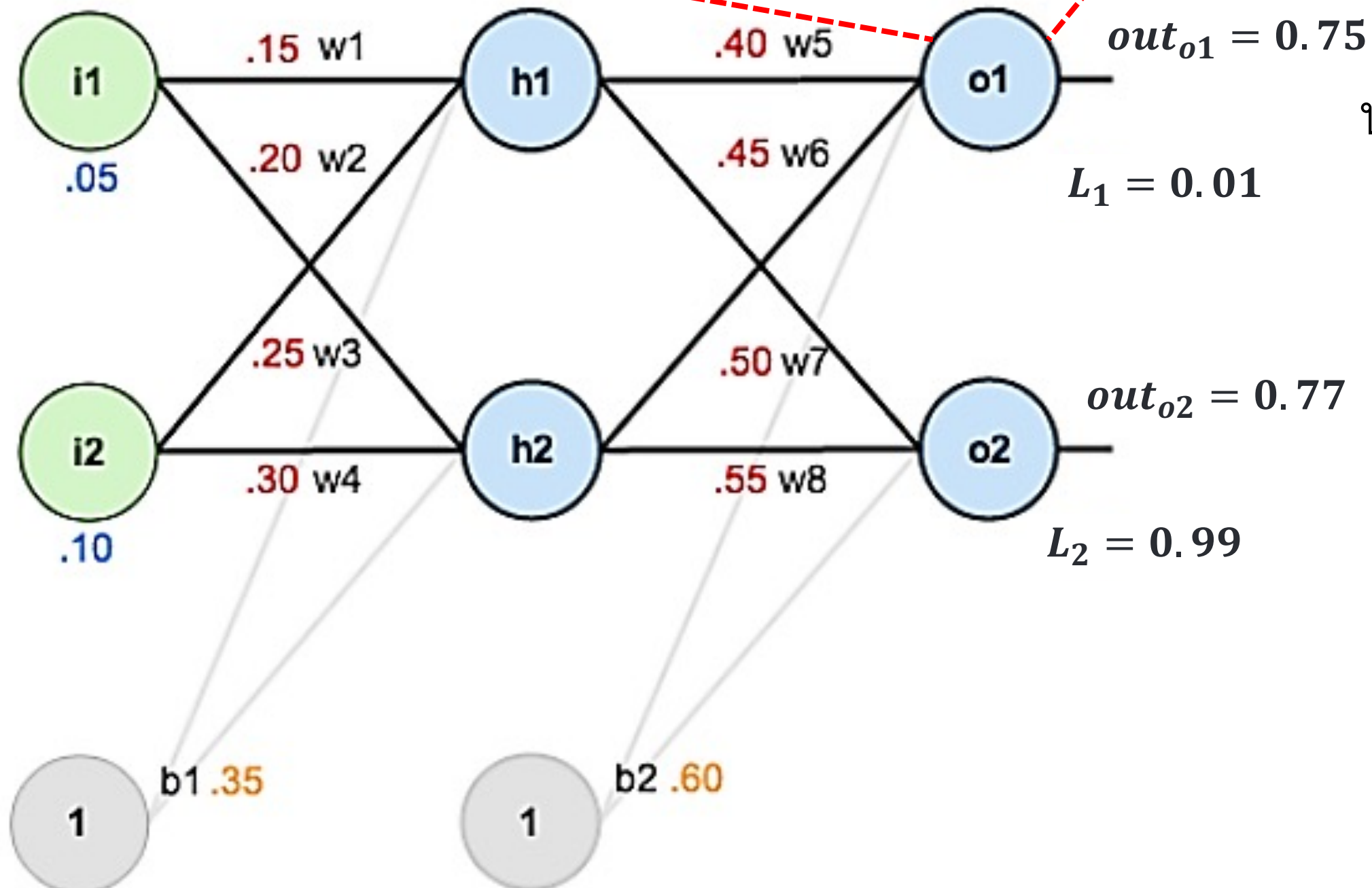
Back Propagation : Output Layer



$$\therefore \frac{\partial E_{\text{total}}}{\partial w_5} = 0.082954$$

นำค่าที่ได้ไปปรับปรุงค่าถ่วงน้ำหนัก w_5 จากสมการต่อไปนี้

$$^*w_5 = w_5 - a \left(\frac{\partial E_{\text{total}}}{\partial w_5} \right)$$

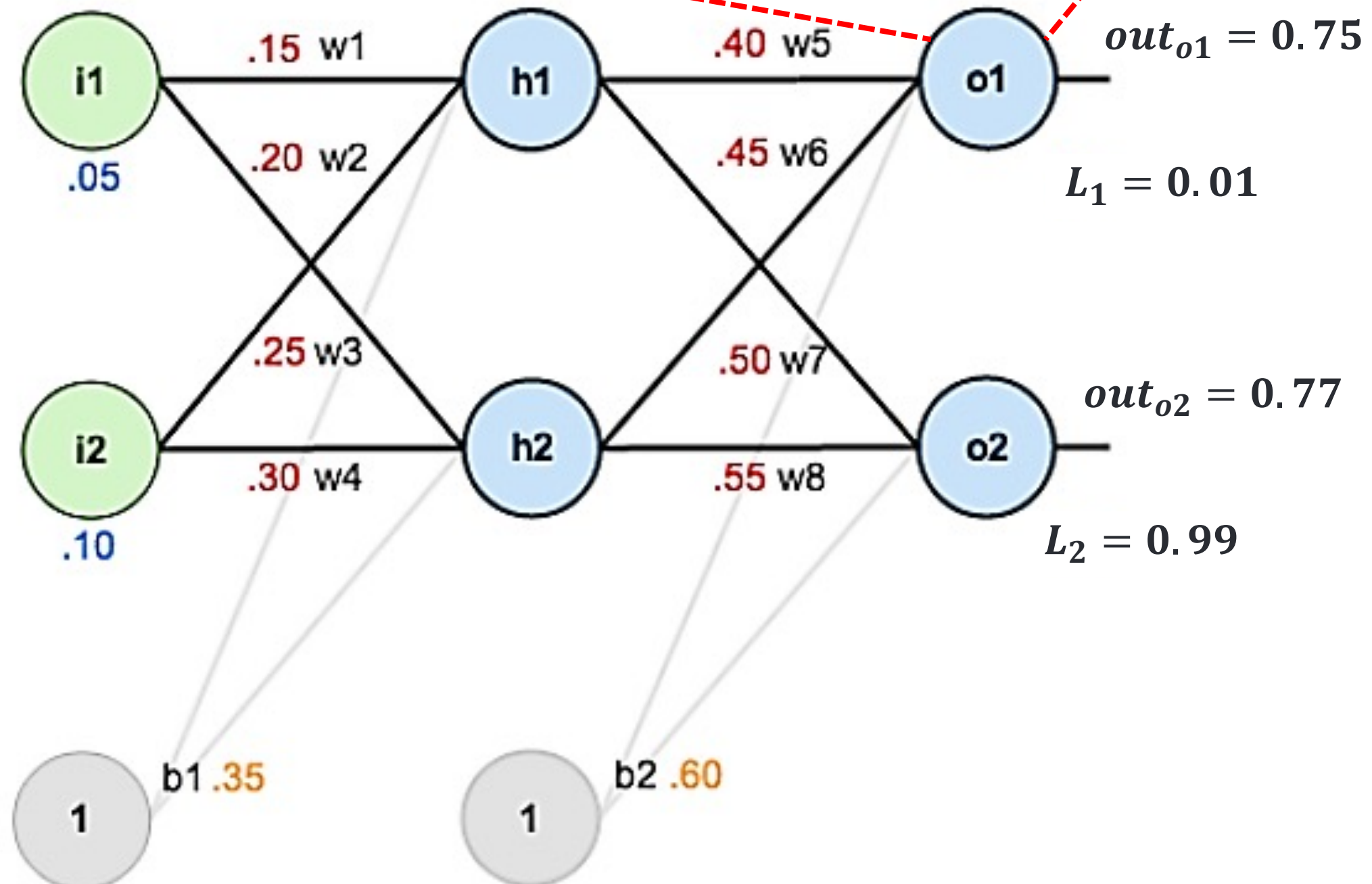
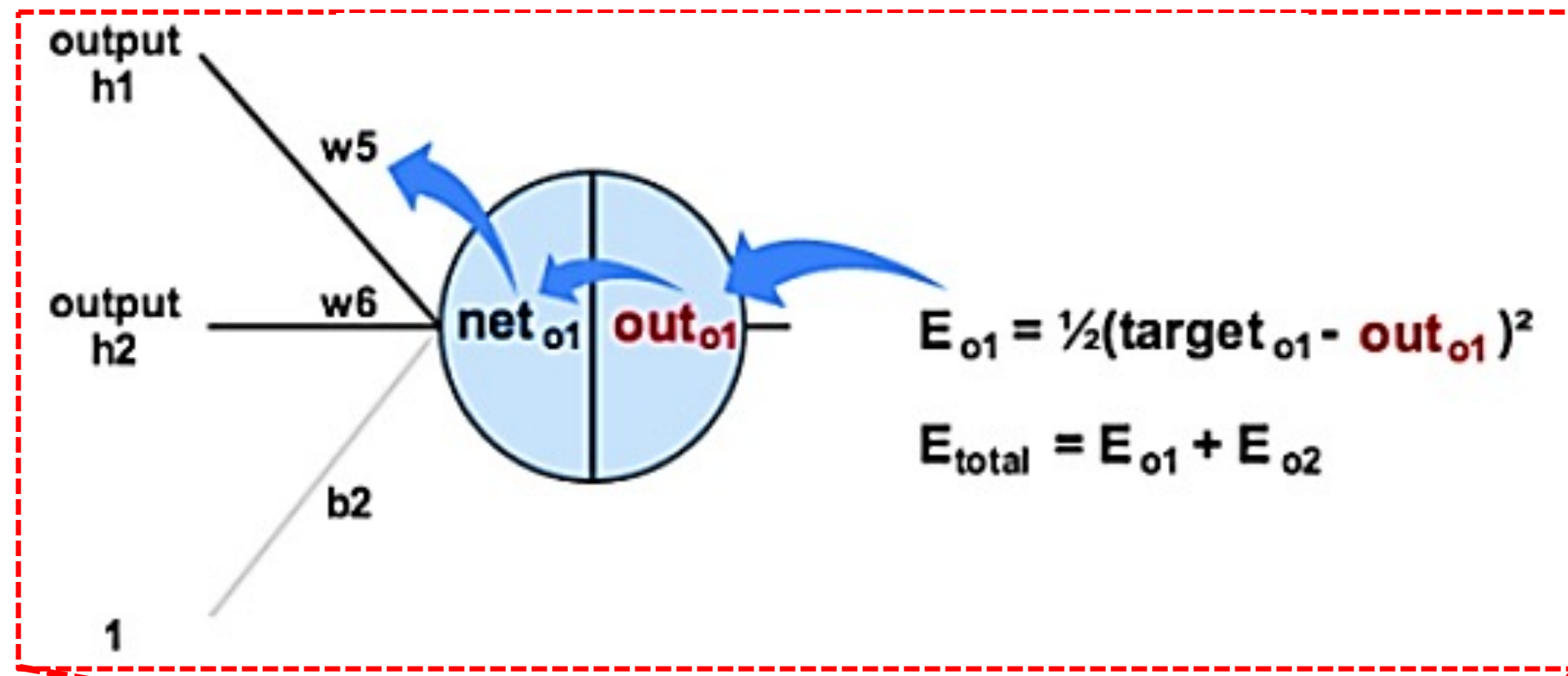


นำค่าถ่วงน้ำหนัก w_5 ไปแทนในสมการข้างต้น นอกจากนี้ กำหนดให้ *Learning rate* คือ 0.5

$$^*w_5 = 0.40 - (0.5)(0.082954)$$

$$^*w_5 = 0.358916$$

แบบฝึกหัด



ให้นักศึกษาลองคำนวณหาค่าถ่วงน้ำหนักต่อไปนี้ โดยกำหนดให้ Learning rate คือ 0.5

$$*w_6 = w_6 - a \left(\frac{\partial E_{\text{total}}}{\partial w_6} \right)$$

$$*w_7 = w_7 - a \left(\frac{\partial E_{\text{total}}}{\partial w_7} \right)$$

$$*w_8 = w_8 - a \left(\frac{\partial E_{\text{total}}}{\partial w_8} \right)$$

เฉลย

$$*w_6 \approx 0.41$$

$$*w_7 \approx 0.51$$

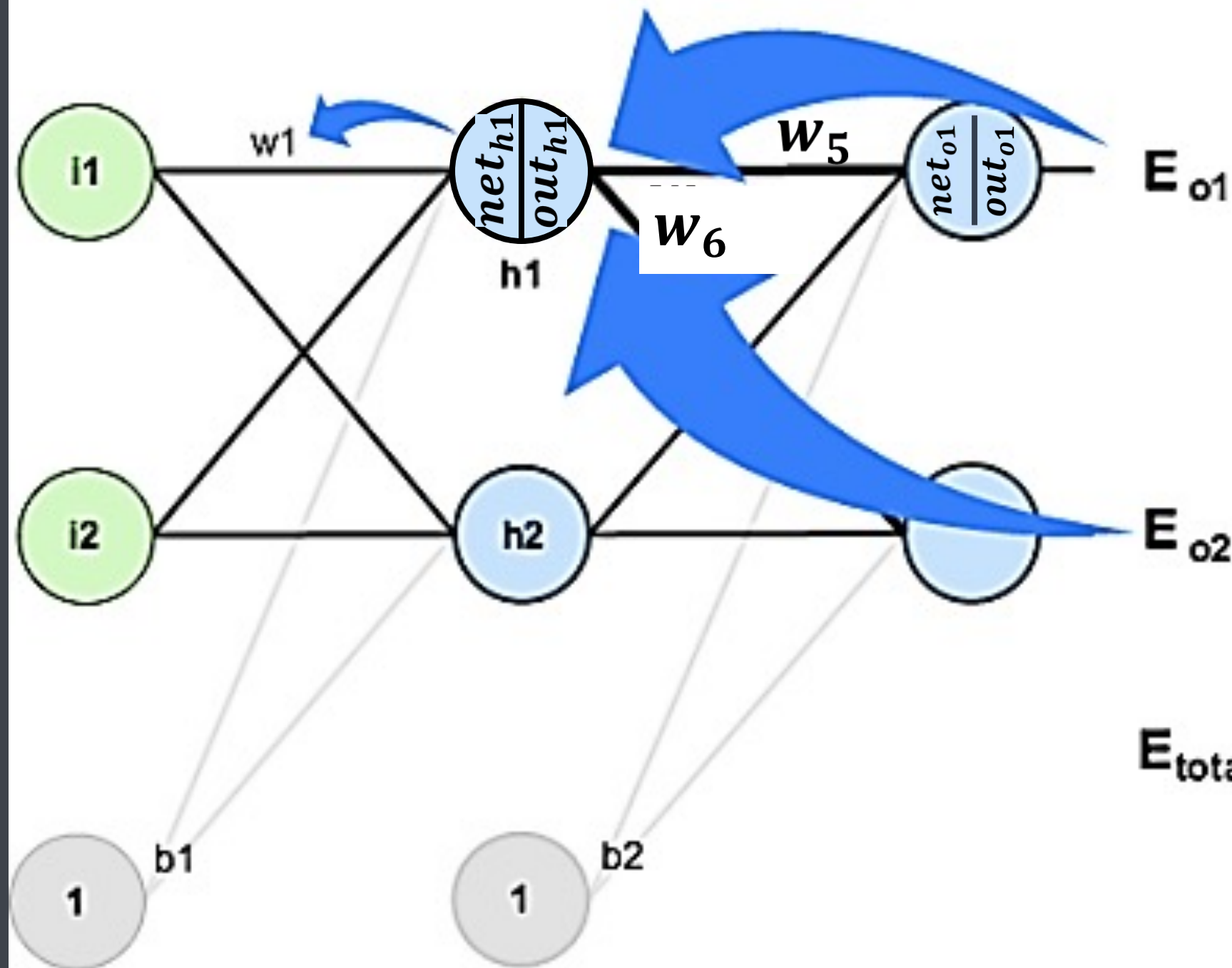
$$*w_8 \approx 0.56$$

Back Propagation : Hidden Layer

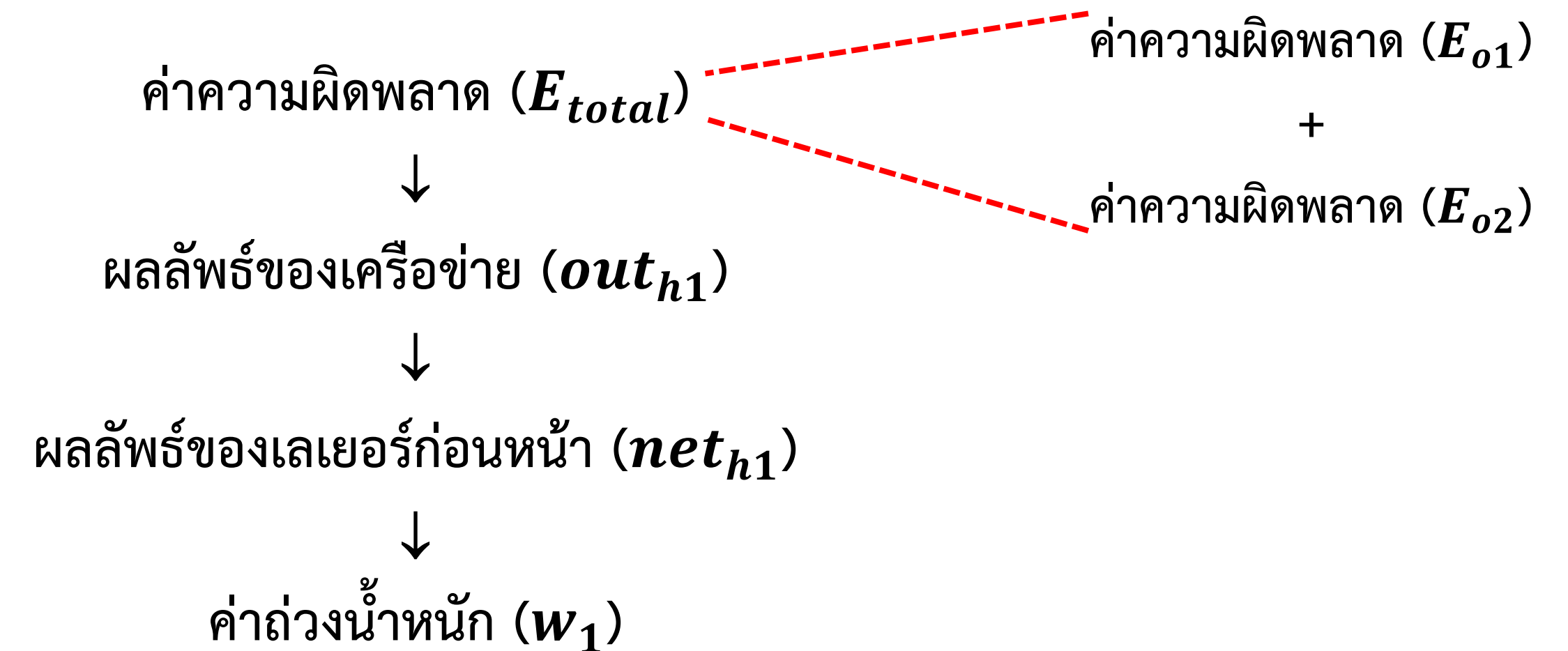
ความต้องการ คือ “เราจะใช้ค่าความผิดพลาดทั้งหมดเพื่อใช้ในการปรับค่าถ่วงน้ำหนัก ณ ตำแหน่ง w_1 ”

$\frac{\partial E_{total}}{\partial w_1}$ คือ การเปลี่ยนแปลงของค่าถ่วงน้ำหนักตัวที่ 1 ที่ส่งผลต่อค่าความผิดพลาด

จากรูปนักศึกษาจะพบว่าการใช้ค่าความผิดพลาดเพื่อปรับปรุงค่าถ่วงน้ำหนักนั้นไม่สามารถคำนวณได้โดยตรง ซึ่งจะพบว่า



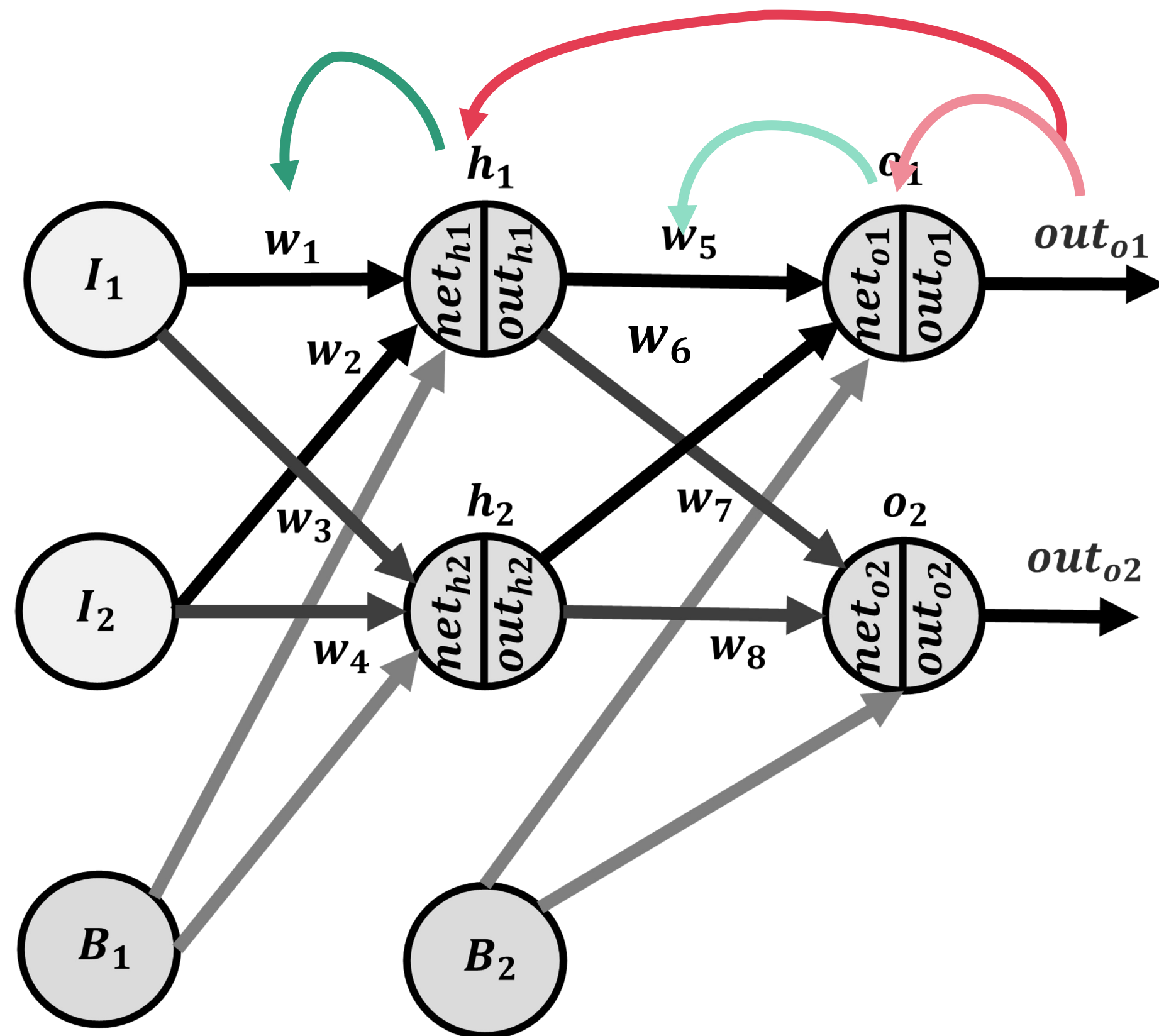
$$E_{total} = E_{o1} + E_{o2}$$



ซึ่งจากกฎของ Chain rule จะสามารถเขียนเป็นสมการคณิตศาสตร์ได้ดังนี้

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1} = \left(\frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}} \right) * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

Back Propagation



ซึ่งจากกฎของ Chain rule จะสามารถเขียนเป็นสมการคณิตศาสตร์ได้ดังนี้

Output Layer

เชื่อมระหว่าง source node ไป destination node

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

เชื่อมระหว่าง
destination
node ไป
ค่าถ่วงน้ำหนัก

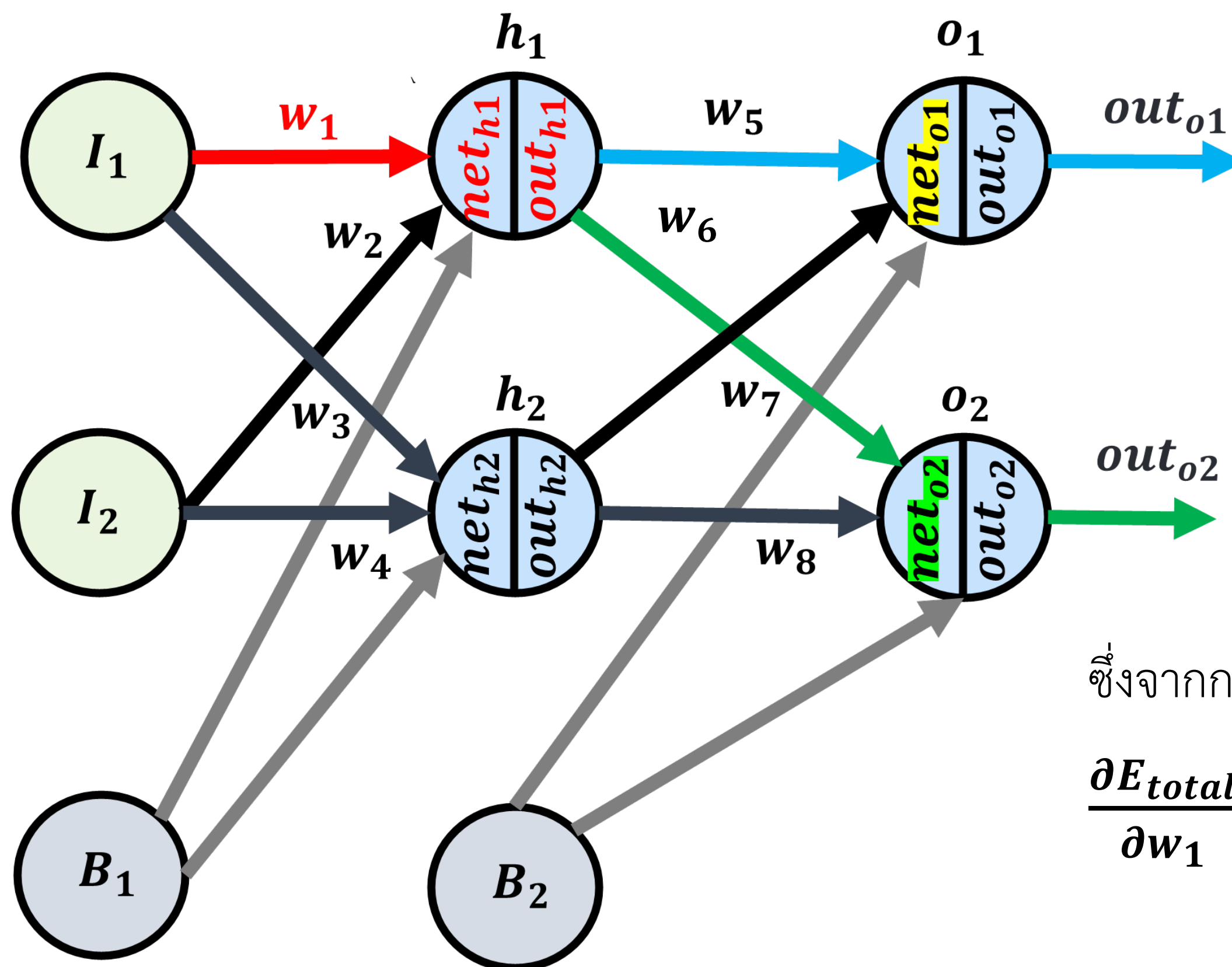
Hidden Layer

$$\frac{\partial E_{total}}{\partial w_1} = \left(\frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}} \right) * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\delta = \frac{\partial E_{total}}{\partial net}$$

$$\frac{\partial net}{\partial w}$$

Back Propagation : Hidden Layer



$$E_{o1} = \frac{1}{2} (L_1 - out_{o1})^2$$

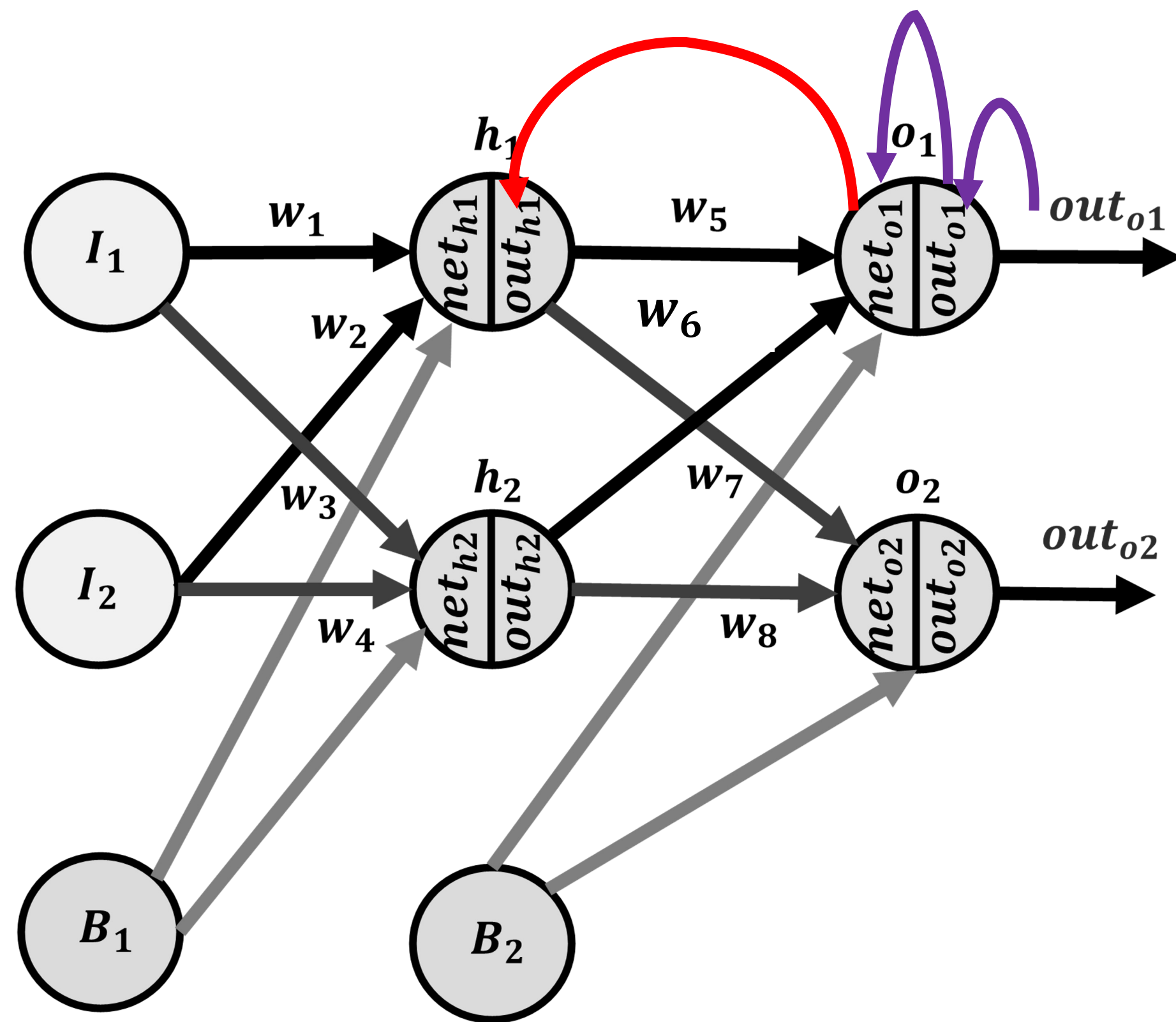
$$E_{o2} = \frac{1}{2} (L_2 - out_{o2})^2$$

$$E_{total} = E_{o1} + E_{o2}$$

ซึ่งจากกฎของ Chain rule จะสามารถเขียนเป็นสมการคณิตศาสตร์ได้ดังนี้

$$\begin{aligned} \frac{\partial E_{total}}{\partial w_1} &= \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1} \\ &= \left(\frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}} \right) * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1} \\ &= \left[\left(\frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}} \right) + \left(\frac{\partial E_{o2}}{\partial net_{o2}} * \frac{\partial net_{o2}}{\partial out_{h1}} \right) \right] * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1} \end{aligned}$$

Back Propagation : Hidden Layer



ต่อ

$$\frac{\partial E_{total}}{\partial w_1} = \left[\left(\frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}} \right) + \left(\frac{\partial E_{o2}}{\partial net_{o2}} * \frac{\partial net_{o2}}{\partial out_{h1}} \right) \right] * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$= \left[\left(\frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}} \right) + \left(\frac{\partial E_{o2}}{\partial net_{o2}} * \frac{\partial net_{o2}}{\partial out_{h1}} \right) \right] * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

↓

$(w_5 * out_{h1}) + (6 * out_{h2}) + b_2$ // พอ diff แล้วได้ w_5

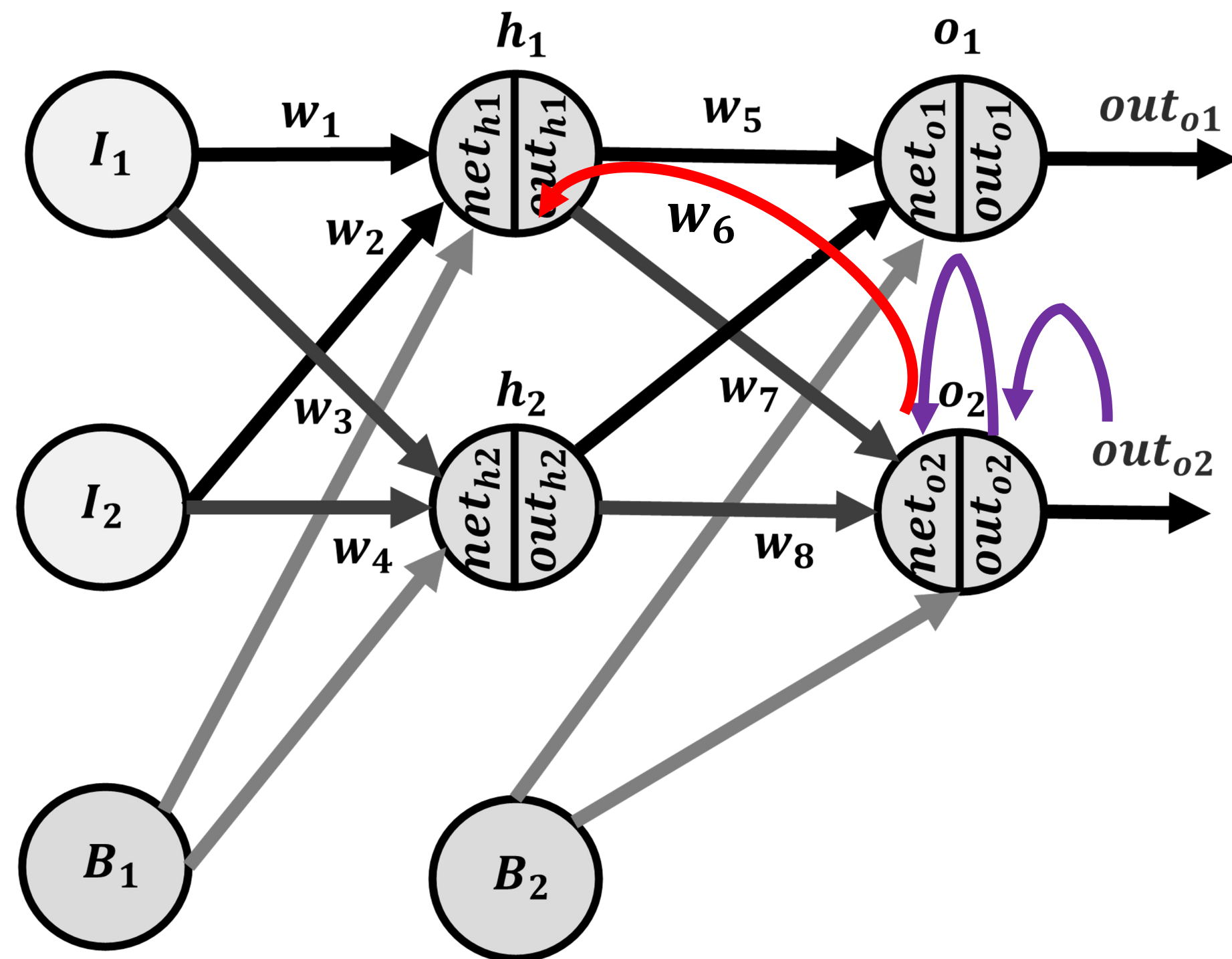
↓

$[-(L_1 - out_{o1})] * [out_{o1}(1 - out_{o1})]$

$$= \left[(0.74 * 0.19 * 0.4) + \left(\frac{\partial E_{o2}}{\partial net_{o2}} * \frac{\partial net_{o2}}{\partial out_{h1}} \right) \right] * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$= \left[(0.05624) + \left(\frac{\partial E_{o2}}{\partial net_{o2}} * \frac{\partial net_{o2}}{\partial out_{h1}} \right) \right] * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

Back Propagation : Hidden Layer



ต่อ

$$\frac{\partial E_{total}}{\partial w_1} = \left[\left(\frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}} \right) + \left(\frac{\partial E_{o2}}{\partial net_{o2}} * \frac{\partial net_{o2}}{\partial out_{h1}} \right) \right] * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$= 0.05624 + \left(\frac{\partial E_{o2}}{\partial out_{o2}} * \frac{\partial out_{o2}}{\partial net_{o2}} * \frac{\partial net_{o2}}{\partial out_{h1}} \right)$$

$$[-(L_2 - out_{o2})] * [out_{o2}(1 - out_{o2})]$$

$$(w_5 * out_{h1}) + (w_6 * out_{h2}) + b_2$$

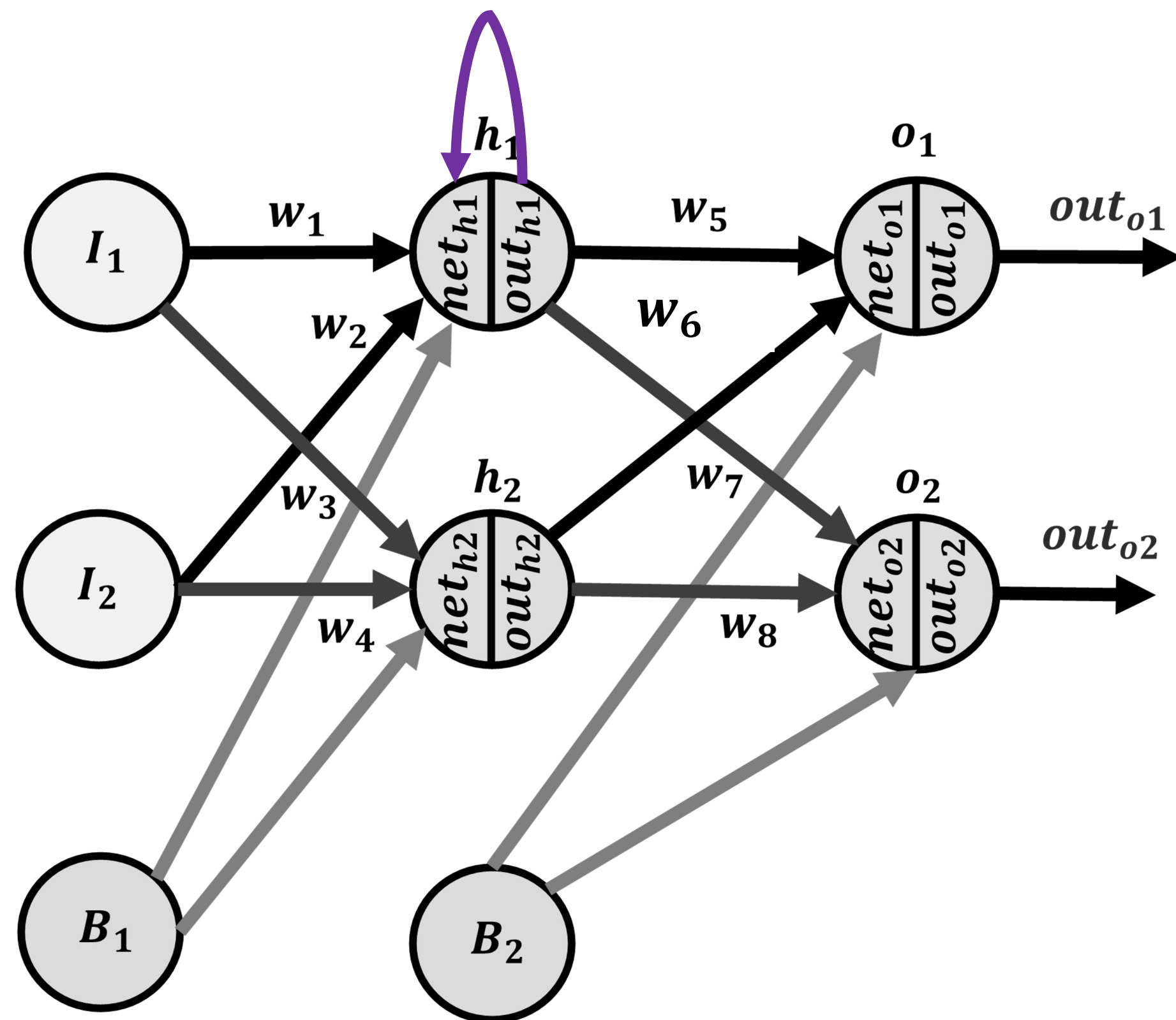
// พอ diff แล้วได้ w_6

$$= 0.05624 + (-0.22 * 0.1771 * 0.45)$$

$$= 0.05624 + (-0.0194) = 0.03684$$

$$\therefore \frac{\partial E_{total}}{\partial w_1} = 0.03684 * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

Back Propagation : Hidden Layer



$$\therefore \frac{\partial E_{total}}{\partial w_1} = 0.03684$$

$$* \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial}{\partial net_{h1}} \left(\frac{1}{1 + e^{-net_{h1}}} \right) = out_{h1}(1 - out_{h1})$$

$$= 0.593(1 - 0.593)$$

$$= 0.2413$$

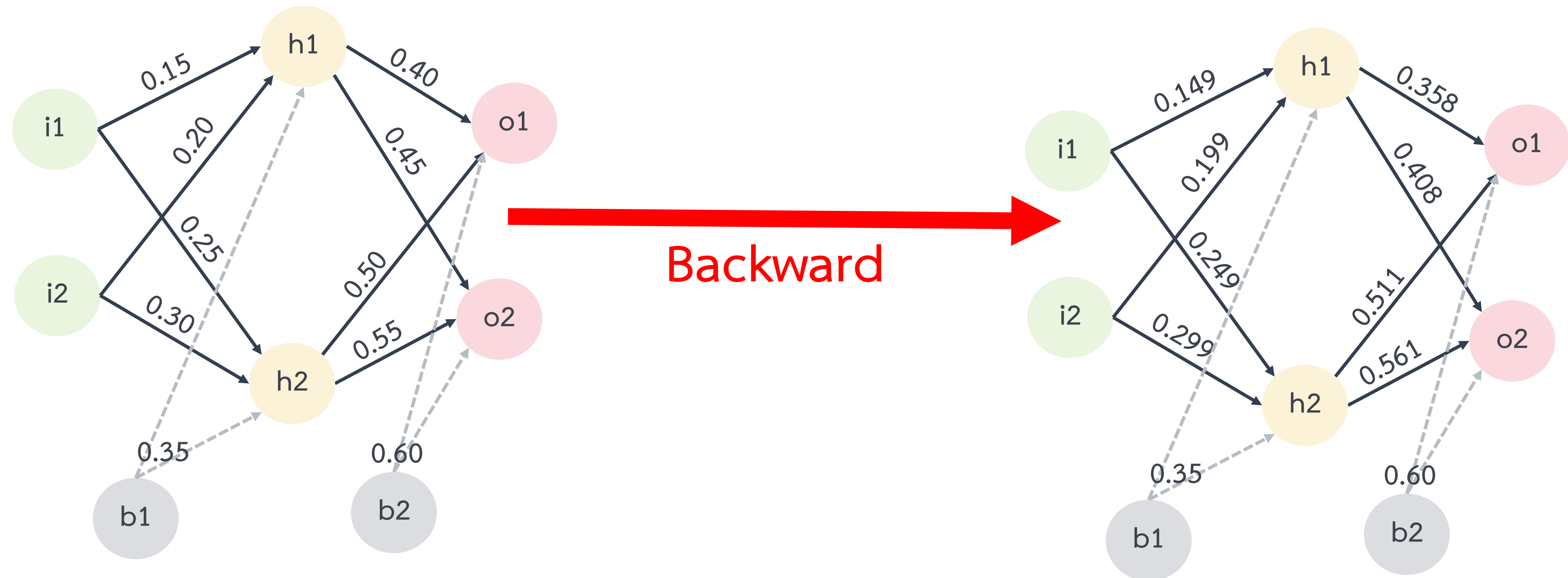
$$\frac{\partial net_{h1}}{\partial w_1} = I_1 = 0.05$$

$$\therefore \frac{\partial E_{total}}{\partial w_1} = 0.03684$$

$$* 0.2413 * 0.05 = 0.0004$$

ดังนั้น $* w_1 = w_1 - a \left(\frac{\partial E_{total}}{\partial w_1} \right) = 0.15 - 0.5(0.0004) = 0.1498$

Back Propagation : Hidden Layer



องค์ประกอบที่ควรรู้สำหรับการฝึกสอนแบบจำลอง



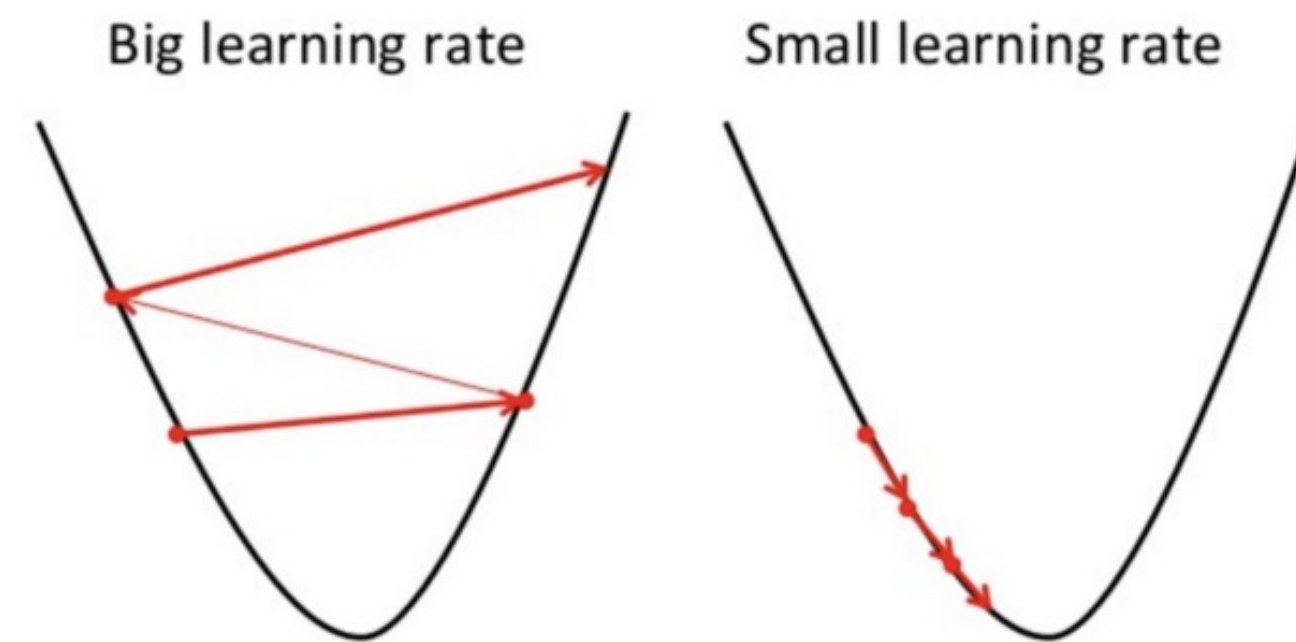
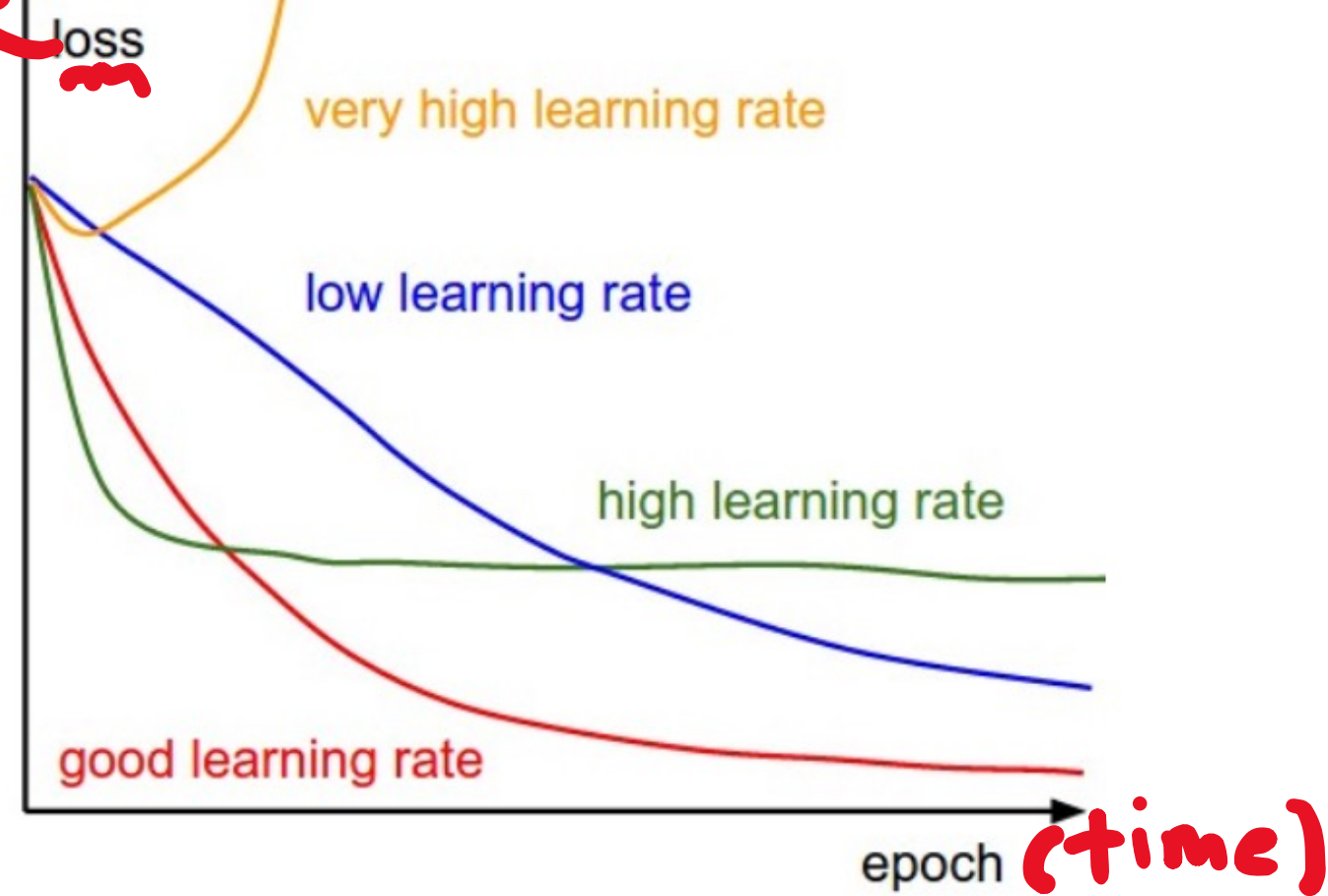
- Loss Function (Objective or Cost Function)
- Optimization
- Learning Rate



Learning Rate (α)

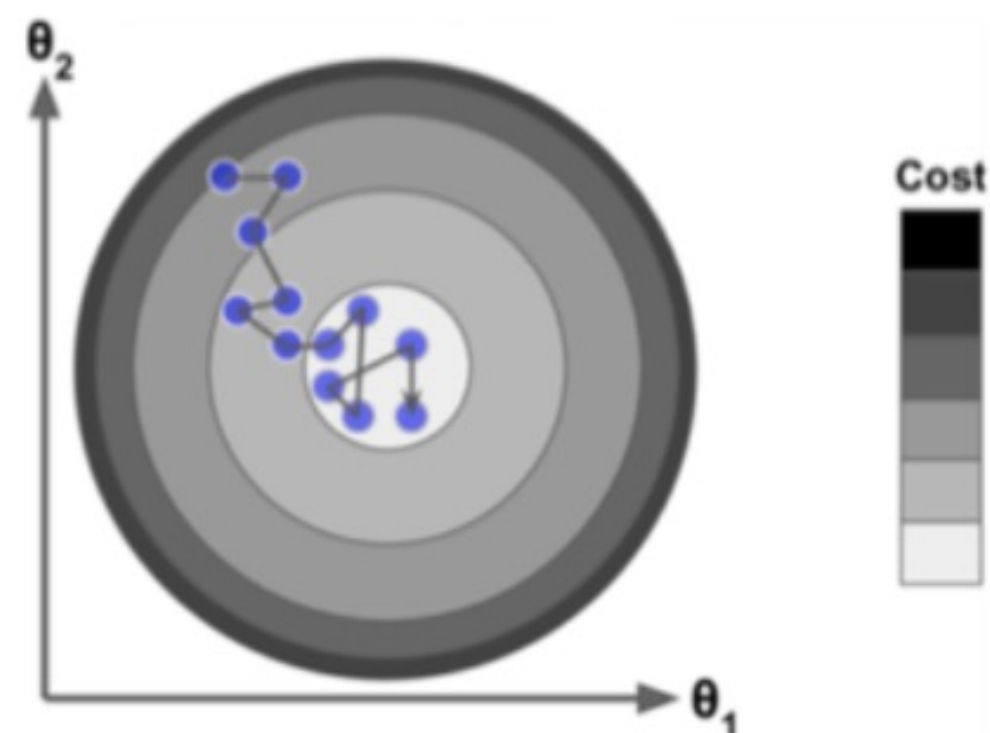
คือ ค่าที่ใช้กำหนดขนาดในการเปลี่ยนแปลงของการเรียนรู้ในแต่ละรอบ

ดู objective function



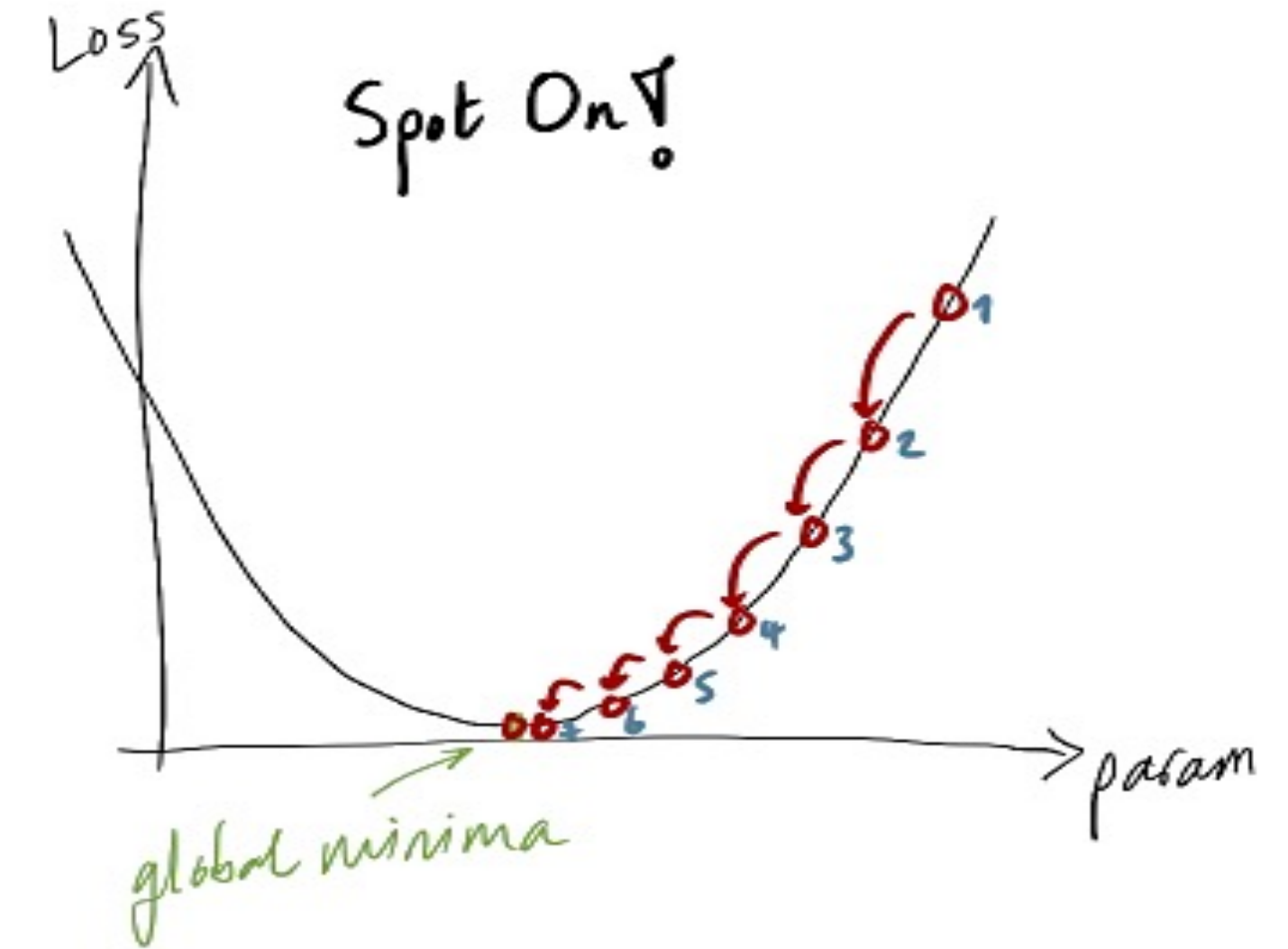
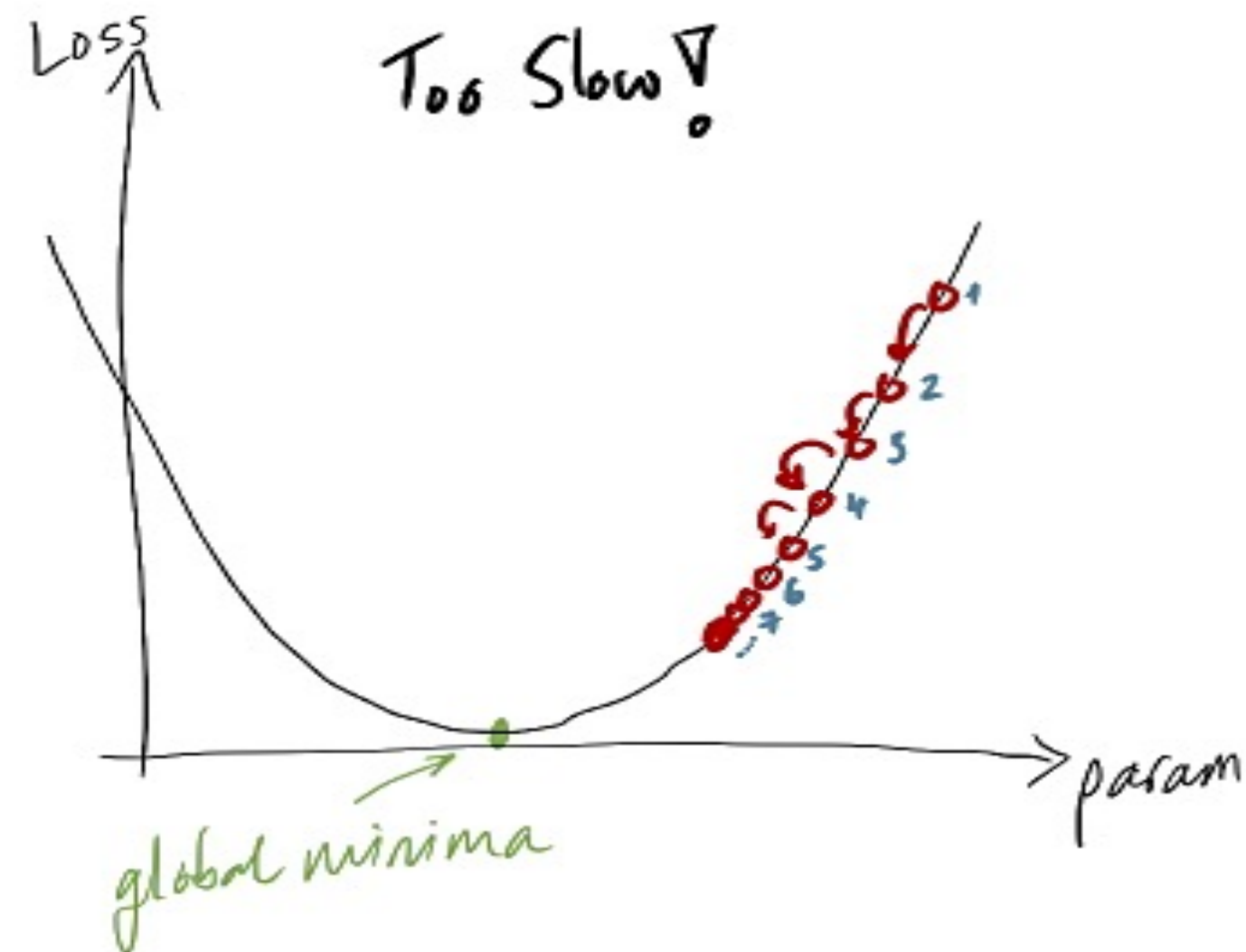
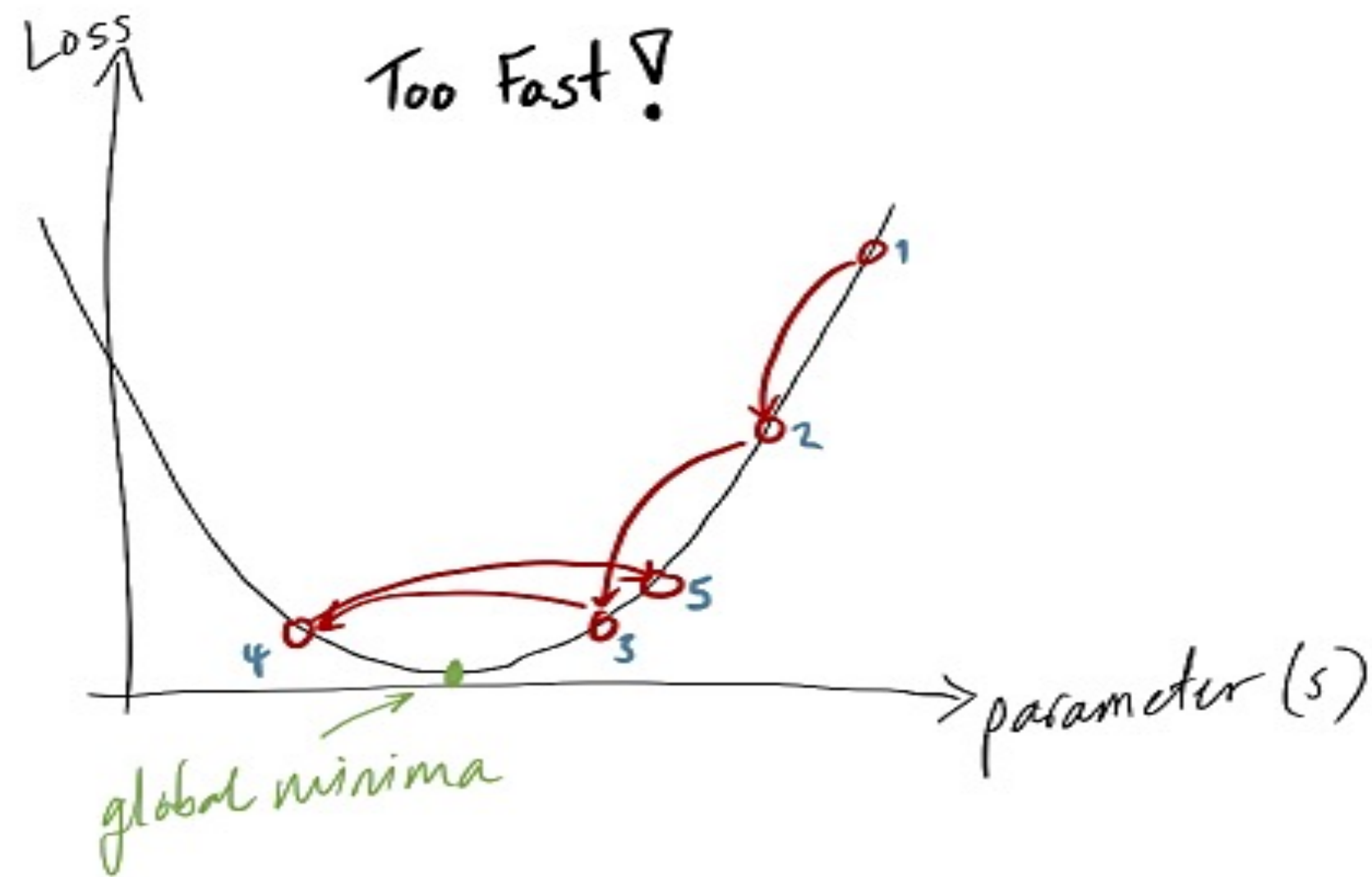
- ถ้า α มีค่าน้อยจะส่งผลให้ใช้เวลาในการประมวลที่นานกว่าค่าของ Objective Function จะเข้าใกล้ค่าต่ำสุด
- ถ้า α มีค่ามากอาจก่อให้เกิดการข้ามหรือไม่พบค่าต่ำสุดหรือค่าใกล้เคียงค่าต่ำสุดของ Objective Function

โดยทั่วไปจะพบว่า Learning rate ที่นิยมในการเรียนรู้ของเครื่องและการเรียนรู้เชิงลึก ได้แก่ 0.001, 0.003, 0.01, 0.03, 0.1, และ 0.3 เป็นต้น



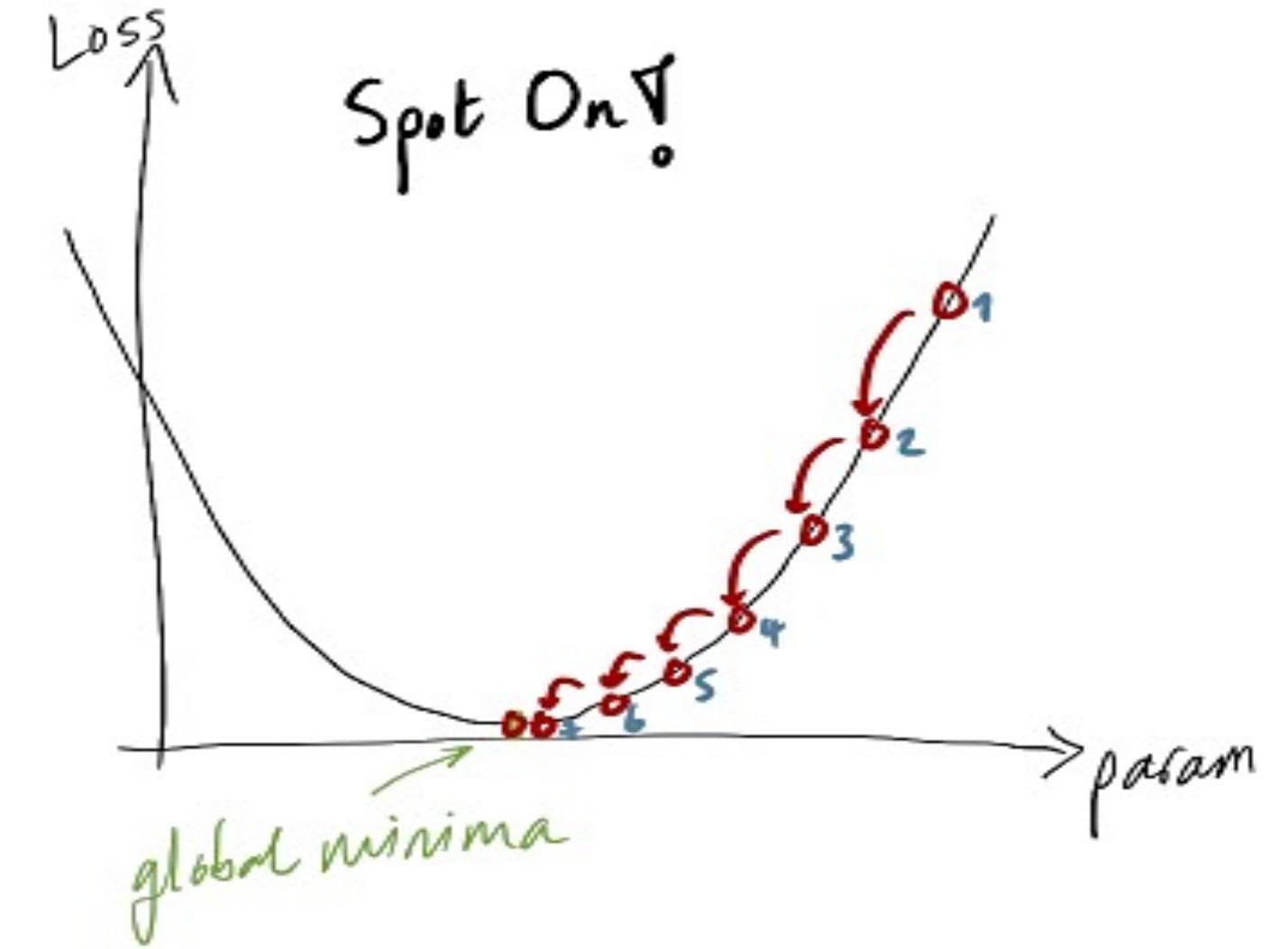
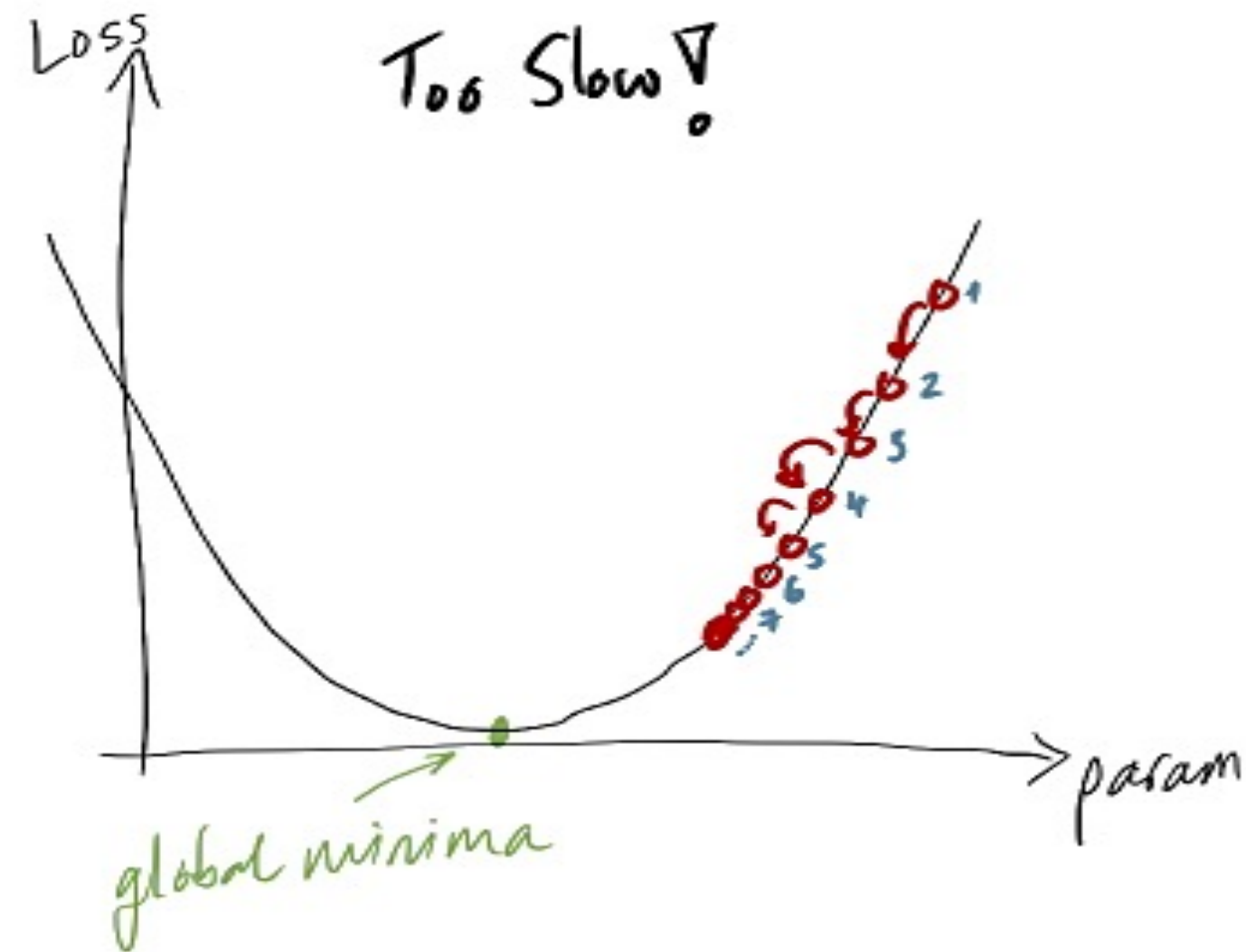
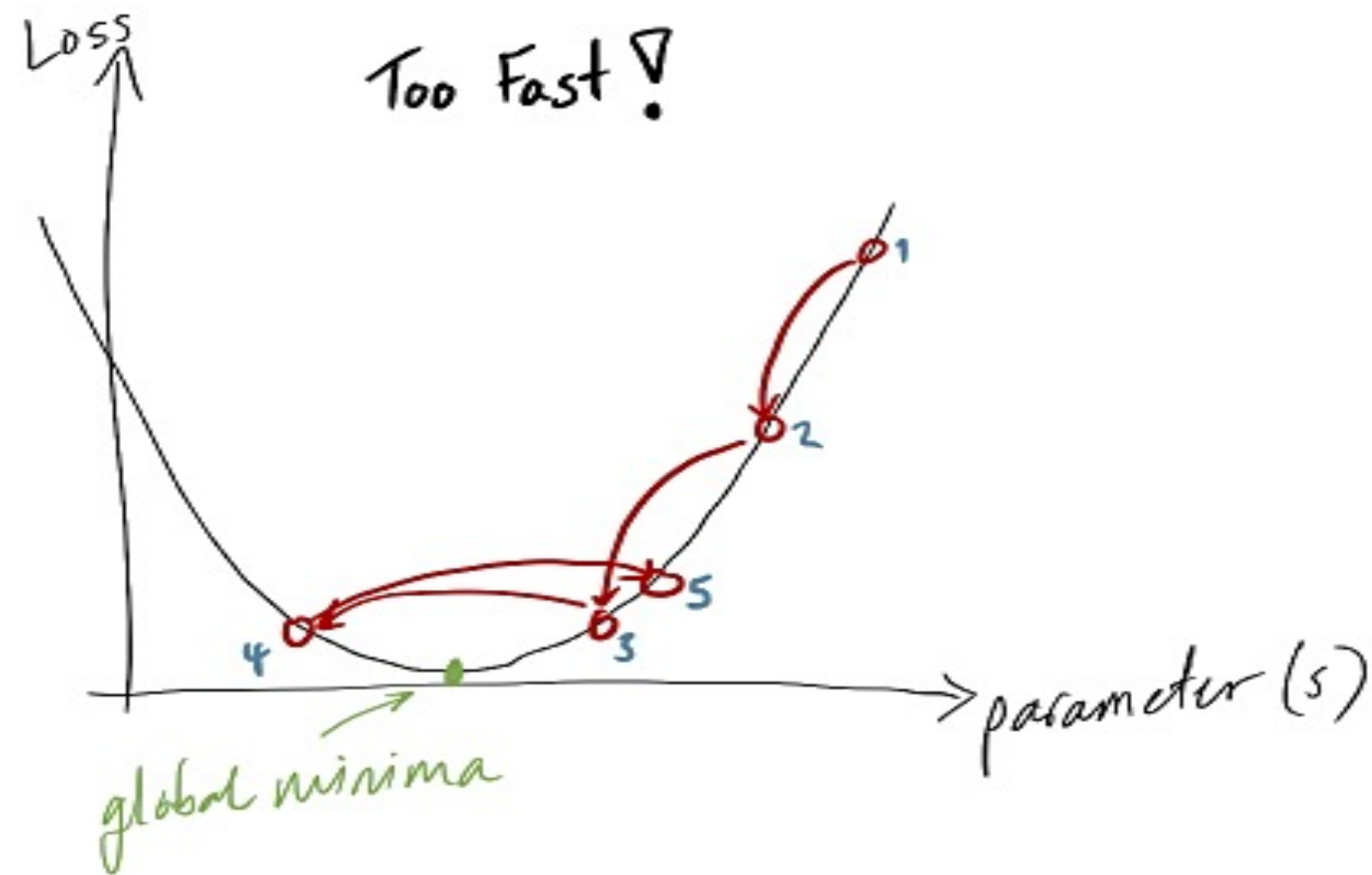
Solution 3: Stochastic GD

Learning Rate & Its Strategies



อัตราการเรียนรู้ (Learning Rate) คือ ไฮเปอร์พารามิเตอร์ที่มีความสำคัญอย่างยิ่ง ในกระบวนการฝึกโมเดล neural network โดยเฉพาะในขั้นตอนการปรับพารามิเตอร์ผ่านเทคนิค Gradient Descent ซึ่งเป็นวิธีการที่ใช้เพื่อลดค่าฟังก์ชันความสูญเสีย (Loss Function)

Learning Rate & Its Strategies



การเลือกค่าของ learning rate มีผลโดยตรงต่อการเรียนรู้

- หากกำหนดค่า Learning Rate สูงเกินไป โมเดลจะอัปเดตพารามิเตอร์แบบก้าวกระโดด ทำให้เรียนรู้เร็วขึ้น แต่เสี่ยงต่อการ “เลยจุดที่ดีที่สุด” (Overshooting) ในทางกลับกัน
- หากกำหนดค่า Learning Rate ต่ำเกินไป การเรียนรู้จะช้าลงมาก และอาจติดอยู่ใน จุดต่ำสุดเฉพาะที่ไม่เหมาะสม (local minima) ดังนั้น การตั้งค่า Learning Rate ที่เหมาะสมจึงเป็นสิ่งที่ต้องสร้างสมดุลระหว่างความเร็วในการฝึกและคุณภาพของผลลัพธ์ที่ได้

Learning Rate & Its Strategies

ความสำคัญของ Learning Rate ไม่ได้มีผลเฉพาะด้านความเร็วในการฝึกเท่านั้น แต่ยังส่งผลต่อความสามารถในการเรียนรู้และการวางตัวของโมเดล ดังนี้

- **ป้องกัน Overfitting** การตั้งค่า Learning Rate ที่เหมาะสมช่วยให้โมเดล ไม่เรียนรู้รายละเอียดของข้อมูลฝึกมากเกินไปจนไม่สามารถใช้งานกับข้อมูลใหม่ได้
- **ป้องกัน Underfitting** ถ้า Learning Rate ต่ำเกินไป โมเดลอาจเรียนรู้ไม่ทันและไม่เข้าใจรูปแบบของข้อมูลจริงอย่างเพียงพอโดยรวมแล้ว Learning Rate ที่เลือกอย่างถูกต้องจะช่วยให้โมเดล เรียนรู้ได้อย่างมีประสิทธิภาพ และสามารถนำไปใช้กับข้อมูลใหม่ได้ดี (generalization)

ไม่มีค่า Learning Rate ที่เหมาะสมเพียงค่าเดียวสำหรับทุกสถานการณ์ ซึ่งการเลือกค่า Learning Rate มักต้องอาศัยการทดลองหลายครั้ง (Trial-and-error) และขึ้นกับ (1) โครงสร้างของโมเดล (2) ลักษณะของข้อมูล และ (3) เทคนิคการปรับพารามิเตอร์ที่ใช้ (เช่น SGD, Adam)

Learning Rate & Its Strategies

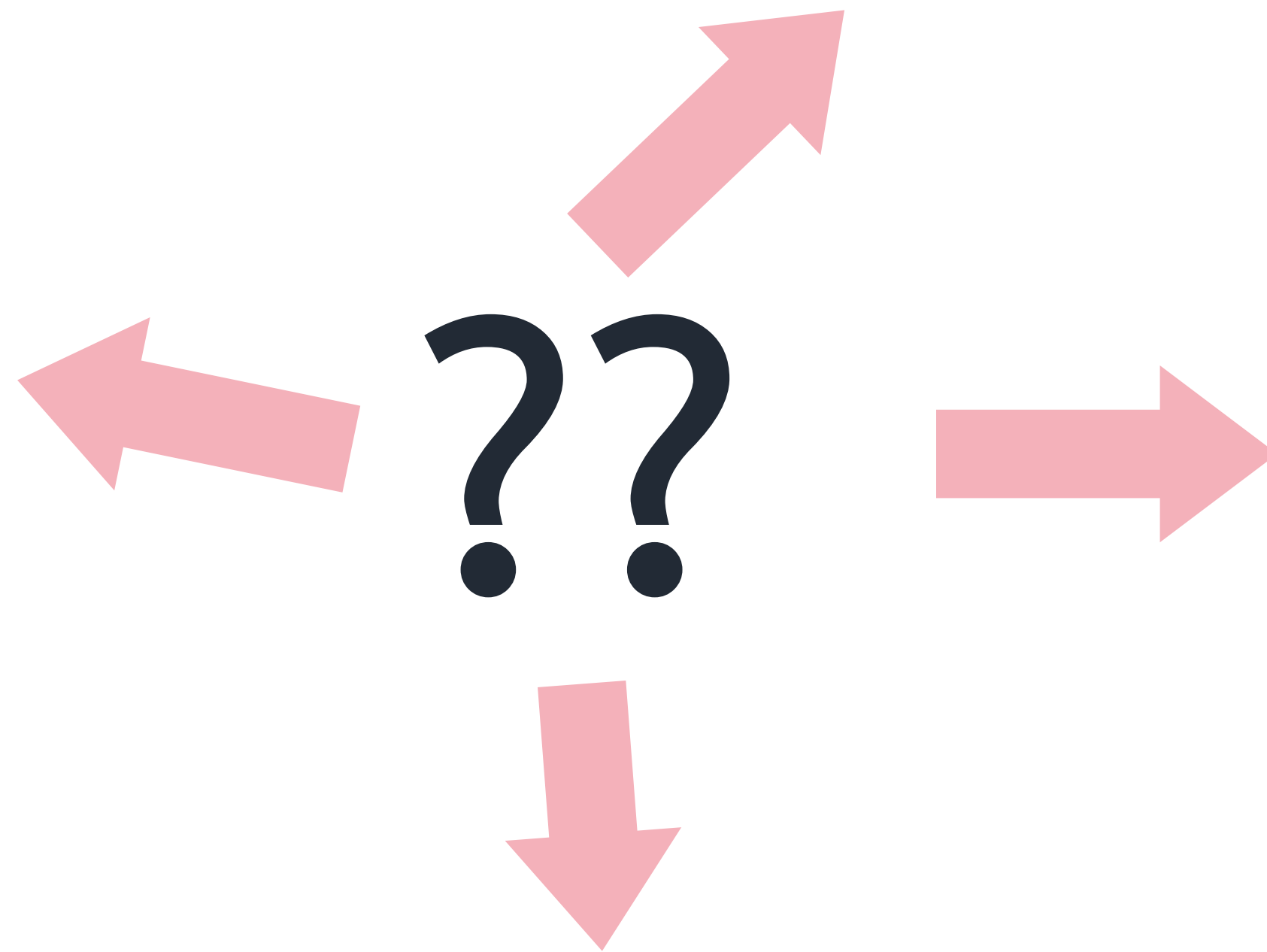
การเปลี่ยนแปลงของค่าถ่วงน้ำหนัก (Weight)
ที่ส่งผลต่อค่าความผิดพลาด (Error)

The diagram illustrates the weight update formula with the following components and annotations:

- Old weight**: An arrow points from this text to the W_x term in the formula.
- Derivative of Error with respect to weight**: An arrow points from this text to the $\left(\frac{\partial \text{Error}}{\partial W_x}\right)$ term in the formula.
- Learning rate**: An arrow points from this text to the α term in the formula.
- New weight**: An arrow points from this text to the $*W_x$ term in the formula.

$$*W_x = W_x - \alpha \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

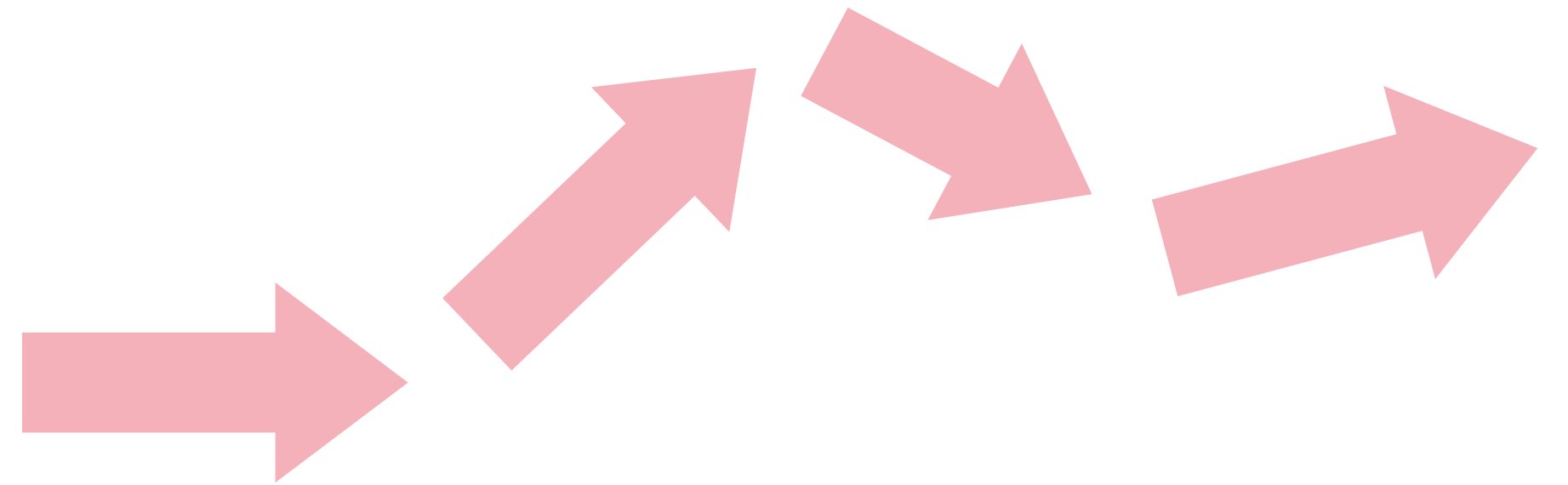
Game 1



Game 2

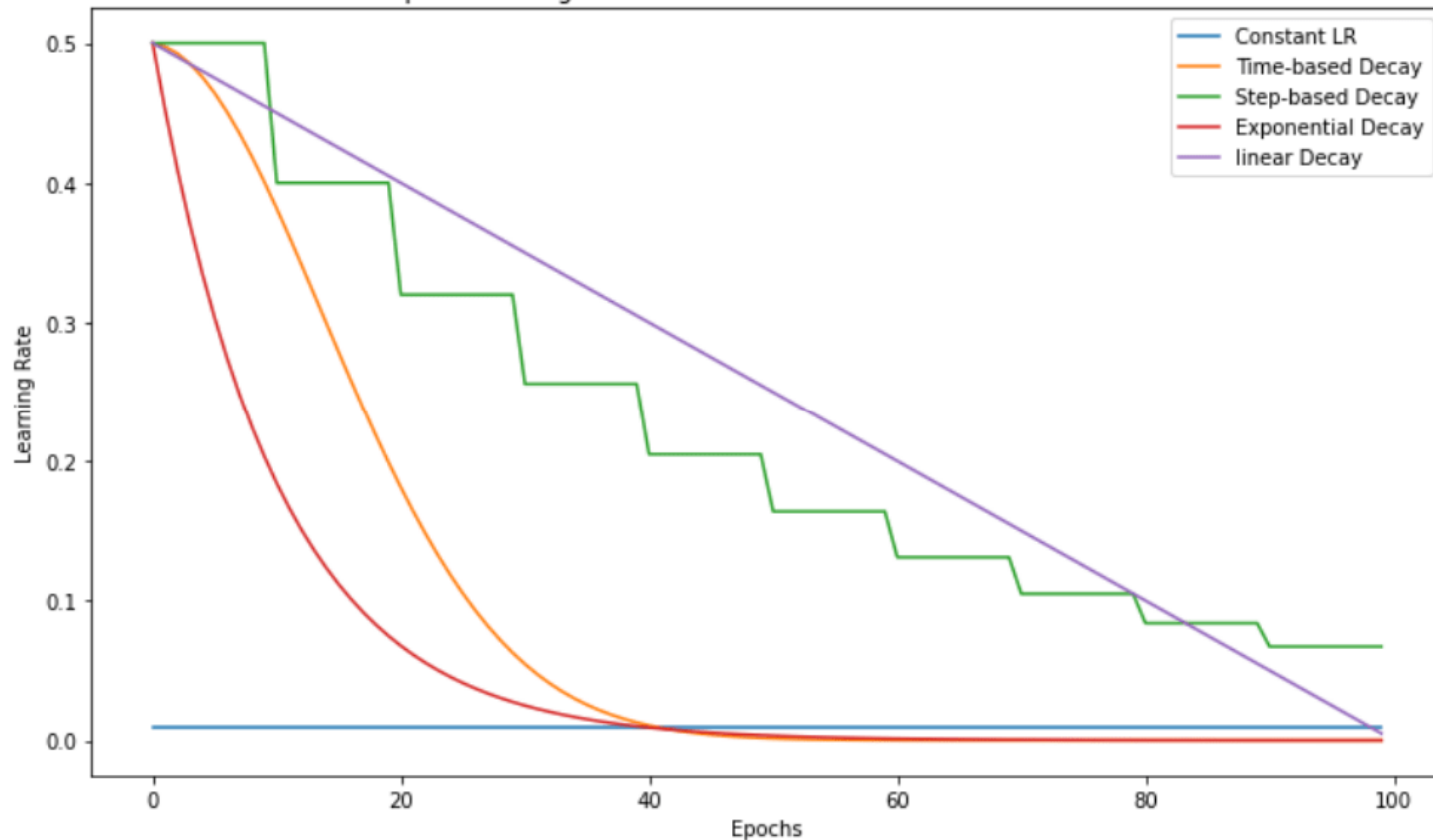


??



Learning Rate & Its Strategies

Compare Learning Rate Curves Generated from Different Schedulers

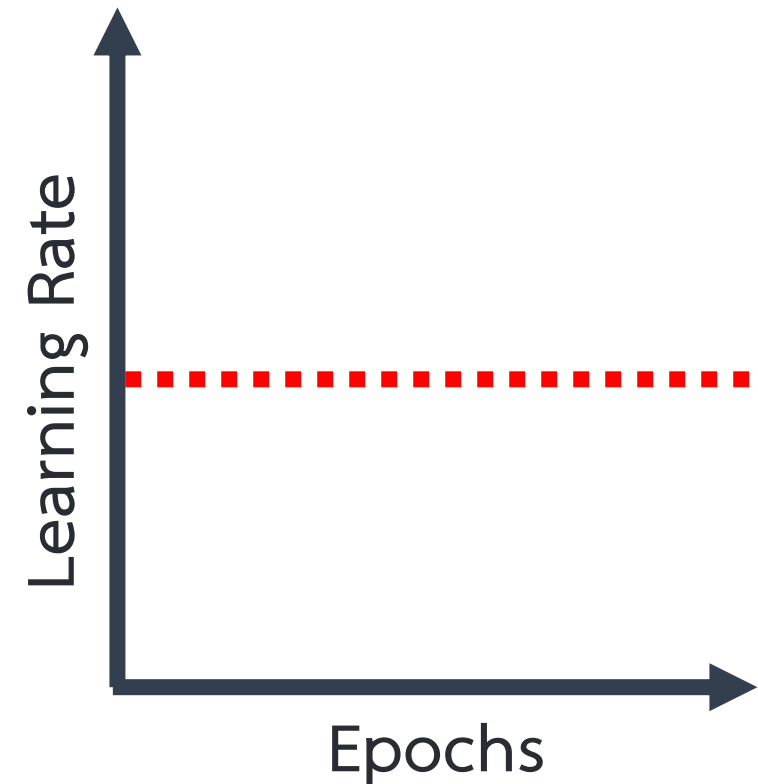


แนวทางในการปรับค่า Learning Rate

- Fixed Learning Rate
- Time-Based Decay
- Step Decay
- Exponential Decay
- Adaptive Learning Rate (Adagrad, RMSprop, Adam)
- Learning Rate Warm-up

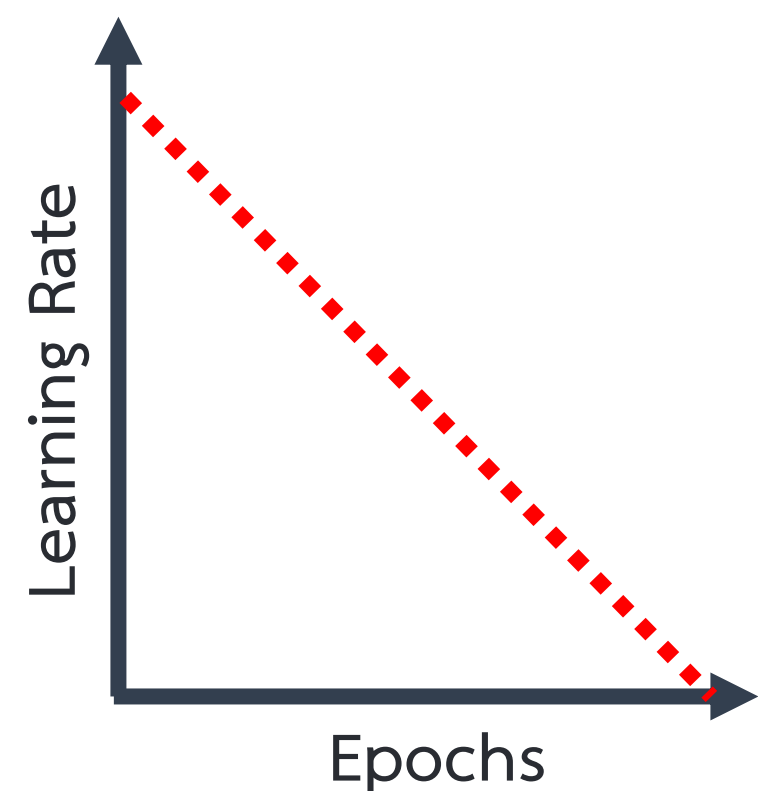
Learning Rate & Its Strategies

อัตราการเรียนรู้คงที่ (Fixed Learning Rate)



- เป็นแนวทางที่กำหนดให้ ค่า Learning Rate คงที่ตลอดการฝึก เช่น 0.01 และไม่เปลี่ยนแปลงตามรอบการฝึก (epoch)
- ทำให้ใช้งานง่าย ไม่ซับซ้อน และทำให้การฝึกมีเสถียรภาพในระดับหนึ่ง
- แต่ไม่ยืดหยุ่นต่อความต้องการของแต่ละช่วงการฝึก และอาจได้ผลลัพธ์ที่ไม่ดีที่สุด เนื่องจาก Learning Rate ที่เหมาะสมในช่วงต้นกับช่วงปลายของการฝึกอาจต่างกัน

ลดค่าอัตราการเรียนรู้ตามเวลา (Time-Based Decay)



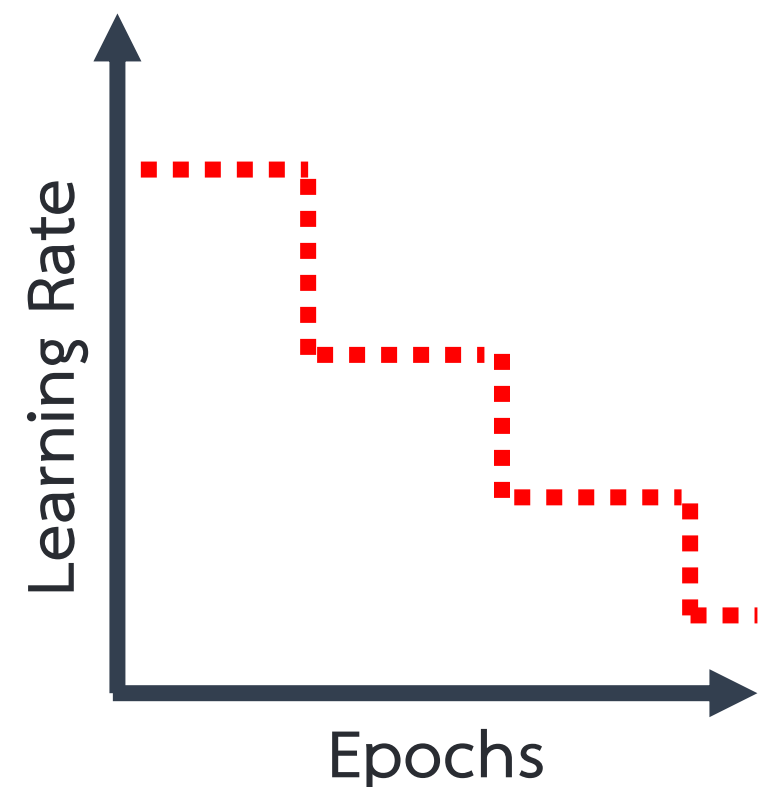
- เป็นแนวทางที่ปรับลดค่า Learning Rate (LR) ตามจำนวนรอบการฝึก (epoch) โดยทั่วไปใช้สูตรที่ลดค่า LR แบบผกผันกับเวลา เช่น

$$LR = LR_{init} \times \frac{1}{1 + k \cdot epoch}$$

- ทำให้โมเดลเรียนรู้เร็วในช่วงต้น (ก้าวใหญ่) แล้วค่อย ๆ ปรับละเอียดเมื่อเข้าใกล้ค่าที่เหมาะสม และปรับค่า Learning Rate ตามความคืบหน้าของการเรียนรู้
- แต่ต้องปรับพารามิเตอร์ decay rate (เช่นค่า k) ให้เหมาะสม ซึ่งหากตั้งค่าไม่ดี อาจลดเร็วหรือช้าเกินไป

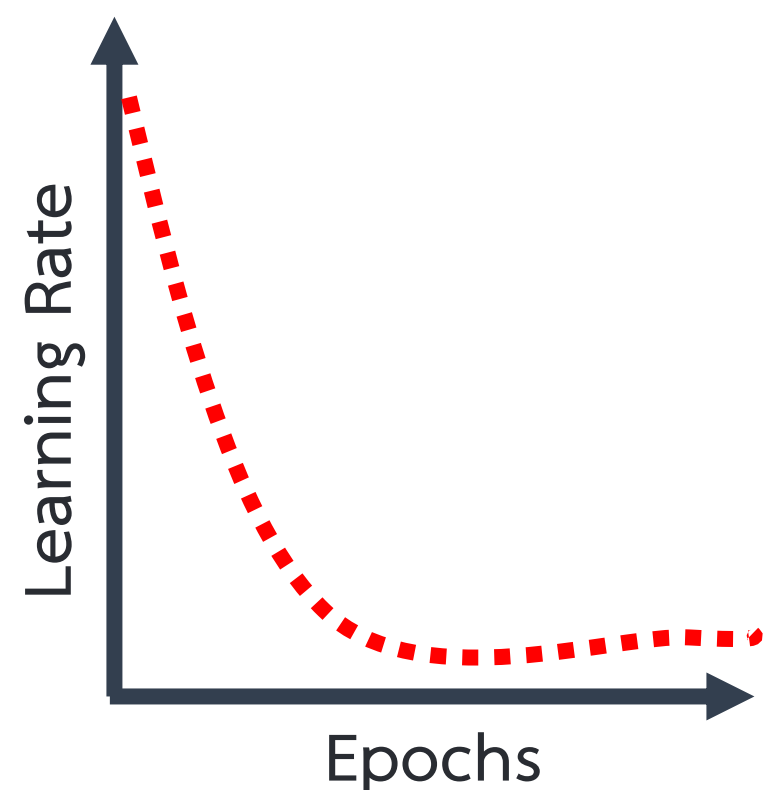
Learning Rate & Its Strategies

ลดค่าอัตราการเรียนรู้เป็นช่วง (Step Decay)



- เป็นแนวทางการลดค่า Learning Rate แบบก้าว (step) โดยกำหนดให้ลดลงทีละช่วงเวลา เช่น ลดลงครึ่งหนึ่งทุก ๆ 10 epochs เช่น เริ่มต้นที่ 0.1, หลัง 10 epochs → ลดเป็น 0.05, หลัง 20 epochs → ลดเป็น 0.025
- ใช้งานง่าย และเหมาะสมกับการผสมผสานการเรียนรู้เร็วในช่วงต้น และการปรับละเอียดในช่วงหลังได้ดี
- แต่ต้องกำหนด “ช่วงเวลา” ที่จะลด Learning Rate และ “อัตราการลด” ไว้ล่วงหน้า ทำให้อาจไม่ยืดหยุ่นหากโครงสร้างข้อมูลเปลี่ยน

ลดค่าอัตราการเรียนรู้แบบเอ็กซ์โพเนนเชียล (Exponential Decay)



- มีหลักการคล้ายกับ time-based decay แต่ลดลงในอัตราเอ็กซ์โพเนนเชียล ซึ่งลดเร็วกว่า time-based โดยอาศัยสูตร

$$LR = LR_{init} \times e^{-k \cdot epoch}$$

- เช่น ถ้า $k = 0.1$, $LR_{initial} = 0.01$ จะได้ Epoch 0: 0.01, Epoch 1: 0.00905, Epoch 10: 0.0037
- ทำให้โมเดลเข้าใกล้ค่าที่เหมาะสมได้เร็ว (เร็วกว่า time-based) จึงเหมาะกับกรณีที่ต้องการ convergence เร็ว
- แต่การลดที่เร็วเกินไปอาจทำให้โมเดลหยุดเรียนรู้ก่อน และต้องระวังไม่ให้ค่า LR ลดต่ำเกินไปจนเรียนรู้หยุดนิ่ง

Learning Rate & Its Strategies

การปรับอัตราการเรียนรู้แบบอัตโนมัติ (Adaptive Learning Rate)

เป็นการที่อัลกอริธึมปรับค่า Learning Rate โดยอัตโนมัติสำหรับแต่ละพารามิเตอร์ ตามค่ากราเดียนต์หรือสถิติที่เกิดขึ้นระหว่างการฝึก โดยไม่ต้องตั้งค่า LR คงที่หรือกำหนด decay แบบชัดเจน ตัวอย่างอัลกอริธึมยอดนิยม ได้แก่

อัลกอริธึม	รายละเอียด
• Stochastic Gradient Descent (SGD)	ใช้ learning rate คงที่
• Adaptive Gradient (AdaGrad)	ปรับ learning rate ตามค่าสะสมของสะสมกราเดียนต์ (ลดเร็วเกินไป ทำให้หยุดเรียนรู้เร็ว)
• Root Mean Square Propagation (RMSProp)	ปรับ learning rate ตามค่าเฉลี่ยของกราเดียนต์

Learning Rate & Its Strategies

Adaptive Gradient (AdaGrad) Optimizer

เป็นการปรับค่า Learning Rate แยกสำหรับแต่ละพารามิเตอร์ โดยพิจารณาจาก ค่ากราเดียนต์สะสม ที่เกิดขึ้นระหว่างการฝึก เพื่อให้พารามิเตอร์ที่มีการเปลี่ยนแปลงบ่อย (gradient สูง) ได้เรียนรู้น้อยลง และพารามิเตอร์ที่เปลี่ยนแปลงน้อยได้เรียนรู้มากขึ้น ซึ่งเหมาะสมกับข้อมูลที่ feature มีความไม่สมดุล

ขั้นตอนการทำงาน

Step 1: กำหนดค่าเริ่มต้น (Initialize Variables)

- กำหนดพารามิเตอร์ w_t เริ่มต้น ได้แก่ น้ำหนัก (weights) และ ไบแอส (biases)
- กำหนดค่าคงที่ขนาดเล็ก ϵ (เช่น $1e-8$) เพื่อป้องกันการหารด้วยศูนย์
- สร้างตัวแปร v_t ซึ่งเก็บผลรวมของค่ากราเดียนต์ยกกำลังสอง โดยเริ่มจากค่า 0

Step 2: คำนวณกราเดียนต์ (Calculate Gradients)

- คำนวณกราเดียนต์ ∇ ของฟังก์ชัน loss ที่มีต่อพารามิเตอร์แต่ละตัว

Step 3: สะสมค่ากราเดียนต์กำลังสอง (Accumulate Squared Gradients)

- สำหรับพารามิเตอร์แต่ละตัว โดยอาศัยสมการดังนี้

$$v_t = v_{t-1} + (\nabla w_t)^2$$

Step 4: ปรับค่าพารามิเตอร์ (Update Parameters)

- ใช้ learning rate ที่ปรับตามค่าของ โดยอาศัยสมการดังนี้

$$w_{t+1} = w_t - \frac{LR}{\sqrt{v_t} + \epsilon} \nabla w_t$$

Learning Rate & Its Strategies

Root Mean Square Propagation (RMSProp) Optimizer

เป็นการปรับค่าอัตราการเรียนรู้ที่ถูกออกแบบมาเพื่อเพิ่มประสิทธิภาพและความเร็วในการฝึกโมเดล Deep Learning โดย RMSProp เป็นการปรับปรุงจาก Gradient Descent โดยปรับค่า learning rate แยกสำหรับแต่ละพารามิเตอร์ และพิจารณา ค่าเฉลี่ยของกราฟาเดียนต์ เพื่อให้การอัปเดตมีความเหมาะสมมากขึ้นในแต่ละพารามิเตอร์

ขั้นตอนการทำงาน

Step 1: กำหนดค่าเริ่มต้น (Initialize Variables)

- กำหนดพารามิเตอร์ w_t เริ่มต้น ได้แก่ น้ำหนัก (weights) และ ไบแอส (biases)
- กำหนดค่าคงที่ขนาดเล็ก ϵ เพื่อป้องกันการหารด้วยศูนย์
- สร้างตัวแปร v_t ซึ่งเก็บผลรวมของค่ากราฟาเดียนต์ยกกำลังสอง โดยเริ่มจากค่า 0

Step 2: คำนวณกราฟาเดียนต์ (Calculate Gradients)

- คำนวณกราฟาเดียนต์ ∇ ของฟังก์ชัน loss ที่มีต่อพารามิเตอร์แต่ละตัว

Step 3: สะสมค่ากราฟาเดียนต์กำลังสอง (Accumulate Squared Gradients)

- สำหรับพารามิเตอร์แต่ละตัว โดยอาศัยสมการดังนี้

$$v_t = \underset{\text{(สะสม)}}{\gamma} \cdot v_{t-1} + \underset{\text{(ล่าสุด)}}{(1-\gamma)} \cdot (\nabla w_t)^2$$

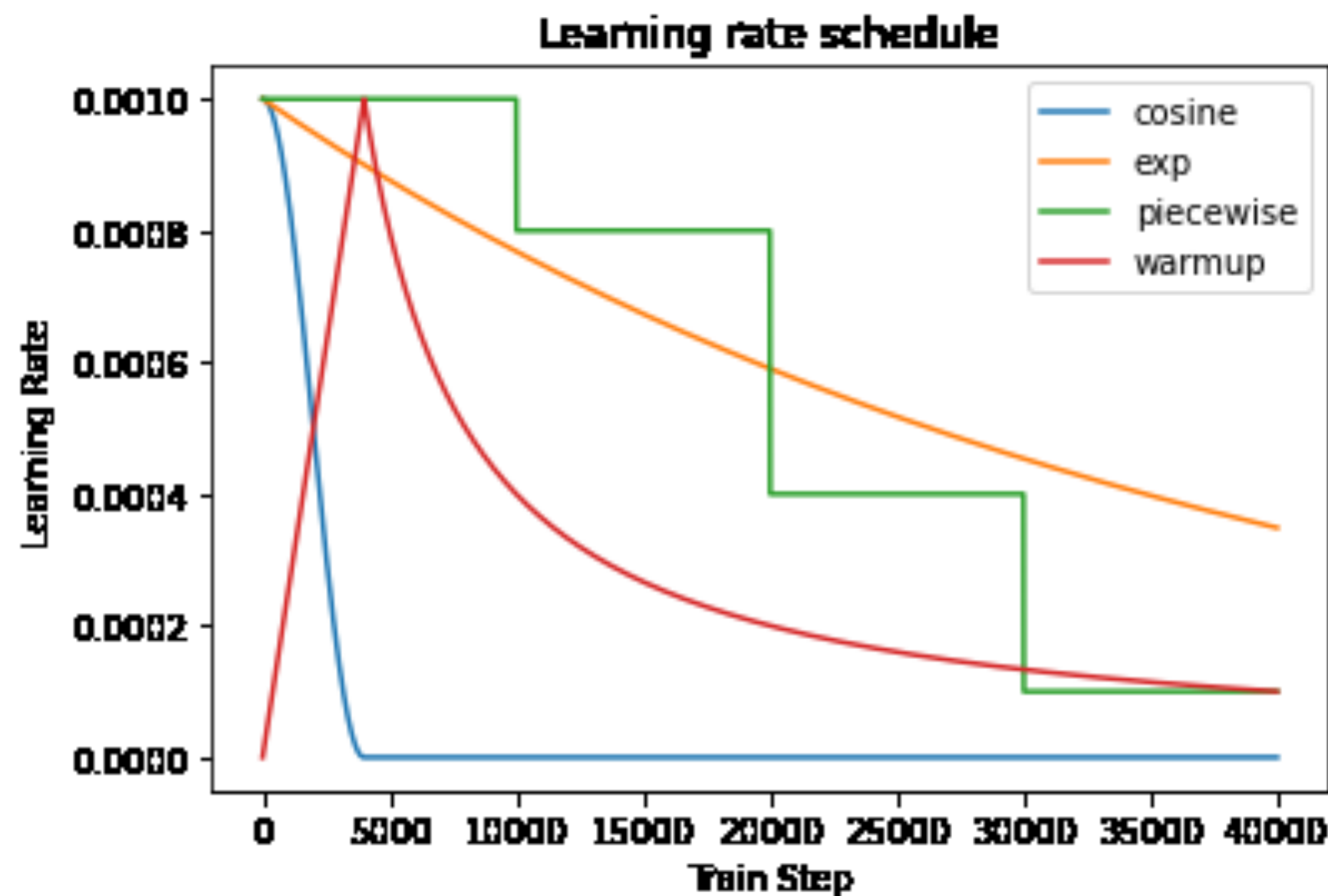
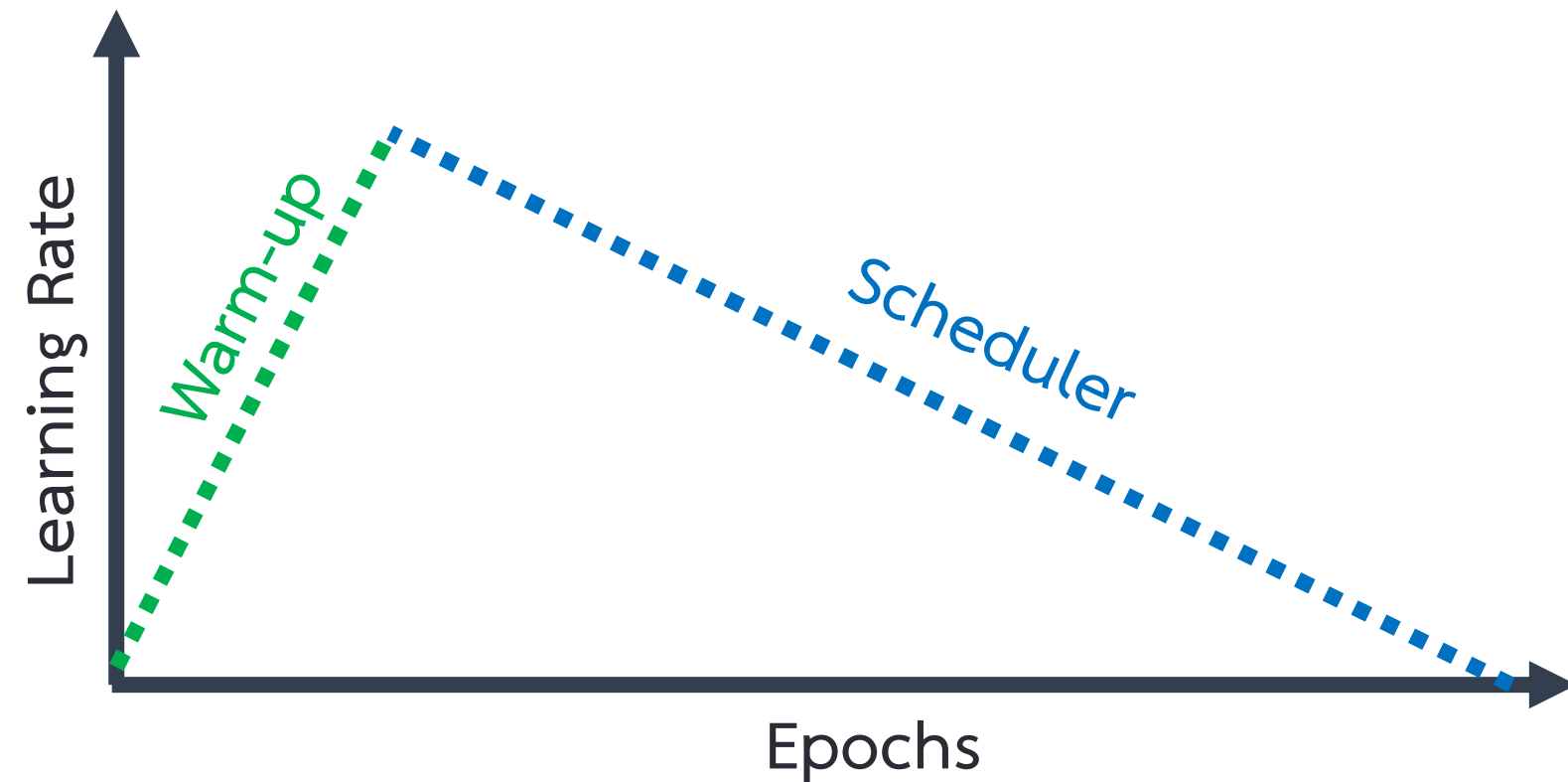
- γ คือ อัตราการคงอยู่ (decay rate) เช่น 0.9

Step 4: ปรับค่าพารามิเตอร์ (Update Parameters)

- ใช้ learning rate ที่ปรับตามค่าของ โดยอาศัยสมการดังนี้

$$w_{t+1} = w_t - \frac{LR}{\sqrt{v_t} + \epsilon} \nabla w_t$$

Learning Rate & Its Strategies



Learning Rate Warm-up Optimizer

คือ การเริ่มต้นฝึกโมเดลด้วยค่า learning rate ที่น้อย ๆ แล้วค่อย ๆ เพิ่มขึ้นไปสู่ค่าที่ตั้งใจ การใช้ งานเทคนิคนี้ช่วยให้โมเดลไม่ปรับพารามิเตอร์มากเกินไปในช่วงแรก เพื่อหลีกเลี่ยง exploding gradients และทำให้ optimizers อย่าง Adam, RMSProp, SGD มีเวลาสร้างค่าเฉลี่ยเคลื่อนที่ที่เสถียรก่อนเริ่มฝึกสอนโมเดล

Guideline สำหรับการใช้งาน Learning Rate Warm-up

- กำหนดค่า Learning Rate เริ่มต้นต่ำมาก ๆ อาทิเช่น 0 หรือ $1e-6$ เพื่อให้ช่วงต้นของการฝึกโมเดลมีความนิ่งและไม่ปรับพารามิเตอร์รุนแรง
- กำหนดช่วง Warm-up (warm-up steps หรือ warm-up epochs)

กรณีใช้งาน	แนวทางแนะนำ
ใช้กับ batch-level (steps)	กำหนดค่า warmup steps ประมาณ 500 - 1000 ที่มีการใช้งาน mini-batches
ใช้กับ epoch-level	กำหนดค่า warmup epochs ประมาณ 3 - 5 จากทั้งหมด 30 - 100 epochs

หมายเหตุ ช่วง Warm-up ไม่มีค่าตายตัว ต้องปรับตามขนาดข้อมูลและความลึกของโมเดล

Learning Rate & Its Strategies

❑ Exploding Gradients

คือ ปรากฏการณ์ที่ค่ากราเดียนต์ (gradients) ที่ได้จากการคำนวณย้อนกลับ (backpropagation) มีค่ากราเดียนต์มากกว่าปกติ จนทำให้การอัปเดตพารามิเตอร์ของโมเดลมากเกินไป ส่งผลให้ (1) ค่าพารามิเตอร์ (weights) เพิ่มขึ้นเร็วผิดปกติ และ (2) ค่าความสูญเสีย (loss) ไม่ลดลงหรือเพิ่มขึ้นทันที การเกิดเหตุการณ์ Exploding Gradients มักเกิดในโมเดลที่มีโครงสร้างลึกมาก (deep) หรือมีลักษณะลำดับต่อเนื่อง เช่น RNN, LSTM เป็นต้น

สัญญาณที่บ่งชี้ว่าเกิด Exploding Gradients

- ค่า loss เพิ่มขึ้นทันทีแบบไม่มีเหตุผล
- ค่า weights กลายเป็นค่า NaN หรือ infinity
- โมเดลไม่ converge หรือ converge ได้ชั่วคราวแล้วหลุด

❑ Vanishing Gradients

คือ ปัญหาที่เกิดขึ้นในโมเดลที่มีหลายชั้น (deep networks) หรือโมเดลที่ใช้กับข้อมูลลำดับ เช่น RNN, LSTM เป็นต้น ซึ่งค่ากราเดียนต์ (gradient) ที่ได้จากการทำ backpropagation มีค่ากราเดียนต์น้อยมากจนเกือบเป็นศูนย์ ส่งผลให้ (1) โมเดลไม่สามารถอัปเดตพารามิเตอร์ในชั้นต้น ๆ ได้ และ (2) การฝึกเป็นไปอย่างช้ามากหรือแทบไม่มีการเรียนรู้

สัญญาณที่พบเมื่อเกิด Vanishing Gradients

- Training loss ลดช้า หรือไม่ลดเลย
- Accuracy ไม่เพิ่ม
- Weights ของชั้นต้น ๆ ไม่เปลี่ยนแปลงเลย

Learning Rate & Its Strategies

ตารางเปรียบเทียบ Vanishing Gradients กับ Exploding Gradients

ด้าน	Vanishing Gradients	Exploding Gradients
ลักษณะ	ค่ากราเดียนต์ใกล้ <u>ศูนย์</u>	ค่ากราเดียนต์มีค่า <u>สูงมาก</u>
ส่งผล	โมเดลเรียนรู้ไม่ได้	Loss พุ่งสูง ทำให้โมเดลไม่เสถียร
แนวทางแก้	• ใช้ Activation Function ที่ลดการสูญหายของค่ากราเดียนต์หายไป อาทิเช่น ReLU • ใช้ Residual Connection • ใช้ Batch Normalization • ใช้ Weight Initialization ที่ดี (โหนด Pretrain)	• ใช้เทคนิค Gradient Clipping • ใช้ Optimizer แบบ Adaptive • ใช้ Learning Rate Warm-up • ใช้ Batch Normalization • ใช้ Weight Initialization ที่ดี (โหนด Pretrain)



ตัวอย่างการใช้งานเทคนิค Linear Warm-up & Linear Decay

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import math
```

โมเดล **CNN** แบบง่าย

```
class SimpleCNN(nn.Module):
    def __init__(self):
        ...
    def forward(self, x):
        ...
```

ข้อมูล **MNIST**

```
transform = transforms.ToTensor()
train_data = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
```

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, 3, 1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, 1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 5 * 5, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```




ตัวอย่างการใช้งานเทคนิค Linear Warm-up & Linear Decay

```
# พารามิเตอร์สำหรับ Warm-up และ Linear Decay
```

```
total_epochs = 20
```

```
warmup_epochs = 5
```

```
lr_start = 0.0
```

```
lr_max = 0.1
```

```
lr_end = 0.001
```

```
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
```

```
model = SimpleCNN().to(device)
```

```
optimizer = optim.Adam(model.parameters(), lr=lr_max)
```

```
criterion = nn.CrossEntropyLoss()
```

```
# สร้าง scheduler แบบ custom โดยที่
```

```
# Epoch 0- 4: 0.0 → 0.1 เพิ่มขึ้นแบบ linear warm-up
```

```
# Epoch 5-34: 0.1 → 0.001 ลดลงแบบ linear decay
```

```
def lr_lambda(current_epoch):
```

```
    if current_epoch < warmup_epochs:
```

```
        return lr_start + (lr_max - lr_start) * (current_epoch / warmup_epochs)
```

```
    else:
```

```
        decay_epochs = total_epochs - warmup_epochs
```

```
        decay_progress = (current_epoch - warmup_epochs) / decay_epochs
```

```
        return lr_max - (lr_max - lr_end) * decay_progress
```

```
scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda=lambda epoch: lr_lambda(epoch))
```



ตัวอย่างการใช้งานเทคนิค Linear Warm-up & Linear Decay

Training loop

```
for epoch in range(total_epochs):  
    model.train()  
    total_loss = 0.0  
    for images, labels in train_loader:  
        images, labels = images.to(device), labels.to(device)  
  
        optimizer.zero_grad()  
        outputs = model(images)  
        loss = criterion(outputs, labels)  
        loss.backward()  
        optimizer.step()  
  
        total_loss += loss.item()
```

ปรับ LR ตาม epoch

```
scheduler.step()
```

ดึงค่าปัจจุบันของ LR

```
current_lr = scheduler.get_last_lr()[0]  
print(f"Epoch {epoch+1:02d}/{total_epochs} | LR: {current_lr:.6f} | Loss: {total_loss:.4f}")
```

⇒ Epoch 1/20	LR: 0.002000	Loss: 2159.7222
Epoch 2/20	LR: 0.004000	Loss: 105.6232
Epoch 3/20	LR: 0.006000	Loss: 50.1183
Epoch 4/20	LR: 0.008000	Loss: 43.7669
Epoch 5/20	LR: 0.010000	Loss: 48.5584
Epoch 6/20	LR: 0.009340	Loss: 53.4687
Epoch 7/20	LR: 0.008680	Loss: 40.0740
Epoch 8/20	LR: 0.008020	Loss: 37.9518
Epoch 9/20	LR: 0.007360	Loss: 32.4723
Epoch 10/20	LR: 0.006700	Loss: 24.1552
Epoch 11/20	LR: 0.006040	Loss: 19.0926
Epoch 12/20	LR: 0.005380	Loss: 13.8010
Epoch 13/20	LR: 0.004720	Loss: 11.9995



ตัวอย่างการใช้งานเทคนิค RMSprop กับ StepLR

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

โมเดล **CNN** แบบง่าย

```
class SimpleCNN(nn.Module):
    def __init__(self):
        ...
    def forward(self, x):
        ...
```

ข้อมูล **MNIST**

```
transform = transforms.ToTensor()
train_data = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
```

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, 3, 1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, 1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 5 * 5, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```




ตัวอย่างการใช้งานเทคนิค RMSprop กับ StepLR

อุปกรณ์คำนวณ

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
model = SimpleCNN().to(device)
```

RMSProp Optimizer + StepLR Scheduler

```
optimizer = optim.RMSprop(model.parameters(), lr=0.01, alpha=0.9, eps=1e-08)
```

```
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=2, gamma=0.5)
```

ลด **LR** ลงครึ่งหนึ่งทุก **2 epochs**

scheduler จะลด **learning rate**

RMSprop จะไม่ลด **learning rate** อัตโนมัติ เพียงแค่ปรับ "น้ำหนักของการอัปเดต" โดยใช้ค่าเฉลี่ยของกราเดียนต์ แต่ค่า **LR** ที่ตั้งไว้ (**lr=0.01**) จะคงที่ตลอด

ยกเว้น คุณใช้ **scheduler** ร่วมด้วย

```
criterion = nn.CrossEntropyLoss()
```



ตัวอย่างการใช้งานเทคนิค RMSprop กับ StepLR

Training Loop

```
total_epochs = 20
for epoch in range(total_epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    running_loss += loss.item()
```

อัปเดต learning rate ทุก epoch *****

```
scheduler.step()
```

แสดงค่า learning rate ปัจจุบัน

```
current_lr = optimizer.param_groups[0]['lr']
print(f"Epoch {epoch+1:02d}/{total_epochs} | LR: {current_lr:.6f} | Loss: {running_loss:.4f}")
```

Epoch 01/20	LR: 0.010000	Loss: 226.3389
Epoch 02/20	LR: 0.005000	Loss: 70.2398
Epoch 03/20	LR: 0.005000	Loss: 36.2566
Epoch 04/20	LR: 0.002500	Loss: 33.4672
Epoch 05/20	LR: 0.002500	Loss: 19.1082
Epoch 06/20	LR: 0.001250	Loss: 16.0898
Epoch 07/20	LR: 0.001250	Loss: 9.6305
Epoch 08/20	LR: 0.000625	Loss: 8.6862
Epoch 09/20	LR: 0.000625	Loss: 6.1066
Epoch 10/20	LR: 0.000313	Loss: 5.6691
Epoch 11/20	LR: 0.000313	Loss: 4.6073
Epoch 12/20	LR: 0.000156	Loss: 4.2695
Epoch 13/20	LR: 0.000156	Loss: 3.8996
Epoch 14/20	LR: 0.000078	Loss: 3.7889



ตัวอย่างการใช้งานเทคนิค RMSprop

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

โมเดล **CNN** แบบง่าย

```
class SimpleCNN(nn.Module):
    def __init__(self):
        ...
    def forward(self, x):
        ...
```

ข้อมูล **MNIST**

```
transform = transforms.ToTensor()
train_data = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
```

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, 3, 1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, 1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 5 * 5, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```




ตัวอย่างการใช้งานเทคนิค RMSprop

อุปกรณ์

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
model = SimpleModel().to(device)
```

ใช้ **RMSProp**

```
optimizer = optim.RMSprop(model.parameters(), lr=0.01)
```

```
criterion = nn.CrossEntropyLoss()
```



ตัวอย่างการใช้งานเทคนิค RMSprop

Training Loop

```
total_epochs = 20
for epoch in range(total_epochs):
    model.train()
    total_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    total_loss += loss.item()
```

แสดงค่า learning rate และ loss

```
current_lr = optimizer.param_groups[0]['lr']
print(f"Epoch {epoch+1}: LR = {current_lr:.6f}, Loss = {total_loss:.4f}")
```

```
Epoch 1: LR = 0.010000, Loss = 283.8609
Epoch 2: LR = 0.010000, Loss = 281.8843
Epoch 3: LR = 0.010000, Loss = 281.9323
Epoch 4: LR = 0.010000, Loss = 278.7583
Epoch 5: LR = 0.010000, Loss = 278.6160
Epoch 6: LR = 0.010000, Loss = 276.5797
Epoch 7: LR = 0.010000, Loss = 276.7791
Epoch 8: LR = 0.010000, Loss = 276.4267
Epoch 9: LR = 0.010000, Loss = 274.7337
Epoch 10: LR = 0.010000, Loss = 274.5698
Epoch 11: LR = 0.010000, Loss = 274.4849
Epoch 12: LR = 0.010000, Loss = 273.8048
Epoch 13: LR = 0.010000, Loss = 272.7538
Epoch 14: LR = 0.010000, Loss = 273.8438
Epoch 15: LR = 0.010000, Loss = 273.1363
Epoch 16: LR = 0.010000, Loss = 273.5064
Epoch 17: LR = 0.010000, Loss = 273.6075
Epoch 18: LR = 0.010000, Loss = 274.0791
Epoch 19: LR = 0.010000, Loss = 271.6588
Epoch 20: LR = 0.010000, Loss = 270.0258
```

Learning Rate & Its Strategies

ตารางแสดงการเปรียบเทียบ Scheduler ทางเลือกอื่นที่ลด Learning Rate ให้อัตโนมัติ

คำสั่ง Scheduler	ลักษณะการปรับเปลี่ยนค่า Learning Rate
StepLR	ปรับค่า Learning Rate ลดแบบขั้นบันได (ทุก N epoch)
ExponentialLR	ปรับค่า Learning Rate ลดลงแบบ Exponential หรือยิ่งลดยิ่งเร็ว
CosineAnnealingLR	ปรับค่า Learning Rate ลดแบบ Cosine หรือแบบโค้ง ซึ่งเหมาะกับ fine-tuning
ReduceLROnPlateau	ปรับค่า Learning Rate ลด เมื่อ loss ไม่ลดลง
LambdaLR	ปรับค่า Learning Rate ด้วยฟังก์ชันที่เรากำหนดเอง

สามารถดูรายละเอียดของ Scheduler แต่ละตัว รวมถึง Scheduler อื่น ๆ ได้ที่เอกสารประกอบของ PyTorch ตามลิงก์นี้
<https://pytorch.org/docs/stable/optim.html>



ตัวอย่างโปรแกรมโครงข่ายประสาทเทียม

```
import numpy as np
import pandas as pd
import torch

# load the training dataset and convert it into torch format
train = pd.read_csv('C:/Users/Bank/Desktop/py/week9/data1.csv')
train.head()

featureTr = np.array([train['x1'], train['x2']]).T
labelTr = np.array([[1, 0, 0], [1, 0, 0], [1, 0, 0], [1, 0, 0], [1, 0, 0], [1, 0, 0], [0, 1, 0], [0, 1, 0], [0, 1, 0], [0, 1, 0],
[0, 1, 0], [0, 1, 0], [0, 1, 0], [0, 0, 1], [0, 0, 1], [0, 0, 1], [0, 0, 1], [0, 0, 1], [0, 0, 1]])

featureTr = torch.from_numpy(featureTr).to(torch.float32)
labelTr = labelTr.astype(int);
labelTr = torch.from_numpy(labelTr).to(torch.float32)

# Use the nn package to define our model.
model = torch.nn.Sequential(
    torch.nn.Linear(2, 3),
    torch.nn.Linear(3, 3)
)

# Define the loss function.
loss_fn = torch.nn.MSELoss(reduction='sum')
```



ตัวอย่างโปรแกรมโครงข่ายประสาทเทียม

```
# Define the learning rate.
learning_rate = 1e-3

# Define the optimizer
optimizer = torch.optim.RMSprop(model.parameters(), lr=learning_rate)

for t in range(2000):
    # Forward pass: compute predicted y by passing x to the model.
    y_pred = model(featureTr)

    # Compute and print loss.
    loss = loss_fn(y_pred, labelTr)

    # Clear buffers
    optimizer.zero_grad()

    # Backward pass: compute gradient of the loss with respect to model
    loss.backward()

    # Calling the step function on an Optimizer makes an update to its
    optimizer.step()
```



ตัวอย่างโปรแกรมโครงข่ายประสาทเทียม

```
# converting testing data into torch format
featureTs = np.array([[13,6]])
featureTs = torch.from_numpy(featureTs).to(torch.float32)

# generating predictions for test set
with torch.no_grad():
    output = model(featureTs)

softmax = torch.exp(output).cpu()
prob = list(softmax.numpy())
predictions = np.argmax(prob, axis=1)

print('output is ', predictions, ' \n Prob : ', output)
```